Deployment-Oriented Neural Network State-Estimation Methods for Battery Management Systems:

Ageing Estimation, Robustness Benchmarking, Embedded Optimization, and Hardware Integration

submitted by
M. Sc.
Florian Rzepka

Chair of Electrical Energy Storage Technology
Institute of Energy and Automation Technology
Faculty IV, Technische Universität Berlin
in partial fulfillment of the requirements for the academic degree of

Doctor of Engineering
Dr.-Ing.

Dissertation

Doctoral Committee:

Chair: tbd
Reviewer: tbd
Reviewer: tbd

Date of the scientific defense: tbd

Berlin, 2026

Preface

This dissertation took shape during my time at the Chair of Electrical Energy Storage Technology at Technische Universität Berlin. Its questions emerged gradually from the research environment there and from my own interest in how artificial intelligence can support battery-state estimation and battery management under real operating conditions rather than in idealized settings. This direction was reinforced by my involvement in the MG-FARM project, an international research collaboration on smart, stand-alone micro-grids for the electrification of remote agricultural regions. Working alongside university and industry partners in Morocco, France, and Algeria, I was able to see at first hand how decentralized energy systems and their battery storage are deployed and operated in places where grid access, connectivity, and maintenance are limited. Taken together, these influences shaped the central question of this thesis: how battery-state estimation can be made not only accurate, but also robust against imperfect measurements and executable on the modest embedded hardware that such systems actually rely on.

I am grateful for the support I received in many forms throughout this work. My sincere thanks go above all to my supervisor, Prof. Dr.-Ing. Julia Kowal, and to the Chair of Electrical Energy Storage Technology at Technische Universität Berlin, for the guidance and the freedom to pursue this topic in my own way. I thank my colleagues at the chair for countless discussions and for a productive working atmosphere, and the partners of the MG-FARM consortium across Morocco, France, and Algeria for their experience and a genuinely international perspective on decentralized energy. I also thank my family and friends for their steady support throughout this time. I am especially grateful to my partner Hannah, whose positive spirit and steady encouragement helped me regain confidence and continue through the difficult phases of this work.

Abstract

Reliable estimates of state of charge (SOC) and state of health (SOH) are central to the safe and efficient operation of lithium-ion battery systems. At the same time, many neural battery estimators are still evaluated mainly as offline models under clean laboratory signals, whereas practical battery management systems (BMS) impose additional requirements through disturbed measurements, limited embedded resources, firmware constraints, and the physical measurement chain. This dissertation therefore develops a deployment-oriented perspective on artificial intelligence for BMS applications. Predictive accuracy is treated as one requirement among others, and is assessed together with robustness, embedded feasibility, and hardware integration.

The work is organized around four connected studies grounded in NMC and LFP ageing datasets and a custom laboratory BMS platform. First, a neural ageing-estimation study shows that a compact multilayer perceptron can estimate SOH with competitive accuracy when cumulative-current information and lag-sequence history are used to represent ageing-relevant temporal context. Second, a measurement-only robustness benchmark compares direct-measurement, hybrid, equivalent-circuit, and data-driven estimator classes under current bias, noise, missing samples, irregular sampling, burst dropout, voltage spikes, ADC quantization, and initialization mismatch. The benchmark demonstrates that clean-condition MAE alone is not sufficient for practical model selection: estimators that are accurate under nominal signals can fail under bias or disturbed sampling, while other classes recover faster or remain more stable under specific faults. Third, an embedded-AI study transfers stateful recurrent SOC and SOH estimators to an STM32 microcontroller through structured pruning and INT8 weight quantization, measuring not only accuracy but also flash, RAM, latency, and energy. Fourth, the custom BMS platform provides the hardware and firmware context in which estimator exchangeability, sensing, and embedded validation can be investigated

reproducibly.

The central conclusion is that AI-enabled battery-state estimation should not be judged by a single accuracy number or by one nominal test. The goal is not to identify one universally best estimator, but to understand how different estimator classes behave across the requirement dimensions relevant to practical BMS deployment. For this purpose, accuracy, disturbance robustness, recovery behaviour, compressibility, runtime cost, and hardware integration must be evaluated together.

Table of Contents

Abbreviations

2RC	Two-resistor-capacitor equivalent circuit model
ADC	Analog-to-digital converter
AFE	Analog front end
AI	Artificial Intelligence
BMS	Battery Management System
CC-CV	Constant-current constant-voltage charging
CV	Constant voltage
DD	Data-driven estimator
DM	Direct-measurement estimator
DoD	Depth of discharge
DWT	Data Watchpoint and Trace
ECM	Equivalent Circuit Model
EFC	Equivalent Full Cycles
EMI	Electromagnetic interference
FP16	16-bit floating-point format
FP32	32-bit floating-point format
GELU	Gaussian error linear unit
GPU	Graphics processing unit

GRU	Gated recurrent unit
HDM	Hybrid direct-measurement model
HECM	Hybrid equivalent-circuit model
HPPC	Hybrid pulse-power characterization
IC	Integrated circuit
INT8	8-bit signed integer representation
KPI	Key performance indicator
LFP	Lithium iron phosphate
LSTM	Long Short-Term Memory
MAE	Mean absolute error
MCU	Microcontroller Unit
MG-FARM	Smart micro-grid research project for remote agricultural regions
MLP	Multilayer Perceptron
MSE	Mean squared error
NMC	Lithium nickel manganese cobalt oxide
NPU	Neural processing unit
NTC	Negative temperature coefficient thermistor
OCV	Open-circuit voltage
ONNX	Open Neural Network Exchange
P95	95th percentile
PV	Photovoltaic
RAM	Random-access memory
RC	Resistor-capacitor

ReLU	Rectified linear unit
RGB	Red-green-blue color representation
RMSE	Root mean squared error
SRAM	Static random-access memory
STM32	STMicroelectronics 32-bit microcontroller family
STM32H7	STM32 high-performance microcontroller family based on Arm Cortex-M7
STM32H753ZI	STM32H7 microcontroller target used for the embedded benchmark
STM32N6	STM32 microcontroller family with integrated neural accelerator
SOC	State of Charge
SOC-GRU	SOC estimator based on a GRU recurrent core
SOH	State of Health
SOH-LSTM	SOH estimator based on an LSTM recurrent core
TU	Technische Universität
UART	Universal asynchronous receiver-transmitter
`.bss`	Linker section for zero-initialized or uninitialized static and global data
`.data`	Linker section for initialized static and global data copied from flash to RAM

Symbols

α	Smoothing factor of the causal SOH post-filter
ΔI	Current quantization step
ΔMAE	Difference in mean absolute error relative to the baseline case
Δt	Sampling interval
ΔT	Temperature quantization step
ΔU	Voltage quantization step
ε	Small numerical constant used in normalization
η	Coulombic efficiency factor in the equivalent-circuit observer
σ_I	Standard deviation of injected current noise
σ_T	Standard deviation of injected temperature noise
σ_U	Standard deviation of injected voltage noise
τ_1, τ_2	Time constants of the two RC branches
$\mathbf{A}$	State-transition matrix of the equivalent-circuit observer
$\mathbf{b}$	Input vector of the equivalent-circuit observer
$\mathbf{c}_t$	LSTM cell state at sample t
$\mathbf{f}_t$	LSTM forget-gate activation
$\mathbf{g}_t$	LSTM candidate-state activation
$\mathbf{h}_t$	LSTM hidden state at sample t

$\mathbf{i}_t$	LSTM input-gate activation
$\mathbf{o}_t$	LSTM output-gate activation
$\mathbf{x}$	State vector or model input vector, depending on context
$\mathbf{z}_t$	Concatenated LSTM input vector including current features and previous hidden state
b	Neural-network bias vector
$C(t)$	Capacity measured at time t
C_b	SOH-scaled capacity used by the equivalent-circuit observer
C_{app}	Apparent capacity used in SOC reference reconstruction
C_{chg}	Charge C-rate in the LFP ageing design
C_{dis}	Discharge C-rate in the LFP ageing design
C_{eff}	Effective capacity obtained from nominal capacity and estimated SOH
C_k	Measured discharge capacity during check-up k
C_{N}	Nominal capacity of the NMC cell
C_{nom}	Nominal cell capacity
E	Energy per inference used in the embedded utility score
F	Flash-memory footprint used in the embedded utility score
H	Recurrent hidden-state size
I	Current
I_k	Current sample at discrete time index k
$\lvert I\rvert_{\max}$	Maximum absolute current magnitude used to scale current-bias scenarios
k	Discrete time index

L_{SOC}	SOC training sequence length
m_k	Median of feature channel k used for robust normalization
n	Number of evaluated samples
P	Covariance matrix of the observer correction step
q	INT8 quantized weight code
Q	Cumulative charge or charge throughput
Q_c	Online cumulative charge state
r_k	Interquartile range of feature channel k used for robust normalization
R	Total RAM usage in the embedded utility score
R_0, R_1, R_2	Ohmic and polarization resistances of the equivalent-circuit model
R^2	Coefficient of determination
s	Quantization scale factor
s_h	Structured-pruning importance score of hidden channel h
SOC	State of charge
SOH	State of health
t	Time
T	Temperature
U	Voltage or composite utility score, depending on context
U_1, U_2	Polarization voltages of the two RC branches
W	Neural-network weight matrix
w	Individual neural-network weight
$\hat{w}$	Reconstructed floating-point weight after INT8 quantization
$x_{t,k}$	Feature value of channel k at sample t

y_i	Measured target value of sample i
$\hat{y}_i$	Predicted target value of sample i
$\mathrm{d}I/\mathrm{d}t$	Current derivative
$\mathrm{d}U/\mathrm{d}t$	Voltage derivative
DoD	Depth of discharge
EFC	Equivalent full cycles
MAE	Mean absolute error
MSE	Mean squared error
OCV	Open-circuit voltage
P95	95th percentile of the absolute error
RMSE	Root mean squared error

List of Figures

List of Tables

Chapter 1

Introduction and Motivation

Lithium-ion batteries are increasingly used in systems that must operate reliably for long periods with limited maintenance access. This includes stationary storage in decentralized energy systems, for example remote microgrids in regions such as North Africa where grid connection can be limited or unavailable. The same requirement appears in mobile and autonomous applications, including field robots, satellites, planetary rovers, and other platforms that have to continue operating even when communication, remote control, or service access is restricted. In all of these cases, the BMS needs reliable information about internal battery states that cannot be measured directly. SOC affects usable energy and power limits. SOH affects lifetime assessment, maintenance planning, and safe operating margins.

AI-based estimation methods are attractive because battery behaviour is nonlinear and depends on ageing, load history, temperature, and operating point. The problem is not simply to obtain a low error on a clean laboratory test set. A method that is useful in a BMS must also handle imperfect measurements, causal online execution, memory and timing limits, and the practical route from trained model to firmware. This dissertation therefore asks how battery-state estimation can be developed and evaluated as a deployment problem rather than only as an offline modelling task.

1.1 Research Aim and Questions

The aim of the dissertation is to develop and evaluate AI-based battery-state estimation methods along the path from data representation to embedded execution. The detailed requirement framework used for this evaluation is introduced in Chapter 2. At the level of the introduction, the work is guided by three research questions:

1. How can simple, low-complexity models capture the complex and time-dependent

ageing behaviour of lithium-ion batteries without resorting to large or elaborate architectures?

2. How do battery-state estimators behave under realistic operating and measurement conditions, and which factors determine their robustness in practice?

3. How can such estimators be transferred to embedded BMS hardware and optimized to run within its memory, latency, and energy limits?

Each question targets a different stage of the development path. The first is a question of model design: ageing develops slowly and depends on the entire load history, so the challenge is to make a small network sensitive to this history instead of adding architectural complexity. The second is a question of evaluation: an estimator that performs well on clean data must be confronted with the disturbed and incomplete signals of a real measurement chain before its true reliability can be judged, for example through controlled sensor bias, noise, missing samples, and timing irregularities. The third is a question of realization: a model is only useful in a BMS if it can be executed, and remain accurate, within the resource and timing budget of the target controller. The concrete methods that address these questions, together with the scope within which they are answered, are described in the following section.

1.2 Contribution and Scope

The dissertation makes three main technical contributions. First, it studies how lag-sequence construction and cumulative-current history can turn a memoryless MLP into a time-aware SOH estimator without making the network architecture itself more complex. Second, it develops a measurement-only robustness benchmark that compares direct-measurement, hybrid model-based, and data-driven SOC-estimator families under one common disturbance protocol. Third, it transfers recurrent SOC and SOH models to STM32 microcontroller execution and evaluates the interaction between model architecture, compression, and measured embedded performance.

These contributions are supported by an experimental basis consisting of two ageing datasets and a custom BMS platform. The hardware platform is not treated as a separate algorithmic contribution in the same sense as the three estimator studies. Instead, it provides the measurement, firmware, and validation environment required to make the deployment claims reproducible.

1.3 Structure of the Dissertation and Development Path

The dissertation is organized so that each early chapter has a distinct role. Chapter 2 defines the BMS requirement framework used throughout the work. Chapter 3 summarizes the relevant state of the art and identifies the remaining gaps. Chapter 4 describes the datasets, embedded targets, and Custom BMS Platform that form the experimental basis. Chapters 5 to 7 contain the three contribution studies: ageing estimation, robustness benchmarking, and embedded AI deployment. Chapter 8 synthesizes the findings across the studies, and Chapter 9 concludes the dissertation.

Several contribution chapters build on previously published or manuscript-based work. In particular, the ageing-estimation study in Chapter 5 is adapted from the published neural-network architecture paper on SOH estimation for stationary storage systems.[1]

Table 1.1: Cumulative structure of the dissertation and function of the core contribution chapters within the overall argument.

Chapter	Role	Primary dimension	Function within the overall argument
4	Experimental and hardware basis	System context and validation basis	Establishes the shared datasets, embedded targets, and Custom BMS Platform used by the contribution studies.
5	Ageing estimation for stationary storage systems	Model design and SOH estimation	Shows how cumulative-current and lag-sequence information can represent ageing history for a compact MLP-based estimator.
6	Robustness benchmarking of embedded battery state estimation	Disturbed-operation performance	Compares estimator families under one measurement-only disturbance protocol and shows why clean-condition error alone is insufficient.
7	Embedded AI for SOC and SOH estimation on microcontrollers	Embedded deployment and compression	Transfers recurrent estimators to STM32 execution and quantifies accuracy, memory, latency, energy, pruning, and quantization effects.

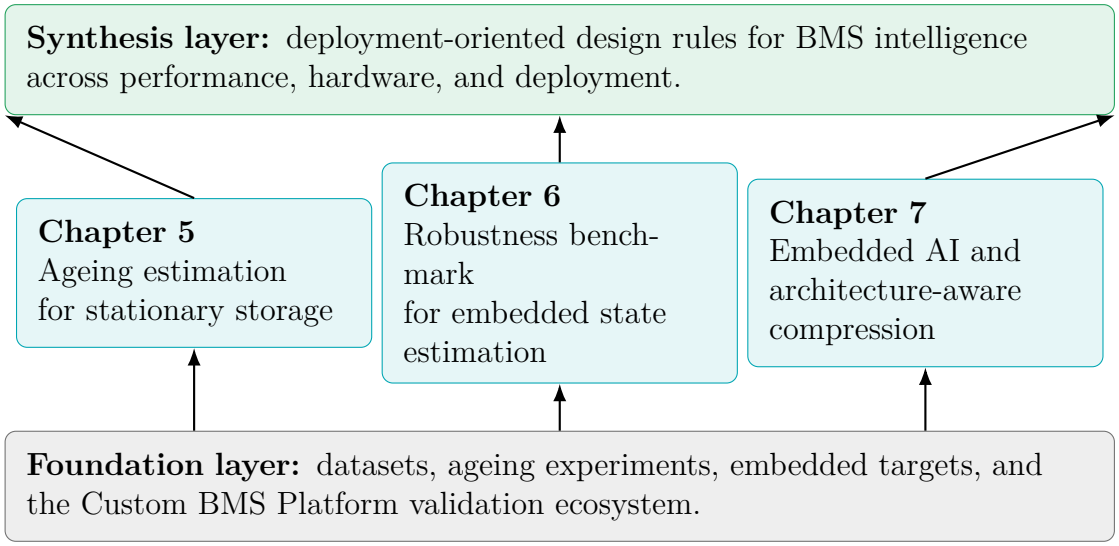

Figure 1.1: Roadmap of the overall work. The study chapters are embedded in a shared technical basis and are connected through a final synthesis toward deployment-oriented BMS intelligence.

Chapter 2

Battery-Management Requirement Framework

A battery-management system supervises a lithium-ion cell or pack and keeps it within safe and efficient operating limits by monitoring voltage, current, and temperature and by controlling charging, balancing, protection, and communication. SOC and SOH are central internal quantities in this control chain. They influence usable energy, power limitation, lifetime-aware operation, and maintenance decisions, but they cannot be measured directly from one sensor channel. They must be inferred from the available measurement stream and from knowledge about the cell or pack.[2, 3, 4]

For a BMS, state estimation is therefore not only an algorithmic accuracy problem. The estimator output has to be reliable enough for operating decisions, the implementation has to fit the controller and sensing chain, and the method has to be transferable into maintainable firmware. Figure 2.1 organizes these constraints into three requirement dimensions. The performance dimension concerns the quality of the estimated state signal. The hardware dimension defines the feasible model size and update rate through memory, computation time, and measurement constraints. The deployment dimension describes the engineering effort required to integrate, adapt, and reuse a method in a BMS environment. Direct-measurement, model-based, hybrid, and data-driven estimators are therefore assessed against the same BMS-related requirements, but they differ in how they trade accuracy, robustness, hardware effort, and deployment complexity.

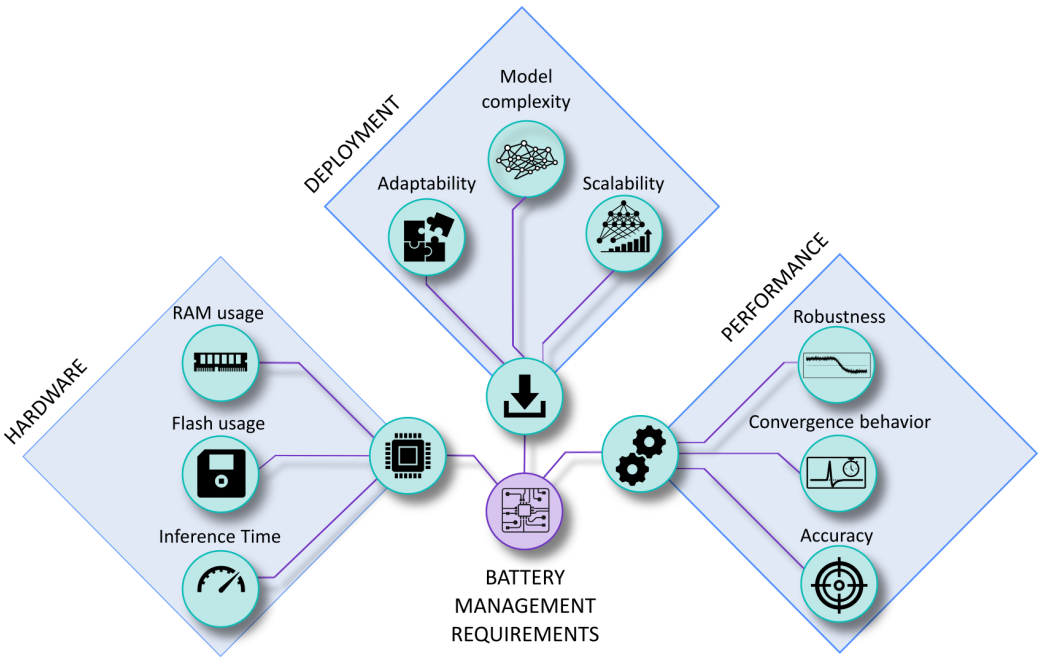

Figure 2.1: Battery-management requirement framework used throughout the dissertation. The BMS defines three connected requirement dimensions for state-estimation methods: performance, hardware, and deployment.

2.1 Performance Requirements

Performance requirements describe whether the estimated SOC or SOH signal is suitable for BMS decisions during operation. In this dissertation, performance is not reduced to average error under clean measurements. It also includes robustness against disturbed input signals and convergence or recovery after an inconsistent initial state. This distinction is important because bias, noise, missing samples, timing irregularities, and communication interruptions can change the estimator ranking even when the clean-data error is low.[5, 6, 7]

2.2 Hardware Requirements

Hardware requirements define the resource envelope in which state estimation can run on the BMS controller. Flash memory limits the amount of code, parameters, and lookup data that can be stored. RAM usage limits the size of recurrent states, intermediate buffers, and runtime data. Inference time determines whether the

estimator can meet the required update rate with sufficient timing margin, and energy per inference becomes relevant when the controller has to operate under strict power budgets.[8, 9] The measurement chain belongs to the same requirement area because sensor resolution, noise, sampling timing, and communication behaviour define the input information available to the estimator.

2.3 Deployment Requirements

Deployment requirements describe the engineering path from a trained or identified method to a reusable BMS implementation. The relevant aspects are model complexity, firmware transfer, calibration effort, adaptability to changed cells or operating conditions, and scalability to related estimation tasks or target platforms. A method is therefore not deployment-ready only because it runs once on a controller. It must also be reproducible, maintainable, and modifiable without rebuilding the full BMS software around it. Compression methods such as pruning and quantization can support deployment, but their effect depends on the architecture and must be verified on the target implementation rather than assumed from the training environment.[10, 11]

2.4 Coverage of the Framework in the Dissertation

The framework is used as an organizing tool, not as a second summary of the dissertation. Chapter 4 supplies the experimental and hardware context needed to make the studies reproducible. Chapter 5 mainly addresses performance and representation for SOH estimation. Chapter 6 focuses on performance under disturbed measurements and recovery conditions. Chapter 7 connects deployment and hardware by measuring how compressed recurrent models behave on the STM32 target. The final synthesis returns to the same three requirement areas to compare what was learned across the studies.

Chapter 3

State of the Art and Technical Background

3.1 Battery Management Systems and State Estimation

Practical BMS implementations are not only estimation algorithms. Commercial and research systems usually combine a cell-monitoring front end, current and temperature sensing, a microcontroller, isolation and communication interfaces, protection logic, and balancing circuitry. A useful distinction is therefore between the battery-monitor or analogue-front-end part and the controller part. Battery-monitor ICs such as TI's BQ79616 provide cell-voltage acquisition, balancing control, and hardware protection functions, while BQ769x2-based reference designs illustrate how monitoring, balancing, current measurement, and pack protection are combined in practical battery-pack electronics.[12, 13, 14] Current sensors and temperature channels then define the measurement quality available to the estimator.[15, 16] The microcontroller is the part of the BMS on which the state-estimation software, communication logic, diagnostics, and higher-level control functions are executed. In larger packs, these functions are commonly distributed across cell-monitor boards, pack controllers, and higher-level energy-management or vehicle controllers. In smaller stationary or laboratory systems, the same functional blocks are often integrated into one compact BMS board.

Within this practical architecture, state estimation is the software layer that converts voltage, current, temperature, and time into quantities that the controller can use. SOC is needed for usable-energy and power decisions, SOH for ageing assessment and maintenance planning, and state of power for operational limits.[2, 17, 4] Established estimator families include direct measurement and Coulomb counting, equivalent-circuit or electrochemical model-based observers, hybrid methods that

combine physical propagation with adaptive or learned correction, and data-driven methods.[18, 19, 20, 21] Direct measurement and Coulomb counting are simple and transparent, but they accumulate current-sensor errors and require anchoring or reset mechanisms. Model-based approaches remain attractive because they combine physical interpretation with voltage-based online correction. Data-driven methods learn the mapping from measured signals and derived features to battery state directly from data and can represent nonlinear dependencies without deriving all model relations explicitly.[22, 23, 24]

3.2 Neural Networks for Ageing and Battery-State Estimation

Neural network methods are attractive in battery applications because they can combine voltage, current, temperature, time history, and derived variables in one nonlinear estimator.[25] Feedforward models such as MLPs can be effective when the time dependence is encoded explicitly in the input representation. Recurrent architectures such as LSTM and GRU networks encode sequence information inside the model and are therefore common for SOC and SOH estimation tasks in which past operating history influences the present state.[26, 27, 28, 29]

The literature has moved from demonstrating that neural models can fit battery datasets toward asking whether they remain reliable across operating conditions and ageing states. Recent work on practical SOH estimation stresses that curated laboratory data do not automatically represent real operating diversity, while SOC studies increasingly report the importance of robustness against measurement noise, sensor bias, and dynamic load profiles.[30, 5, 6] This creates a clear distinction between model accuracy as an offline regression metric and estimator quality as an operational BMS property.

3.3 Embedded AI, Compression, and Model-Hardware Co-Design

Embedded AI in battery applications adds another constraint layer. A model that performs well on a workstation must still fit the flash and RAM budget of the target controller, meet latency requirements, preserve numerical stability, and reproduce

the intended online state handling in firmware.[31, 8, 9] Existing studies show that neural-network-based SOC and SOH estimators can run on microcontroller-class hardware and even on dedicated BMS-oriented edge devices.[32, 33, 34, 35] However, many demonstrations still report resource use or accuracy only at a high level and do not document the full transfer chain from preprocessing and scaling to stateful streaming execution on the controller.

The hardware trend also points in this direction. Whereas many classical BMS implementations run estimation software on general-purpose Arm Cortex-M microcontrollers, newer MCU families increasingly integrate dedicated AI accelerators or neural-processing units. Examples include STMicroelectronics' STM32N6 with the Neural-ART accelerator, NXP's MCX N series with the eIQ Neutron NPU, Renesas' RA8P1 with Arm Ethos-U55, Alif Semiconductor's Ensemble E1/E3/E5/E7 family with Ethos-U55 acceleration, and Silicon Labs' EFR32xG26/PG26 devices with AI/ML hardware acceleration.[36, 37, 38, 39, 40] This does not mean that every BMS estimator requires a dedicated NPU, especially for compact recurrent models. It does show, however, that embedded AI is becoming a hardware-supported design direction rather than only a software optimization problem.

Compression is one route to this transfer. Quantization reduces numerical precision and can strongly reduce weight storage. Pruning removes weights, units, or channels and can reduce both memory and computation when the sparsity pattern matches the hardware implementation.[10, 41, 11, 42] For recurrent models, structured sparsity is particularly relevant because hidden-state dimensions and gate matrices directly determine runtime cost.[43, 44, 45, 46] The current state of the art therefore supports embedded neural-network estimation in principle, but it also shows that pruning and quantization have to be evaluated together with architecture and firmware, not as architecture-independent guarantees.[47, 48]

3.4 Research Gap Derived from the State of the Art

The current state of the art is therefore uneven. Practical BMS hardware is mature and commercially available, and the core measurement and protection functions are well established. At the same time, many advanced AI-based battery-state estimators are still reported mainly as offline models, often without the same level of detail on sensing constraints, disturbed input signals, hidden-state handling, scaling constants, firmware transfer, and measured controller behaviour. This makes it

difficult to judge whether a method that performs well in a dataset study would remain useful in an embedded BMS.

The most relevant gap is therefore not a missing estimator family, but the missing connection between estimator accuracy, robustness under realistic measurement faults, and embedded execution. Direct, model-based, hybrid, and data-driven methods are often evaluated in separate studies, so their strengths and weaknesses under one shared disturbance protocol remain difficult to compare. Embedded-AI demonstrations show that neural-network estimators can run on microcontrollers, but they do not always document the full transfer chain from preprocessing to stateful streaming inference. Compression studies further show that pruning and quantization can reduce model size, but for battery-state estimation it remains important to verify how these methods interact with the specific recurrent architecture and the target firmware implementation.

Chapter 4

Experimental Basis and Hardware Ecosystem

The experimental basis of this work combines laboratory battery datasets, embedded execution targets, and the Custom BMS Platform for battery management. This combined setup makes it possible to study battery-related artificial intelligence not only as an offline modelling problem, but also as an embedded systems problem shaped by sensing, communication, firmware partitioning, and hardware constraints. The resulting experimental ecosystem forms the common basis for the ageing-estimation work, the robustness benchmark, and the embedded implementation studies.

4.1 Data Sources and Ageing Campaigns

This dissertation draws on two distinct battery datasets with different cell chemistries, ageing protocols, and application contexts. The first dataset uses NMC cells and supports the ageing-estimation study in Chapter 5. The second uses LFP cells and forms the experimental basis for both the robustness benchmark in Chapter 6 and the embedded SOC/SOH study in Chapter 7. Across both datasets, the directly measured quantities are voltage, current, temperature, and time. Dataset-side processing then derives labels for training and evaluation, such as capacity, SOC, and SOH, as well as engineered quantities such as transferred charge and cumulative charge state. Table 4.1 provides a compact side-by-side comparison of the two campaigns.

Table 4.1: Key properties of the two battery datasets used in this dissertation. The NMC dataset supports the SOH ageing-estimation study in Chapter 5, while the LFP dataset provides the experimental basis for the robustness benchmark in Chapter 6 and the embedded deployment study in Chapter 7.

Property	NMC Dataset	LFP Dataset
Cell type	INR18650 NMC	JGCFR18650 LFP
Nominal capacity	2,6 A h	1,8 A h
Nominal voltage	3,7 V	3,2 V
Number of cells	14	15
Campaign type	Cyclic ageing	DoE cyclic ageing
Varied parameters	SOC_{mean}, DOD, C-rate, temperature	C_{chg}, C_{dis}, DOD
Temperature	35 °C / 45 °C	25 °C (constant)
Data source	TU Berlin	TU Berlin / MG-FARM[49]
Used in chapters	Chapter 5	Chapters 6 and 7

4.1.1 NMC Dataset for Stationary-Storage Ageing Estimation

The first dataset consists of cyclic ageing tests on Li-ion INR18650 NMC cells carried out at Technische Universität Berlin. The cells have a nominal capacity of 2,6 A h, a nominal voltage of 3,7 V, a maximum charging voltage of 4,2 V, and a discharge cut-off at 2,75 V. In total, 14 cells are investigated across seven ageing scenarios, with two cells per scenario. The scenarios systematically vary four operating parameters: mean state of charge, depth of discharge, discharge C-rate, and operating temperature. This parametric design is intended to reproduce representative operating windows of stationary-storage systems rather than highly dynamic mobile profiles. Charging follows a CC-CV procedure, while discharging uses constant current. Periodic capacity tests inserted at defined intervals track the SOH progression over time and supply the degradation labels used for model training.

The state of health is defined as the ratio of the current cell capacity to the nominal capacity,[50, 51]

$$\mathrm{SOH}(t) = \frac{C(t)}{C_{\mathrm{N}}}, \tag{4.1}$$

where $C(t)$ is determined from periodic capacity tests in which the cell is fully charged and then discharged under controlled conditions. Because capacity tests

exist only at discrete time points, the SOH label is linearly interpolated onto the continuous measurement timeline. Figure 4.1 shows the resulting SOH progression for all 14 cells over both equivalent full cycles and absolute time in days. Temperature has a pronounced impact on degradation, whereas the interactions between the remaining parameters are less obvious from direct visual inspection alone.

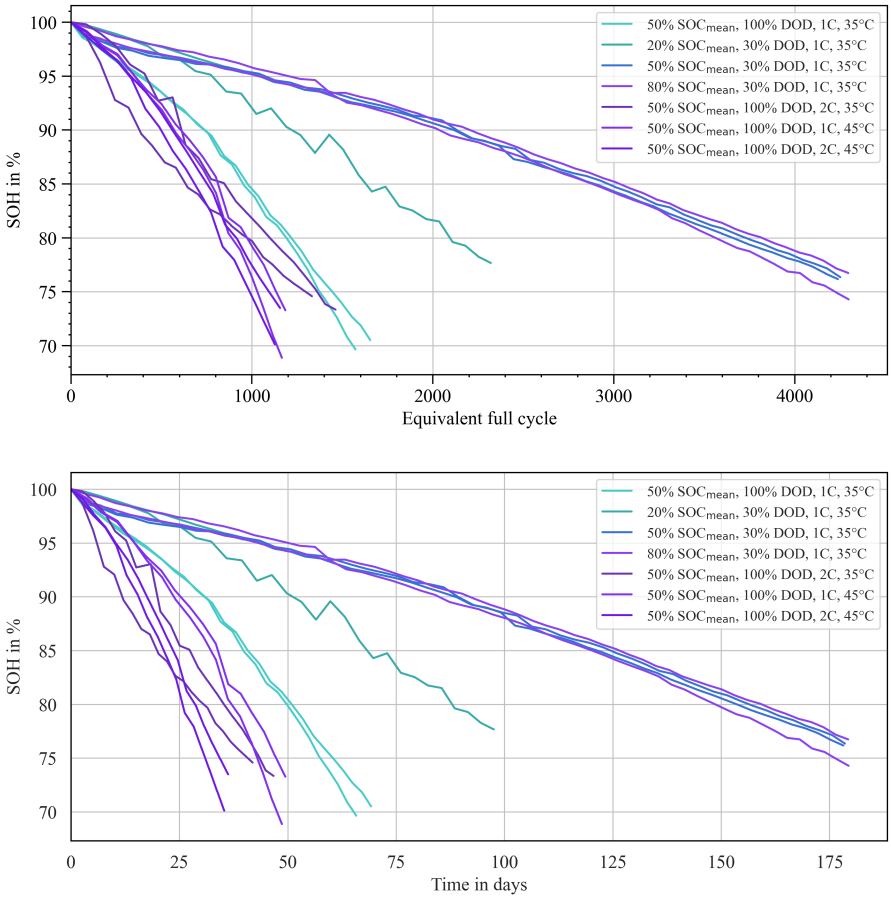

Figure 4.1: SOH progression of the 14 investigated NMC cells: top over equivalent full cycles and bottom over absolute time in days. Colors denote the seven ageing scenarios, with two cells per scenario.

This dataset is used exclusively in Chapter 5, where the ageing scenarios and chapter-specific feature engineering are described in detail.

4.1.2 LFP Dataset for Robustness Benchmarking and Embedded Deployment

The second dataset is the published LFP DoE Cycle Ageing Dataset recorded at Technische Universität Berlin within the MG-FARM project.[49] It uses commercially available JGCFR18650-1800mAh-3.2V LFP cells with a nominal capacity of 1,8 A h and a nominal voltage of 3,2 V. The ageing campaign follows a three-factor design-of-experiments scheme conducted at a constant ambient temperature of 25 °C. The three varied factors are the charge rate C_{chg} from 0.3 C to 1.0 C, the discharge rate C_{dis} from 0.5 C to 3.0 C, and the depth of discharge DoD from 38% to 71%. A mixed full and fractional factorial plan assigns 15 test cells across eight operating points. Between ageing loops, dedicated check-up phases with capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterisation are inserted. Stationary-storage load profiles derived from residential PV home storage are additionally applied so that the data contain both tightly controlled diagnostics and application-oriented dynamic behaviour. Raw measurements are available at 1 Hz and include voltage, current, terminal temperature, and auxiliary channels.

A design of experiments is used here to structure the ageing campaign so that several operating influences can be varied systematically within a manageable number of cells.[52] Instead of changing only one operating variable at a time, the DoE plan combines charge rate, discharge rate, and depth of discharge into defined operating points. This makes it possible to observe not only the isolated influence of one factor, but also differences that arise from their combined loading conditions. For the later SOC and SOH studies, this is important because the estimators are not trained and evaluated on one narrow cycling regime. They see cells that have aged under different current rates and cycling depths, while the check-up phases still provide comparable reference information across the campaign. The DoE structure therefore gives the dataset both experimental control and operational diversity, which is useful for robustness benchmarking and embedded estimator validation.

Figure 4.2 visualizes the three-factor operating-point design. Figure 4.3 shows a representative segment of the resulting trajectory structure, combining cyclic ageing blocks, check-up phases, and stationary-storage load-profile segments. Figure 4.4 shows the SOH spread across cells and operating points over the full campaign duration.

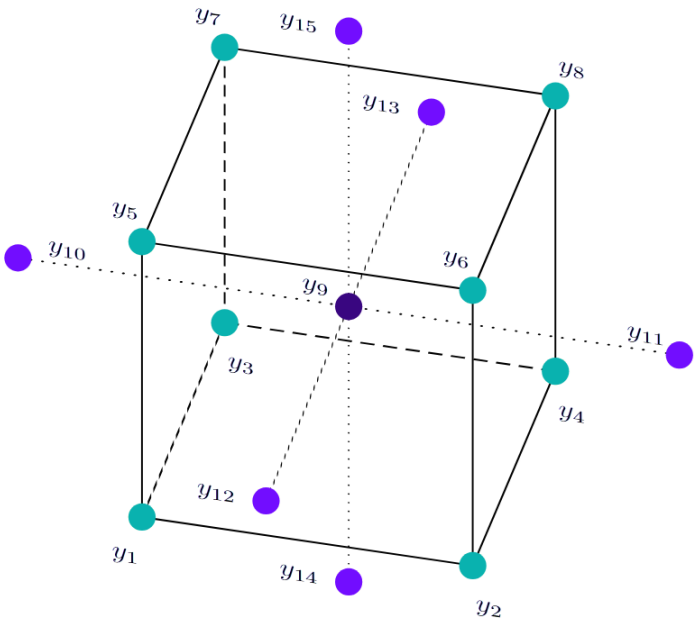

Figure 4.2: Design-of-experiments cube of the LFP ageing dataset. The three axes correspond to charge C-rate, discharge C-rate, and depth of discharge. The highlighted points define the factorial test matrix assigned to the 15 ageing cells.

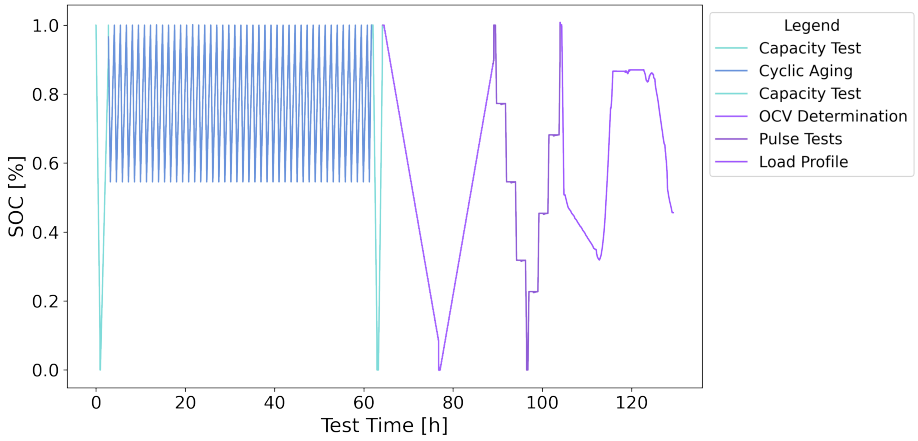

Figure 4.3: Representative SOC trajectory illustrating the experimental structure of the LFP campaign. The sequence combines cyclic ageing operation, diagnostic check-up blocks, and stationary-storage load-profile segments, capturing both laboratory reference phases and application-oriented dynamic behaviour.

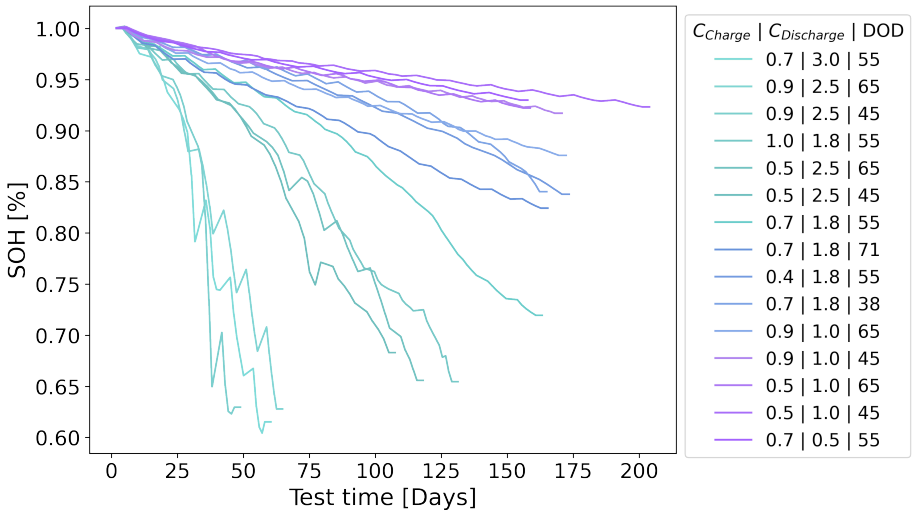

Figure 4.4: SOH progression across the 15 LFP cells over the full ageing campaign. The spread between trajectories reflects the different operating points in the factorial plan and motivates the use of data-driven SOH estimators that generalise across diverse loading histories.

Across both Chapters 6 and 7, the SOC and SOH labels are constructed from dataset-side processing. The SOC label is derived from Coulomb counting with drift compensation,

$$\mathrm{SOC}(t) = \mathrm{SOC}(t_0) - \frac{1}{C_{\mathrm{app}}} \int_{t_0}^{t} I(\tau)\, \mathrm{d}\tau, \tag{4.2}$$

with resets to $\mathrm{SOC} = 1$ after sustained constant-voltage full-charge phases to reduce long-horizon drift. The SOH label is obtained from periodic capacity check-ups,

$$\mathrm{SOH}_k = \frac{C_k}{C_{\mathrm{nom}}}, \tag{4.3}$$

where C_k is the measured discharge capacity and C_{nom} is the nominal capacity.[2, 53] Both labels rely on dataset-side information, namely full-capacity discharge tests and sustained CV-based full-charge resets, that is not available during embedded deployment. They therefore serve as reference labels for model training, estimator calibration or validation, and evaluation in Chapters 6 and 7.

This dataset is used in Chapter 6 for the robustness benchmark and in Chapter 7 for the embedded SOC and SOH estimation study. Chapter-specific feature engineering and normalisation are described in the respective chapters.

4.2 Embedded Targets and Execution Chain

The embedded work relies on two complementary targets: the final Custom BMS Platform hardware and an STM32H755-based evaluation-board setup for rapid firmware iteration and controlled implementation tests. The integrated BMS hardware is used whenever sensing, balancing, protection, communication, telemetry, and on-board state estimation must be validated as one coupled system rather than as isolated software modules, whereas the evaluation-board setup is used whenever estimator code, toolchain details, memory demand, and runtime behaviour have to be checked quickly before transfer to the full Custom BMS Platform.

In both cases, the deployment path follows the same practical sequence: state-estimation methods are first prototyped in Python or MATLAB, then rewritten as plain C modules, compiled into the STM32 firmware, and executed locally on the target hardware instead of off-board. During validation, the firmware records voltage, current, temperature, and relevant intermediate estimator variables. After each run, these traces are aligned with higher-fidelity host-side SOC and SOH reference labels. The workflow does not use a hardware-in-the-loop setup, which keeps the evaluation focused on firmware execution, measured board behaviour, and comparison against host-side reference traces. This split between evaluation-board work and integrated BMS operation is important for the dissertation because it separates pure implementation questions from questions of sensing, protection, and full system integration.

4.3 Custom BMS Platform

The custom laboratory hardware developed in this work is referred to simply as the Custom BMS Platform. It is a physically realised 4s laboratory BMS designed to monitor and protect the cells, execute on-board SoC and SoH estimation, perform balancing, and provide the technical basis for controlled experiments with embedded battery intelligence. A central reason for the in-house design is adaptability: in contrast to a commercial off-the-shelf BMS, the Custom BMS Platform gives direct access to the sensing chain, firmware structure, interfaces, and estimator integration, so that state-estimation methods can be exchanged, modified, and benchmarked repeatedly on the same hardware basis. The board should therefore be understood less as a finished product and more as a configurable research platform that was built deliberately to support rapid iteration, customisation, and

comparative testing.

4.3.1 System Objective and Laboratory Scope

The present implementation is a compact and reproducible 4s laboratory BMS for battery ageing studies and embedded state-estimation experiments at TU Berlin EET. Because only a small number of cells can be connected, the platform is intended for laboratory work, including ageing tests, cyclic and calendar experiments, pack and general hardware tests, and representative charging and discharging profiles, rather than for unrestricted field deployment in large battery racks or real applications. Exact field loads are not reproduced one-to-one in the laboratory. Instead, representative operating profiles are approximated within a controlled setup that allows calibration, diagnostics, repeated validation, and structured dataset generation.

This laboratory orientation is deliberate because it enables benchmark-style comparison of different state-estimation approaches under the same measurement chain, including classical model-based observers and modern data-driven or neural methods executed directly on the STM32 controller. The platform concept is explicitly geared toward experimentation: one hardware base should support multiple estimator variants without rebuilding the full BMS around each method. The validation campaigns include duty cycles derived from decentralized stationary-storage use cases with limited maintenance access, and these tests were used to record voltage drift, current ripple, temperature gradients, and embedded estimator outputs under realistic operating conditions.

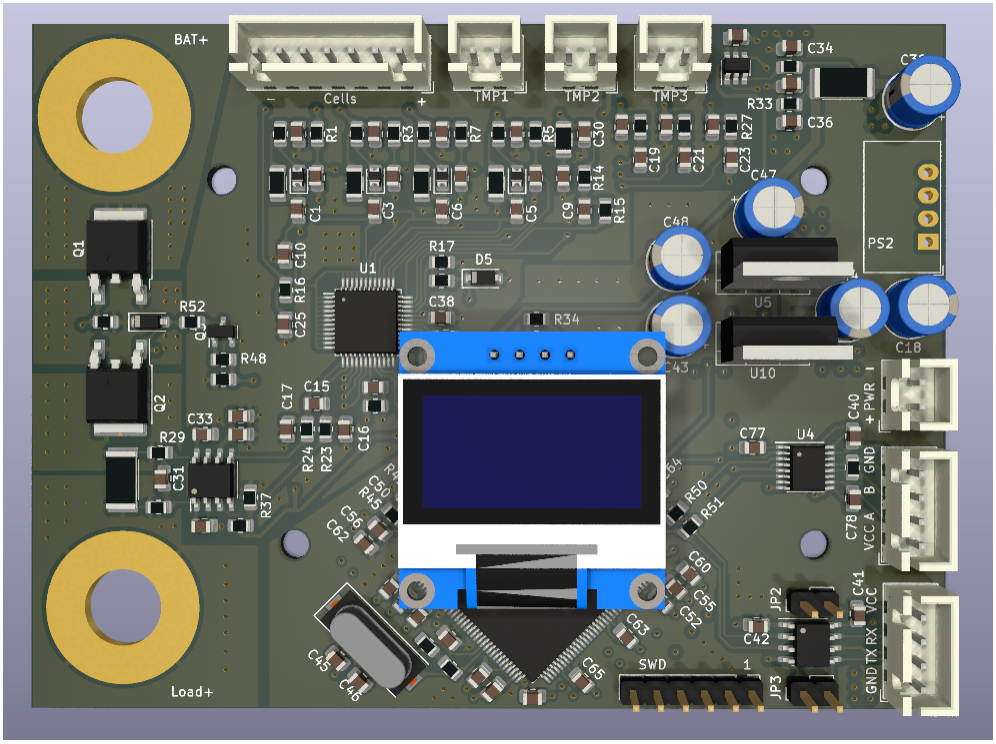

Figure 4.5: Board-level view of the Custom BMS Platform with integrated sensing, controller, communication, and auxiliary interfaces.

4.3.2 Hardware Architecture

The Custom BMS Platform can be understood as three tightly coupled hardware domains: the TLE9012DQU-based sensing and balancing stage, the current-measurement and protection interface, and the controller/communication island around the dual-core STM32H755. The board layout keeps the high-current zone, shunt, and relay path physically separated from the low-voltage controller and communication domain. This separation matters not only for safety, but also for electromagnetic compatibility and stable measurement quality.

The Infineon TLE9012DQU forms the measurement backbone of the pack: in the laboratory build it supervises four series-connected cells with buffered RC filtering and provides cell-voltage and temperature acquisition together with passive balancing capability. The active front-end includes balancing circuitry, while unused monitor inputs are terminated according to the 12-to-4 cell adaptation of the reference design so that measurement integrity is preserved on the active channels. Temperature instrumentation is extended beyond the standard TLE inputs by an NTC harness that can connect additional probes at cell terminals, busbars,

or ambient reference points, which provides a finer spatial thermal picture during ageing and state-estimation tests. The controller domain uses an STM32H755ZITx dual-core microcontroller. This design choice is central because it creates a clean architectural separation between conventional BMS functionality and the estimator layer that is investigated in the dissertation.

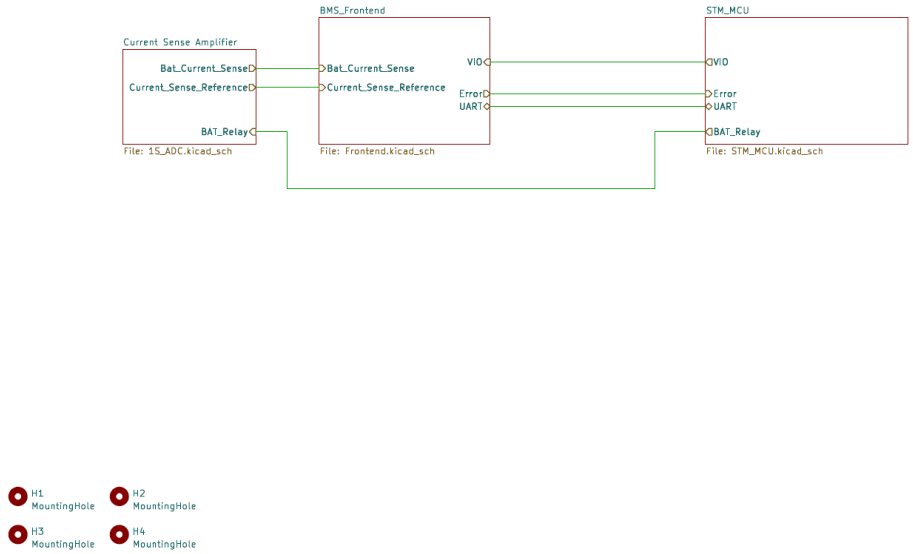

Figure 4.6: Reduced system-level block view of the Custom BMS Platform showing the interaction between current sensing, BMS front-end, and microcontroller domain.

4.3.3 Firmware and Execution Concept

The firmware concept is intentionally centred on the dual-core STM32 architecture. The Cortex-M4 core is used for conventional BMS functionality such as measurement handling, supervision, and general system management, whereas the Cortex-M7 core is reserved for the computationally heavier state-estimation routines. This split is important for the dissertation because it makes the estimator layer exchangeable: different SoC and SoH methods can be integrated, tested, and replaced on the M7 side without redesigning the complete BMS firmware structure. The platform was built exactly for this purpose, namely that one stable hardware and BMS basis should support repeated experimentation with different embedded-AI methods, different estimator classes, and different implementation variants. From the perspective of the dissertation, the central contribution of the firmware architecture is therefore not the low-level implementation detail itself,

but the fact that it creates a reusable and customisable execution environment for embedded battery state estimation.

4.3.4 Experimental Relevance for Embedded AI

The Custom BMS Platform is not only a monitoring board, but an experimental platform for comparing estimator classes under realistic embedded conditions. Its main value for the dissertation lies in controllability and openness: the same measurement hardware can be reused while state-estimation methods are changed, refined, compressed, or replaced. This makes it possible to compare classical model-based observers and data-driven or neural approaches on one common embedded basis instead of distributing them across unrelated hardware environments. The Custom BMS Platform therefore closes the gap between algorithm development and embedded validation, and it was chosen precisely because it can be modified and extended in ways that would be difficult with a purchased closed BMS. At the same time, the 4s implementation remains deliberately compact: it serves as a reproducible laboratory baseline from which later transfer to larger stacks or a 16s/48 V variant can be reasoned without changing the overall control and estimator architecture.

Chapter 5

Neural Network Architecture for Ageing Estimation in Stationary Storage Systems

5.1 Application Context and Related Work

State-of-health estimation is a central prerequisite for reliable stationary storage operation, especially in smart-grid and microgrid settings in which storage units must remain available over long maintenance intervals and under fluctuating renewable-power conditions.[54, 55] This application context is more demanding than conventional laboratory estimation tasks because smart-grid storage operates under varying load, charging, and renewable-infeed conditions and therefore requires methods that remain informative outside narrowly standardized test windows. The ageing tests considered in this chapter are designed to reflect these operating characteristics in a controlled laboratory setting so that the later findings remain relevant for practical stationary-storage deployment rather than only for idealized benchmark cases.

Within this field, the literature usually distinguishes between model-based and data-driven SOH estimation. Model-based methods exploit explicit physical battery descriptions, whereas data-driven methods learn the relation between measured operating variables and the degradation state directly from data. The data-driven route is chosen here because it is better suited to strongly nonlinear ageing behaviour and does not require the same level of electrochemical model knowledge for practical deployment.[56, 57] Representative model-based alternatives include Kalman-filter-based joint SOC/SOH estimation, particle-filter-based ageing tracking, and fractional-order ageing models. Reported errors in this group range from about 1 % for dual adaptive Kalman-filter approaches to around 3 % for capacity-

fade estimation and 4 % for power-fade estimation, while fractional-order-model studies report remaining-capacity errors near 3,1 %. These approaches can therefore achieve strong nominal accuracy, but they depend on accurate parameter identification, carefully tuned filters, and, in some cases, computationally demanding physical formulations. Their robustness may further deteriorate when the initial SOC is uncertain, when empirical ageing relations do not generalize cleanly across operating regimes, or when offline parameter-identification steps are required before deployment.[58, 59, 60]

On the measurement-driven and machine-learning side, the discussion covers pressure-based abnormality detection, partial-voltage and incremental-capacity methods, combinations of deep learning and Kalman filtering, autoregressive prediction, support-vector-machine approaches, fuzzy-entropy features, and SOC-domain preprocessing. Direct-measurement approaches based on reduced charging-voltage windows report average errors below 2,5 %, and combinations of deep learning and Kalman filtering reduce estimation errors to about 2 %. At the same time, these studies reveal typical limitations such as sensitivity to selected operating windows and charging conditions, growing estimation errors at higher degradation levels, dependence on feature validity, susceptibility to abnormal data points, and limited transferability to varying operating conditions.[61, 62, 63, 64, 65, 66, 67, 68] More recent hybrid and sequence-based approaches process raw voltage, current, and temperature histories directly by neural networks or combine data-driven inference with physics-informed components. Sequence-to-sequence hybrid degradation prediction has, for example, reported mean absolute percentage errors below 2,5 % for the capacity trajectory and 6,5 % for the remaining usable cycles when only 20 % of the data are used, with further reductions to below 1,3 % and 5 % when 50 % of the data are available. These approaches are methodologically close to the later development path of this dissertation. The present ageing-estimation study nevertheless starts from a deliberately simpler MLP baseline in order to test how far feature engineering and lag-sequence construction can already push a comparatively lightweight model.[69, 70, 71, 72]

The intention is therefore not to claim that MLPs are universally superior to recurrent or hybrid models. The central assumption is instead that a suitable representation of voltage, temperature, and current-throughput history can make a simple feedforward network surprisingly effective while keeping the model easier to train, easier to interpret structurally, and closer to later deployment-oriented questions. This choice also reflects a practical concern: for continuously operated

storage systems, classical SOH assessment from clearly delimited capacity tests is often unavailable online, so the estimator must infer degradation from operational trajectories observed in less structured states.[73, 74, 22] The methodological contribution of the chapter lies less in proposing an exotic neural-network architecture than in combining a compact MLP with a preprocessing pipeline tailored to stationary-storage data. In particular, lag-sequence construction allows the network to learn from segmented operating histories instead of relying on explicitly repeated capacity-test signatures, which improves data efficiency and keeps storage and training effort manageable. The inclusion of cumulative-current information broadens the input representation beyond instantaneous current, voltage, and temperature and is intended to improve robustness under heterogeneous operating conditions.

Under this framing, the approach is meant as a resilient compromise between accuracy, interpretability, and deployability. It targets imperfect and partially irregular measurement streams, avoids dependence on detailed electrochemical cell models, and is designed to remain transferable in principle across different stationary-storage configurations when sufficient training data are available. The chapter is organized accordingly: it first introduces the dataset and preprocessing logic, then the MLP structure and evaluation setup, and finally the combined result discussion and chapter conclusion. The overall workflow discussed in these sections is summarized later in Figure 5.3.

5.2 Raw Cell Dataset, Parameters and Measured Quantities

The NMC dataset used in this chapter is described in Section 4.1. In brief, it consists of cyclic ageing tests on 14 INR18650 NMC cells at Technische Universität Berlin, grouped into seven scenarios with two cells each. All cells start from an initial SOH of 100 %. Table 5.1 lists the seven scenarios and their parametric variation. The four varied parameters, namely mean state of charge, depth of discharge, discharge C-rate, and operating temperature, are selected to reproduce representative operating windows of stationary storage systems rather than highly dynamic mobile-load profiles.[75] The chosen mean-SOC levels are particularly intended to reflect stationary-storage operation around recurring operating points instead of idealized full-charge/full-discharge behaviour, and their systematic variation makes

it possible to examine how the main operating conditions shape both degradation progression and the later estimation problem.

After a defined number of ageing cycles, capacity tests are inserted to determine the degradation state of the cells, consistent with established battery ageing and calendar-testing practice.[76] Because the sampling intervals depend on the respective process step, all measurements are resampled to 1 s before feature engineering is applied. One scenario with 20 % mean state of charge, 30 % depth of discharge, 1C discharge rate, and 35 °C contains premature test termination and irregular measurements for cells 3 and 4. These data are nevertheless retained in the training pool in order to test whether meaningful ageing relations can still be learned from imperfect measurements. This is a deliberate robustness test: real stationary-storage data may also contain atypical interruptions, measurement irregularities, or operating deviations caused by changing demand, grid disturbances, or physical boundary effects. The model is therefore not only exposed to ideal trajectories but also to non-ideal data in order to assess whether the learned ageing relation remains usable under realistic conditions.

Table 5.1: Ageing scenarios of the stationary-storage dataset, showing the seven investigated test cases and the systematic variation of SOC_{mean}, DOD, discharge C-rate, and operating temperature.

Scenario	Cell No.	SOC_{mean}	DOD	$C\text{-}Rate_{dis}$	T (°C)
1	1 and 2	50%	100%	1C	35
2	3 and 4	20%	30%	1C	35
3	5 and 6	50%	30%	1C	35
4	7 and 8	80%	30%	1C	35
5	9 and 10	50%	100%	2C	35
6	11 and 12	50%	100%	1C	45
7	13 and 14	50%	100%	2C	45

5.3 Preprocessing Method and Feature Engineering

5.3.1 Cumulative Current

The preprocessing pipeline is built around the argument that instantaneous current alone is not sufficient as an explanatory variable for ageing estimation. Cumulative

current is introduced as the central preprocessing feature because it captures the two quantities that are most relevant at this stage: the time dependence of the measurements and the cumulative throughput stress experienced by the cell. In its basic form, integrating the absolute current over time encodes not only the present loading level but also the prior electrochemical exposure and, therefore, a compact proxy for the cyclic age that is not visible from an instantaneous sample alone,

$$Q(t) = I_{\text{cumulative}}(t) = \int_0^{\Delta t} |I(t)|\, dt. \tag{5.1}$$

The formulation is also physically motivated by the interaction between current and voltage over a sequence: their joint evolution carries ageing-relevant information that is closer to an impedance-like gradient than a single current reading. The preprocessing is therefore extended by distinguishing charging and discharging contributions, because both directions can induce different voltage gradients and different effective ageing signatures. The corresponding error reduction can be observed empirically, but because these effects are strongly collinear, the improvement cannot be attributed cleanly to one isolated mechanism alone. In this formulation, cumulative current becomes the feature that allows a relatively simple feedforward network to still access ageing-relevant temporal context without explicitly using a recurrent structure. Voltage and temperature are retained as further direct inputs, so the final feature set combines operating-state information with a compact representation of the prior load exposure.

5.3.2 Estimation of the State of Health

The SOH definition, the construction of SOH labels from periodic capacity tests, and the resulting progression across all 14 NMC cells are described in Section 4.1 (Equation 4.1, Figure 4.1). Temperature has a pronounced impact on degradation, which motivates the correlation analysis below as a more structured view of the relation between operating parameters and SOH.

5.3.3 Correlation Analysis

To better understand which measured quantities should be retained for learning, the SOH trajectories are supplemented with the correlation analysis summarized in Figure 5.1. This matrix provides a more structured view of the relation between operating parameters and SOH because the influence of several operating variables

and their mutual interactions is not sufficiently clear from the SOH trajectories alone. The resulting heatmap indicates that temperature and cumulative current, both in positive and negative direction, show the clearest direct relation to SOH. In contrast, the time derivative of the voltage, although itself strongly temperature-dependent, does not show a comparably direct influence in the linear correlation view.

The correlation analysis can only reveal linear dependencies and therefore serves only as a starting point for feature selection rather than as a substitute for the later model-based evaluation. Non-linear effects may still be relevant and can subsequently be captured by the neural network. Even with this limitation, the analysis provides a concrete justification for retaining temperature and cumulative-current descriptors as core inputs for the final model.

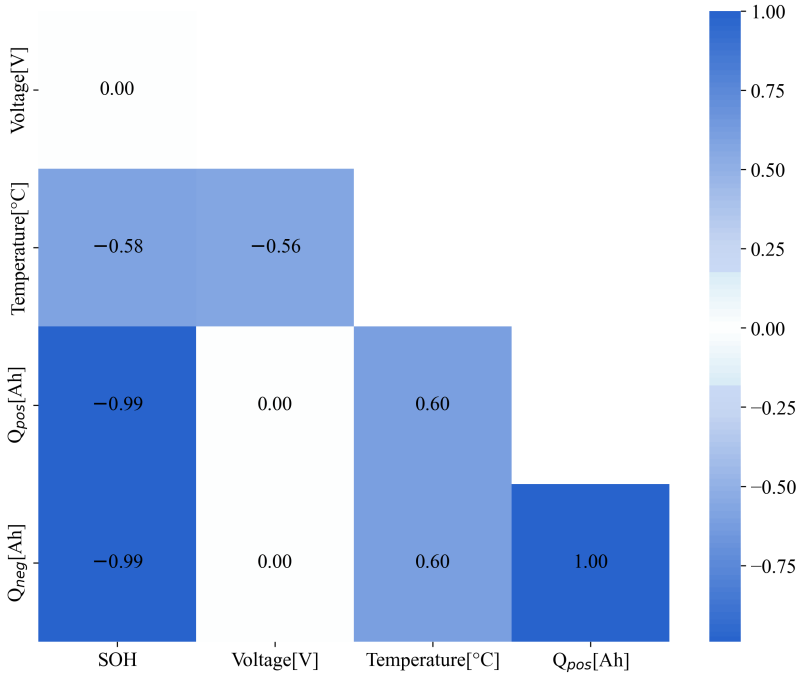

Figure 5.1: Correlation heatmap illustrating the linear relation between SOH and candidate input variables, including temperature and cumulative current Q in positive and negative direction. Correlation coefficients near $+1$ or -1 indicate strong linear dependence, whereas values near 0 indicate little or no direct linear relation.

5.3.4 Formation of the Lag Sequence

Each learning sample is constructed as a lag sequence of voltage, temperature, and cumulative current, with the SOH at the current point in time used as the corresponding label, as illustrated in Figure 5.2. This transforms the continuous measurement stream into structured sliding-window samples that can be processed by a feedforward network without passing the complete life history of an individual cell directly into the model. The inclusion of past input values is intended to improve prediction quality, while the sliding window defines the temporal duration of the usable history. The initial parameterisation uses a time resolution of 30 min and 12 lag steps, which corresponds to a history length of 6 h. This starting choice depends on the expected usage pattern, the charging and discharging durations, and the presence of short high-current phases in the data.

The time resolution has to match the actual charging and discharging behaviour. If the spacing is too coarse, relevant dynamics disappear. If it is too fine, the model sees an unnecessarily long and noisy representation that can make training less effective. If short pulses fall within one interval, they are represented only through the averaged feature values of that interval. This design can also be linked to the sampling theorem: the spacing between data points has to remain compatible with the temporal structure of the charging and discharging cycles so that the network can still recognise the relevant periodic behaviour.[77] Because the raw measurements are resampled before sequence generation, the preprocessing step also acts like a low-pass filter and strongly attenuates high-frequency noise components, so noise is not treated as a dominant issue for the present estimation setup. A further benefit of the lag-sequence approach is that the training samples are no longer tied to one specific cell trajectory. Instead of memorising individual cells, the model is trained on many local windows drawn from the full dataset.

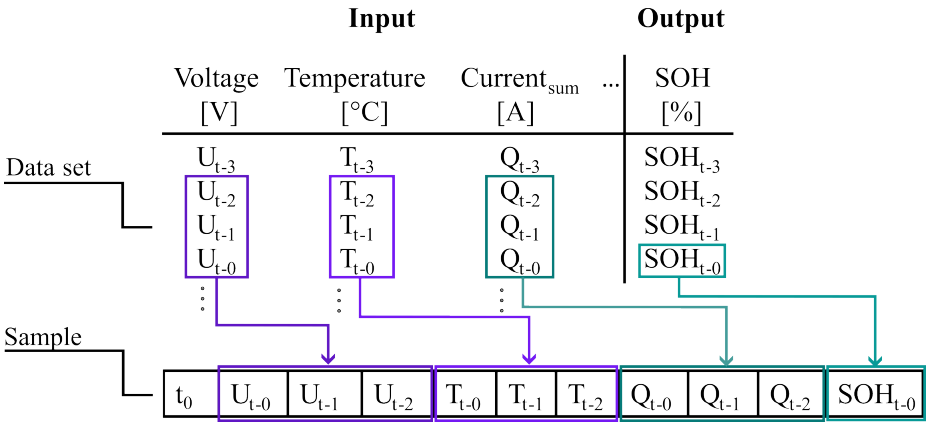

Figure 5.2: Illustration of lag-sequence construction from raw cell measurements into supervised MLP samples. Voltage, temperature, and cumulative current form the structured input history, while the SOH at the current time step serves as the corresponding label.

5.3.5 Training, Validation, and Test Splits

The available data are separated into training, validation, and test subsets, as summarised in Figure 5.3. The training split is used to adapt the network weights, while 20% of the available training sequences are reserved for validation. The validation split is used during training to monitor overfitting and generalisation behaviour and to support hyperparameter decisions without exposing the model to the final test cells. At the same time, several complete cells are withheld from the training process for the final performance assessment. In total, four out of fourteen cells are retained for this independent accuracy evaluation. This choice is important because the method is intended to generalise to previously unseen cells and not only to previously unseen windows extracted from known cells. The explicit separation of training, validation, and test data therefore makes the resulting performance assessment more robust and more informative.

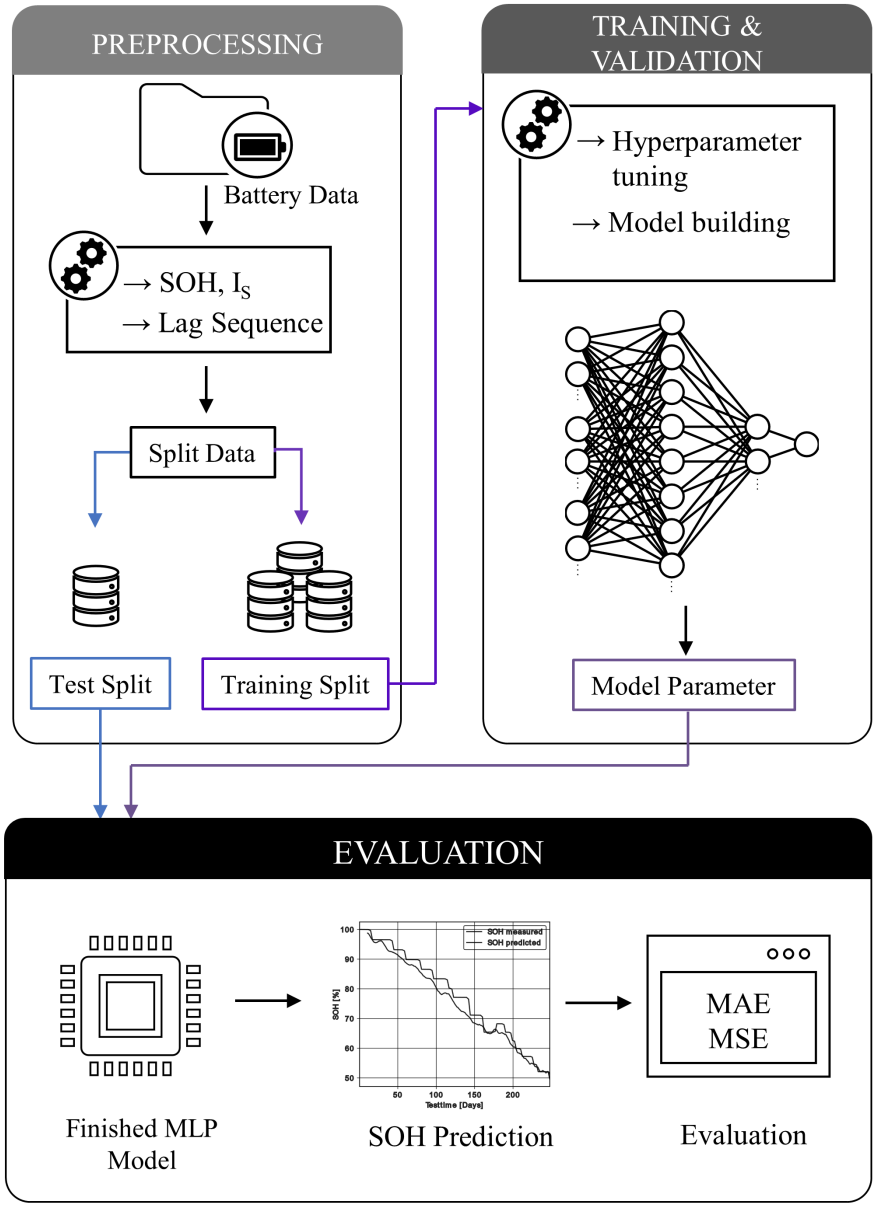

Figure 5.3: Workflow of the proposed MLP-based SOH estimation approach, from raw-cell preprocessing and data splitting through training and hyperparameter tuning to prediction and final evaluation.

5.3.6 Data Scaling

All input features are scaled to the interval between 0 and 1 before training.[78] This step is necessary because the numerical ranges of the features differ strongly: voltage remains in a narrow interval between 2,75 V and 4,2 V, whereas cumulative

current can rise to several thousand ampere-hours across the full ageing experiment. Uniform scaling improves numerical stability, supports convergence during optimisation, and avoids a situation in which large-magnitude inputs dominate the network weights while smaller but still relevant signals are effectively ignored.[78] The same scaling convention must be applied consistently to the training, validation, and test data so that the learned mapping is evaluated under comparable feature ranges and the resulting performance metrics remain interpretable across all splits.

5.4 Multilayer Perceptron Neural Network Model Structure

The selected model type is a multilayer perceptron with fully connected layers. A convolutional interpretation is not pursued because the data do not exhibit a spatial image-like structure. Recurrent architectures such as LSTMs are also not used at this stage, because the temporal dependence is already encoded through the lag-sequence construction and because the recurrent alternatives are more resource-intensive and more difficult to tune in the chosen setting.[79] This architecture choice is also connected to the later hyperparameter-search setup: the chapter keeps the estimator architecture feedforward and shifts the main optimization effort to lag-window design, network depth, node count, and training parameters, which are then searched efficiently with Hyperband.[80]

As illustrated in Figure 5.4, the MLP consists of an input layer, several dense hidden layers with ReLU activation, and a single linear output node for the SOH estimate. The original starting point is intentionally small and uses one hidden layer with eight nodes, and the final network depth and width are then determined in the hyperparameter search. In the tuned configuration, summarised in Table 5.2, the model grows into a deep dense architecture with nine hidden/output stages and a final linear regression output. The resulting structure remains conceptually simple from an architectural point of view, but it is substantially larger than the initial baseline.

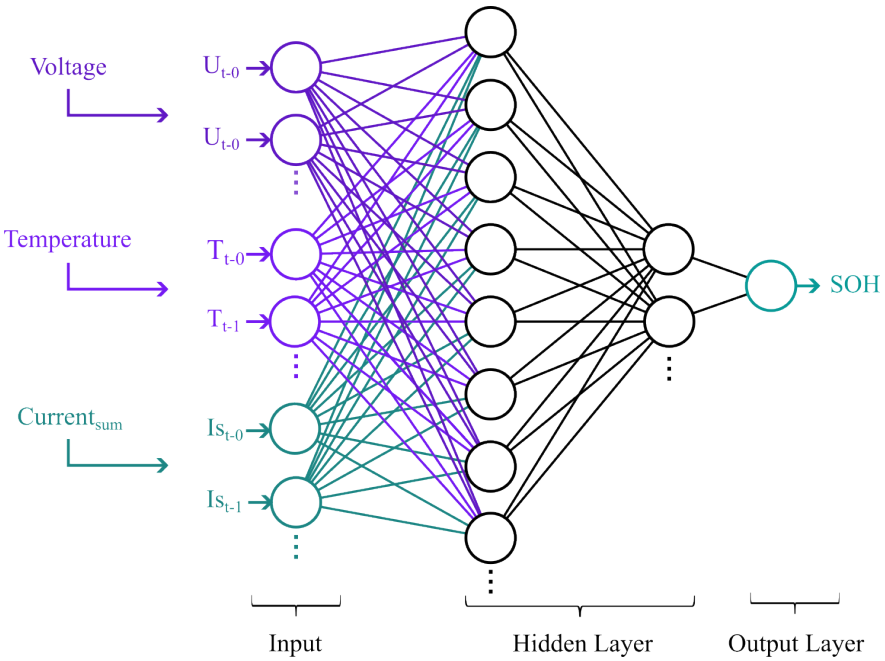

Figure 5.4: Illustration of the multilayer perceptron (MLP) used for SOH estimation. The network receives voltage, temperature, and cumulative current as inputs, with the number of input nodes scaling with the number of lag-sequence elements in a sample. Each input node is fully connected to the hidden layers, which is characteristic of an MLP, and a single output node provides the SOH estimate. The figure highlights the feedforward flow of information from the input features through the hidden representation to the final prediction.

Table 5.2: Final tuned configuration of the SOH-estimation MLP after hyperparameter optimisation, listing the number of nodes and the activation function used in each layer of the network.

Layer	Nodes	Activation Function
0	128	ReLU
1	256	ReLU
2	256	ReLU
3	256	ReLU
4	128	ReLU
5	512	ReLU
6	128	ReLU
7	256	ReLU
8	256	ReLU
9	1	Linear

5.5 Evaluation Metrics and Hyperparameter Tuning of the Neural Network

This section defines the error metrics used for training, validation, and final testing before summarising the hyperparameter tuning of the MLP. During training and validation, model quality is monitored primarily with the mean squared error,

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \tag{5.2}$$

where n is the number of evaluated samples, y_i is the measured SOH value of sample i, and $\hat{y}_i$ is the corresponding prediction. After training, the independent test cells are evaluated with both MSE and mean absolute error,

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{5.3}$$

MAE is particularly useful for interpretation because it remains in the same unit as the SOH target. The stored SOH and prediction series use the normalized 0 to 1 representation, but the original evaluation scripts multiply both series by 100 before calculating the reported MAE and MSE values. The ageing-estimation results

in this chapter are therefore reported primarily on the 0 to 100% SOH scale, with MAE in percentage points and MSE in squared percentage points. Where comparison with later chapters benefits from the dissertation-wide 0 to 1 convention, the corresponding normalized values are stated explicitly.

The hyperparameter search covers the number of lag steps, the temporal resolution of the lag sequence, network depth, number of nodes, batch size, kernel regularisation, optimizer type, and activation function. Within this set, the number of lag steps and the temporal resolution are the most method-specific parameters, because they define how much history is provided to the model and at which temporal granularity it is represented. The number of nodes controls the representational capacity of the network, whereas the network depth influences how strongly the model can build more abstract internal feature combinations across layers. Both parameters therefore affect the balance between flexibility, generalisation, and model complexity. Hyperband is used as the main search strategy because it distributes training resources adaptively, evaluates many candidate settings quickly, and allocates more epochs only to the most promising configurations.[80]

The study reports that ADAM outperforms RMSProp, Adadelta, and Adagrad for this task, that larger batch sizes reduce overfitting tendencies, and that explicit kernel regularisation does not lead to a meaningful improvement. Batch size also affects the training trade-off directly: larger batches are computationally efficient and converge reliably, whereas smaller batches can improve generalisation at the cost of longer or less stable optimisation. The optimizer analysis is complemented by an activation-function comparison, in which ReLU yields the lowest loss in the tested setup while sigmoid, tanh, and softmax perform worse. The models are trained on approximately 15,000 sequences with varying batch-size and epoch combinations, and larger batches converge to a comparable minimum while reducing overfitting tendencies.

5.6 Results and Discussion

This section evaluates how the two most influential sequence-design parameters, namely the number of lag steps and the temporal resolution, affect the final SOH-estimation performance. The result presentation and interpretation are kept together because the numerical errors depend directly on the chosen history length, the temporal aggregation, and the ageing trajectory of the evaluated cell. For the

detailed analysis, Figure 5.5 compares three exemplary cells aged under different conditions. Four time resolutions are considered, namely 1 min, 10 min, 30 min, and 60 min, while the number of lag steps is varied between 8, 12, 16, and 20. All MAE values in this section are shown in percentage points of SOH to remain directly interpretable on the original 0 to 100% SOH scale.

Across the exemplary cells, the most favourable behaviour appears at intermediate temporal aggregation rather than at the shortest or longest tested sampling intervals. This matches the characteristic charge and discharge durations of the investigated scenarios, which typically fall in the range of approximately 20 min to 1 h. If the time resolution is too large, relevant behaviour is averaged out and important information is lost. If it is too small, the training process becomes less effective, the effective histories become unnecessarily long, and the resulting representation does not translate automatically into better prediction quality. The MAE matrix shows the smallest errors for 16 lag steps at a time resolution of 10 min, which corresponds to a history length of 160 min. Extending the sequence length beyond 20 steps is reported as disadvantageous because the resulting history becomes excessively long and does not improve prediction quality further.

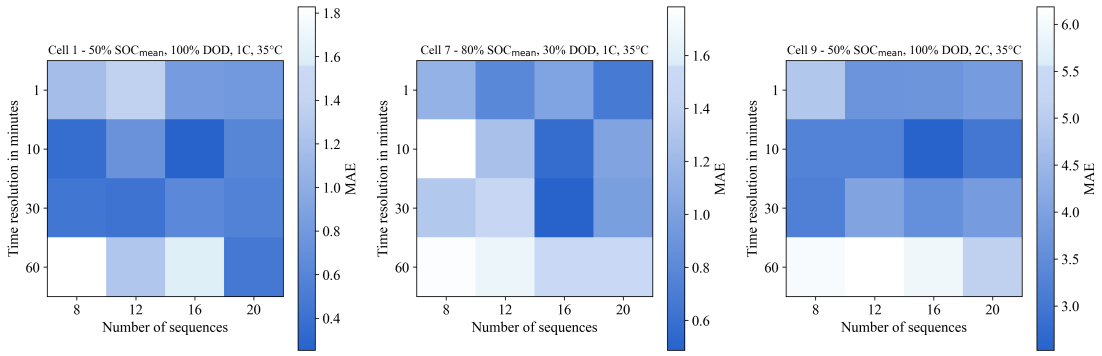

Figure 5.5: Matrix of MAE values shown in percentage points of SOH on the 0 to 100% SOH scale. The corresponding normalized MAE on a 0 to 1 SOH scale is obtained by scaling the displayed values accordingly. The colour scale indicates the magnitude of the prediction error and highlights that the most favourable operating region is found at intermediate temporal aggregation rather than at the shortest or longest tested interval.

Figure 5.6 compares the predicted and measured SOH trajectories. Cells 7 and 1 represent typical ageing behaviour and are reproduced well, whereas a cell with a pronounced U-shaped SOH progression leads to a substantially larger prediction

error. Cells 7 and 1 are therefore used as representative examples of typical ageing behaviour, whereas cell 9 serves as a deliberately selected atypical case. Figure 5.7 complements this trajectory-based view with a direct comparison between predicted and measured SOH values. For the favourable individual cells, the reported best errors are MSE values of $0.410\,\%^2$ and $0.118\,\%^2$ and MAE values of $0.487\,\%$ and $0.253\,\%$. These correspond to normalized MAE values of 0.00487 and 0.00253 on a 0 to 1 SOH scale. Across the evaluated cases, the average MAE is $1.10\,\%$, corresponding to a normalized MAE of 0.0110, and the average MSE is $2.95\,\%^2$, corresponding to a normalized MSE of $2.95 \cdot 10^{-4}$. The prediction plots additionally show that a post-rolling filter is applied to reduce visual noise and to make the comparison between measured and estimated SOH easier to interpret.

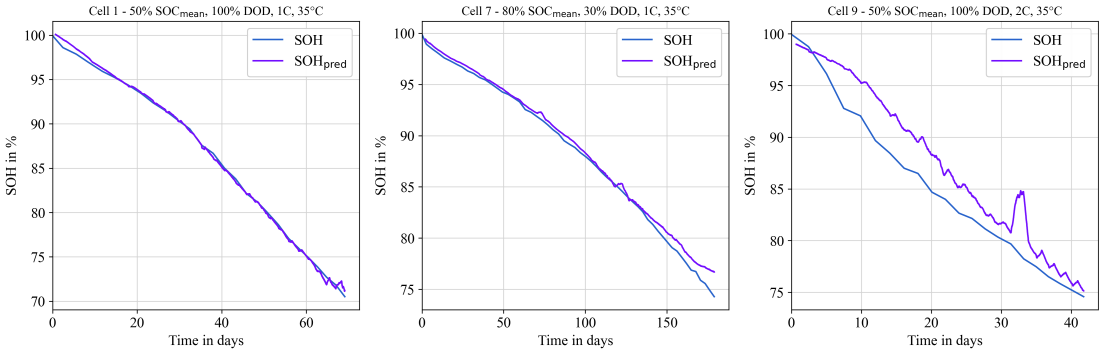

Figure 5.6: Comparison of predicted and measured SOH trajectories over time for three representative test cells. Cells 7 and 1 show the typical ageing progression most frequently observed in the dataset and are estimated accurately by the network, whereas Cell 9 was intentionally chosen as an atypical example with a pronounced U-shaped SOH curve. A post-rolling filter is applied to reduce visual noise and to make the agreement and deviation between measured and predicted trajectories easier to interpret.

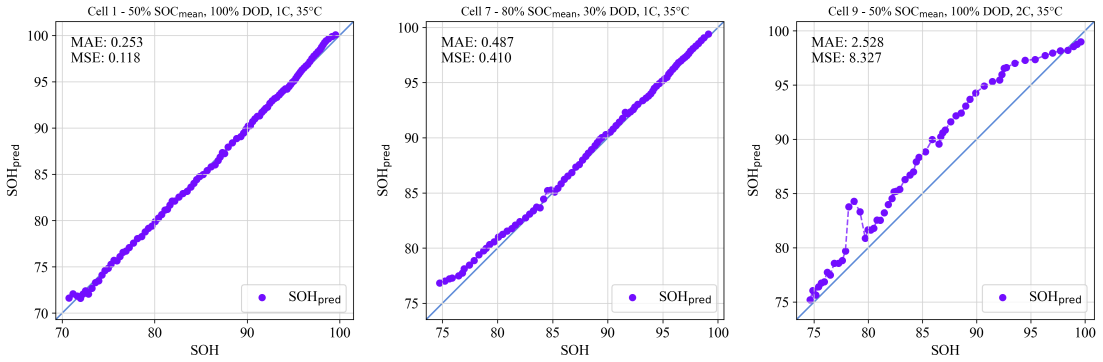

Figure 5.7: Scatter comparison between predicted and measured SOH values for the three representative test cells. Cells 7 and 1 exhibit only small dispersion and low absolute error, indicating accurate prediction, whereas Cell 9 shows substantially larger deviation in both MSE and MAE because of its atypical ageing behaviour. Error values are shown on the 0 to 100% SOH scale, with MAE in percentage points and MSE in squared percentage points.

The effect of removing the problematic cells 3 and 4 from the training basis is also examined. In that variant, the aggregate error improves only marginally to an MAE of 1.07 % (normalized MAE 0.0107) and an MSE of 2.52 %^{2} (normalized MSE $2.52 \cdot 10^{-4}$). At the same time, the behaviour on the atypical cell 9 becomes more unstable, which indicates that cleaner data alone do not replace broader data diversity. The result section therefore already makes a broader methodological point: the exact optimum for sequence length and time resolution depends strongly on the available dataset, and direct comparison with other literature remains difficult if the underlying cell data differ substantially.

5.7 Conclusion

The ageing-estimation study shows that a meaningful SOH estimate can be obtained with a comparatively simple feedforward model when the preprocessing and feature engineering are aligned with the underlying ageing problem. The central result is not only the achieved average MAE of 1.10 % and average MSE of 2.95 %^{2}, but the way in which this accuracy is reached: cumulative-current features and lag-sequence construction supply temporal and throughput-related context to an otherwise memoryless MLP. The chapter therefore demonstrates that input representation can be as important as model class, especially when the target variable

evolves slowly and is only observed directly during periodic capacity tests.

The results also define the practical boundary of the approach. The best lag-sequence setting in the investigated dataset uses 16 lag steps with a time resolution of 10 min, corresponding to a history length of 160 min, while the generally useful settings are found at intermediate temporal resolutions rather than at the shortest or longest tested intervals. This optimum is not universal. It depends on the charge and discharge durations, the operating profiles, and the available ageing trajectories. The atypical behaviour of cell 9 and the limited benefit of removing cells 3 and 4 show that neural SOH estimation is strongly constrained by data coverage and label quality. Cleaner data can reduce some error, but it cannot replace a broad dataset that contains the relevant ageing modes.

Within the dissertation, this chapter should therefore not be read as a claim that feedforward MLPs are universally preferable to recurrent or hybrid models. Its role is to establish a first data-driven baseline in which ageing-relevant temporal information is represented explicitly while the estimator architecture itself remains deliberately simple. Because the method relies on directly measured quantities such as voltage, temperature, and current-derived throughput features, the concept is not tied to one narrow cell technology in principle, but transfer to other chemistries or applications would require sufficiently rich training data and renewed validation of the lag-sequence settings.

This baseline role is important for the broader argument of the dissertation. Once a workable representation for ageing information has been identified, the next question is no longer only how well the estimator performs under nominal conditions, but how different estimator classes behave under deployment-relevant disturbances such as bias, noise, missing samples, and initialization errors. This motivates the shift toward the robustness benchmark in the following chapter. Building on this robustness perspective, the later chapters extend the analysis to recurrent LSTM and MLP models for SOC and SOH estimation, where the aim is no longer only to compare estimator concepts at a general level, but to examine whether stateful recurrent models offer advantages when temporal information has to be processed causally over long streaming sequences and ultimately transferred to embedded microcontroller execution.

Chapter 6

Robustness Benchmarking of Embedded Battery State Estimation

6.1 Motivation and practical relevance

This chapter investigates deployment-facing battery-state estimation under realistic Battery Management System conditions rather than under clean laboratory input signals alone. Reliable SOC and SOH estimation is a core BMS function because it directly shapes safety margins, usable energy, operating limits, charge control, and degradation-aware management in both mobile and stationary battery systems.[17, 20, 81] Practical deployment quality is not defined by clean-condition accuracy alone, however, because real BMS operation is exposed to sensor bias, additive noise, missing samples, irregular timing, restart-related initialization mismatch, and transient signal spikes caused by the measurement chain or by communication faults.[5, 6, 7]

In the requirement framework introduced in Figure 2.1, this chapter therefore occupies the performance branch of the dissertation. The focus is not a full production-BMS validation campaign, but the question of how estimator classes behave when realistic measurement-side disturbances are applied under otherwise matched conditions. Estimator quality is accordingly interpreted as a three-part performance problem consisting of nominal baseline accuracy, convergence behavior after state inconsistency, and robustness under disturbed measurements and measurement-continuity faults.

6.2 State of the art in battery state estimation

The relevant literature usually distinguishes between direct-measurement or integration-based estimators, typically centered on Coulomb counting, model-based estimators built around equivalent-circuit, observer-based, or electrochemical structure, hybrid approaches that combine physical structure with learned adaptation, and purely data-driven recurrent or sequence models.[18, 17, 20, 81] Direct-measurement approaches remain attractive because they are simple, computationally inexpensive, and easy to deploy, but they are structurally exposed to current bias, missing data, and initialization errors because they do not contain an internal correction mechanism.[18, 5]

Equivalent-circuit and observer-based estimators offer physical interpretability and voltage-based correction capability, but their practical performance depends strongly on parameter quality, operating-point observability, and measurement quality.[19, 20, 5] Hybrid approaches combine physical state propagation with learned adaptation of capacity, parameters, or auxiliary states and are therefore attractive for joint SOC/SOH estimation.[82, 83, 84] Such combinations can improve baseline calibration, but they do not automatically remove structural sensitivity to measurement disturbances. Data-driven recurrent estimators can exploit nonlinear relations between voltage, current, temperature, and derived online variables, and more recent work has also shown that such models can be prepared for embedded deployment through TinyML-oriented implementations, pruning, and quantization.[27, 34, 32, 85, 81] Figure 6.1 summarizes this estimator-family structure.

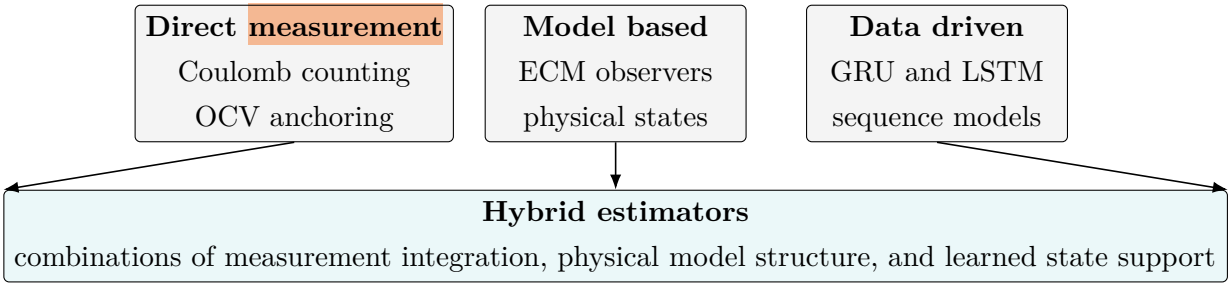

Figure 6.1: Estimator-family structure used to organize the robustness benchmark. Direct-measurement, model-based, and data-driven methods form the three basic estimator families, while hybrid estimators combine elements of these families.

6.3 Performance and embedded deployment gap

The main benchmark gap is not only that much of the literature still emphasizes clean-data accuracy, average error metrics, or computational efficiency, but also that comparatively fewer studies analyze estimator behavior under realistic measurement faults and timing disturbances.[17, 34, 32, 85] Where robustness is studied, it often remains confined to a single estimator family, so cross-family comparisons under the same disturbance protocol and under comparable embedded-ready execution conditions are still limited.[5, 6]

Broader BMS benchmarking studies have highlighted the importance of reproducible scenario design and deployment-relevant validation, but they do not by themselves close the gap between clean-data estimator comparison and measurement-level performance benchmarking across fundamentally different estimator classes.[7] This gap is especially relevant in embedded BMS design, where estimator selection must jointly consider nominal accuracy, convergence after inconsistency, robustness under disturbed measurements, and computational constraints.

6.4 Contribution and study objectives

The central objective is a measurement-only robustness benchmark in which estimator performance is isolated from broader system-level effects such as hardware-resource optimization or implementation-effort comparison. The benchmark is deliberately restricted to disturbances applied only to measurable inputs and their timing, including quantization effects, bias, additive noise, missing samples, irregular sampling, burst dropout, voltage spikes, and initialization mismatch. This design keeps the scenarios physically interpretable and allows the resulting failure modes to be compared under reproducible conditions.

The chapter compares four embedded-ready estimator classes on the same LFP ageing trajectory and under the same disturbance protocol: a direct-measurement model (DM), a hybrid direct-measurement model (HDM), a hybrid equivalent-circuit model with voltage feedback (HECM), and a data-driven recurrent model (DD). The health information is intentionally kept consistent across the SOH-aware variants. HDM uses the direct-measurement SOC update together with the shared data-driven LSTM-based online SOH estimator, HECM uses the same LSTM-based SOH support inside a voltage-corrected equivalent-circuit observer, and DD combines a GRU-based SOC estimator with the same LSTM-based SOH context. Un-

der clean conditions, HDM provides the lowest baseline error with MAE = 0.0024 (0,24 %), followed by HECM with 0.0090 (0,90 %), DD with 0.0100 (1,00 %), and DM with 0.0311 (3,11 %). Under disturbed measurements, however, the ranking changes substantially, which is the core reason for the benchmark. The benchmark should therefore be read as a controlled cross-class comparison that isolates estimator-side failure modes under matched data, matched disturbance definitions, and matched online execution rules. Its purpose is not to identify one universal winner, but to expose complementary strengths and weaknesses across estimator families and to translate them into decision-oriented guidance for embedded BMS design.

6.5 Methodology

6.5.1 Estimator classes under test

Figure 6.2 summarizes the benchmark chain from measured signals to online feature reconstruction, disturbance injection, estimator execution, and global or local performance evaluation. The comparison covers four estimator classes with deliberately different correction mechanisms: DM is the fixed-capacity Coulomb-counting reference, HDM adds capacity adaptation through the shared online SOH estimate, HECM adds voltage-residual feedback through a SOH-aware two-RC observer,[19, 20] and DD represents the compact recurrent neural estimator with stateful GRU-based SOC prediction.[27, 81]

HDM, HECM, and DD all use the same LSTM-based online SOH estimator. The benchmark therefore compares how the SOC estimators use a common health context instead of comparing different SOH backbones. The causal online quantities required by several estimators, especially cumulative charge Q_c, equivalent full cycles EFC, derivatives, and hourly SOH aggregates, are reconstructed from the measured stream during benchmark execution. This prevents precomputed dataset-side features from entering only one estimator class.

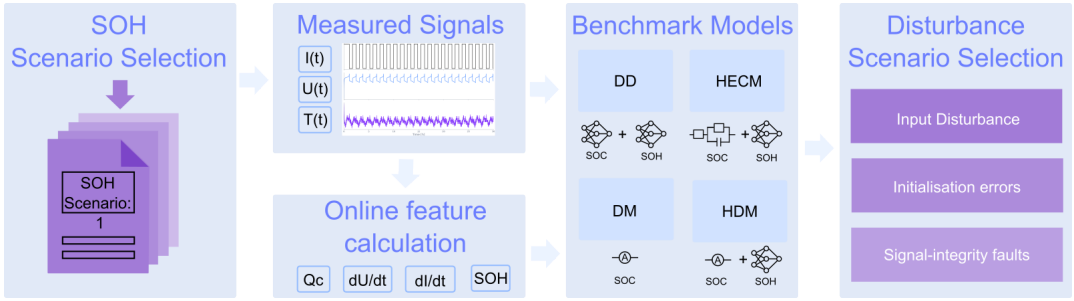

Figure 6.2: Methodology overview of the robustness benchmark. Measured current, voltage, temperature, and time signals are first converted into causal online auxiliary quantities such as cumulative charge Q_c, derivative channels, equivalent full cycles EFC, and shared SOH context. The resulting online stream is then processed by the four estimator classes under matched execution conditions, exposed to measurement-only disturbance scenarios, and finally evaluated through global and local performance metrics that separate baseline accuracy, convergence behavior, and disturbed-operation robustness.

6.5.2 SOC estimation methods

Direct-measurement model (DM)

The overview in Figure 6.2 is implemented through four estimator variants that differ in their SOC correction mechanism, but share the same causal measurement stream. The direct measurement branch first constructs the cumulative charge state

$$Q_c(k) = Q_c(k-1) + \frac{I_k \Delta t_k}{3600\,\mathrm{s\,h^{-1}}}, \tag{6.1}$$

where I_k is the measured current and Δt_k is the elapsed time between two samples. The division by $3600\,\mathrm{s\,h^{-1}}$ converts the current-time product into ampere-hours. In the DM estimator this charge state is mapped to SOC with a fixed nominal capacity,

$$\widehat{\mathrm{SOC}}_{\mathrm{DM}}(k) = \mathrm{clip}\left(1 + \frac{Q_c(k)}{C_{\mathrm{nom}}}, 0, 1\right). \tag{6.2}$$

Hybrid direct-measurement model (HDM)

The same integration core is also used in the HDM estimator. The difference is that the denominator is updated through the shared online SOH estimate,

$$C_{\mathrm{eff}}(k) = C_{\mathrm{nom}} \cdot \widehat{\mathrm{SOH}}(k), \tag{6.3}$$

which gives the hybrid direct-measurement SOC estimate

$$\widehat{\mathrm{SOC}}_{\mathrm{HDM}}(k) = \mathrm{clip}\left(1 + \frac{Q_c(k)}{C_{\mathrm{eff}}(k)}, 0, 1\right). \tag{6.4}$$

DM and HDM therefore differ only in the assumed usable capacity. Both variants keep the same constant-voltage full-charge reset logic: if the upper-voltage condition is sustained for the configured duration, the internal charge state is reset to $Q_c = 0$ and the SOC estimate is anchored at full charge.

Hybrid equivalent-circuit model (HECM)

The HECM extends the current-driven charge balance with voltage-residual correction. It is implemented as a two-RC observer with state vector

$$\mathbf{x}_k = \begin{bmatrix} \widehat{\mathrm{SOC}}_k & U_{1,k} & U_{2,k} \end{bmatrix}^\top, \tag{6.5}$$

where $U_{1,k}$ and $U_{2,k}$ denote the polarization voltages of the two RC branches. The prediction step uses the previous current sample and propagates the state according to

$$\mathbf{x}_k^- = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{b}_k I_{k-1}, \tag{6.6}$$

with

$$\mathbf{A}_k = \mathrm{diag}\left(1, e^{-\Delta t_k/\tau_1}, e^{-\Delta t_k/\tau_2}\right), \qquad \mathbf{b}_k = \begin{bmatrix} \eta \Delta t_k / C_b \\ R_1\left(1 - e^{-\Delta t_k/\tau_1}\right) \\ R_2\left(1 - e^{-\Delta t_k/\tau_2}\right) \end{bmatrix}. \tag{6.7}$$

Here $C_b = C_0 \widehat{\mathrm{SOH}}$ is the SOH-scaled capacity used by the observer. The voltage prediction used for the correction step is

$$\hat{U}_k = \mathrm{OCV}\left(\widehat{\mathrm{SOC}}_k, \widehat{\mathrm{SOH}}_k, I_k\right) + U_{1,k} + U_{2,k} + R_i I_k. \tag{6.8}$$

The Kalman correction then uses the residual between this predicted terminal voltage and the measured voltage to update the internal state. The parameters R_i, R_1, R_2, τ_1, τ_2, OCV, and dOCV/dSOC are not treated as constants. Instead, they are interpolated from pre-identified SOC/SOH lookup surfaces with separate charge and discharge parameterizations. This is the main structural reason why HECM can re-anchor the integrated SOC estimate through voltage information while still using the same online SOH context as HDM and DD.

Data-driven SOC model (DD)

The DD estimator uses a stateful GRU instead of an explicit observer equation. At each online time step it receives the feature vector

$$\mathbf{z}_k = \begin{bmatrix} U_k & I_k & T_k & \widehat{\mathrm{SOH}}_k & Q_c(k) & \frac{\mathrm{d}U}{\mathrm{d}t}(k) & \frac{\mathrm{d}I}{\mathrm{d}t}(k) & \Delta t_k \end{bmatrix}^\top . \tag{6.9}$$

Figure 6.3(a) shows the block structure of this model. In its floating-point base configuration, a single GRU layer with hidden size 96 processes the eight-channel feature vector. An MLP head with one hidden layer of size 96, ReLU activation, and a sigmoid output then maps the recurrent state to a SOC estimate constrained to $[0, 1]$. The benchmark itself uses the deployment-prepared variant of this model, in which structured pruning reduces the hidden size from 96 to 67 and the weights are stored as INT8 values (Section 6.5.4). During training, the recurrent core is exposed to causal sequence segments of

$$L_{\mathrm{SOC}} = 2024 \tag{6.10}$$

consecutive time steps. During benchmark execution the same model is run statefully and causally, so the hidden state is carried forward sample by sample and the MLP head produces a continuous SOC trajectory without access to future samples.

The derivative channels in the DD input are rebuilt online from the disturbed measurement stream,

$$\frac{\mathrm{d}I}{\mathrm{d}t}(k) = \frac{I_k - I_{k-1}}{\Delta t_k}, \qquad \frac{\mathrm{d}U}{\mathrm{d}t}(k) = \frac{U_k - U_{k-1}}{\Delta t_k}. \tag{6.11}$$

Together with the explicit Δt_k channel, these quantities allow the neural model to distinguish local voltage or current dynamics from timing changes under irregular sampling. The inclusion of $Q_c(k)$ also ensures that the DD model receives the same charge-throughput information that drives DM and HDM.[27, 81]

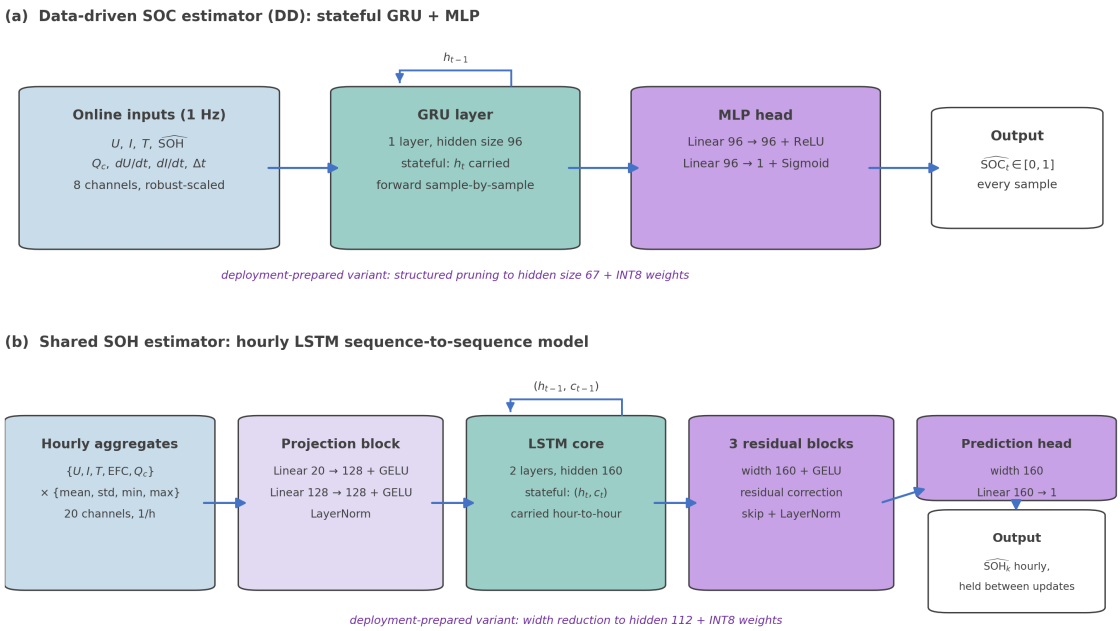

Figure 6.3: Block-level architectures of the data-driven estimator components used in the benchmark. (a) The DD SOC estimator processes the eight online input channels with a single stateful GRU layer of hidden size 96, followed by an MLP head with one ReLU-activated hidden layer of size 96 and a sigmoid output. (b) The shared SOH estimator forms a 20-dimensional hourly feature vector by computing four statistics for each of the five causal base signals $\{U, I, T, \text{EFC}, Q_c\}$, projects this vector through a two-layer feature-projection block of width 128, processes the projected sequence with a two-layer stateful LSTM of hidden size 160, and maps the recurrent representation through three residual MLP blocks and a prediction head of width 160. The deployment-prepared variants used in the benchmark are described in Section 6.5.4.

6.5.3 SOH estimation method

HDM, HECM, and DD all use the same LSTM-based SOH estimator. Its input is reconstructed causally from the five online base signals $\{U, I, T, \text{EFC}, Q_c\}$. For every hourly update, the mean, standard deviation, minimum, and maximum are computed over the samples contained in the respective hour. The resulting SOH input is therefore not a set of 20 independently measured channels, but a 20-dimensional feature vector obtained from five base signals and four aggregation statistics per signal.[86, 87, 28, 1]

Figure 6.3(b) summarizes the architecture. The first stage is a learned feature-

projection block with two linear layers of width 128, layer normalization, GELU activation, and dropout. This block transforms the heterogeneous hourly statistics into a task-specific representation before recurrent processing. The second stage is a two-layer unidirectional LSTM with hidden size 160. It is operated strictly causally: hidden and cell states are forwarded from one hourly aggregate to the next, and no future measurements are used. The third stage consists of three residual MLP blocks with width 160. In compact form, each residual block refines the recurrent representation as

$$\mathbf{y} = \mathrm{LayerNorm}(\mathbf{x} + \mathrm{Dropout}(W_2\,\mathrm{GELU}(W_1\mathbf{x}))), \tag{6.12}$$

so that the block learns a correction to the LSTM representation instead of replacing it completely. A final prediction head of width 160 maps the refined representation to one scalar SOH estimate for the current hourly step.

During benchmark execution the first SOH output is anchored with the configured initial SOH value, and later estimates are held piecewise constant between hourly updates. This common health context is then consumed differently by the SOC estimators: HDM uses it through C_{eff}, HECM uses it in the observer capacity and lookup scheduling, and DD receives it as one channel of its recurrent SOC input. The deployment-prepared SOH-LSTM variant used for the footprint-constrained benchmark is described in Section 6.5.4.

6.5.4 Embedded-oriented model preparation

The benchmark focuses on estimator behavior under disturbed measurements, but the compared variants are still constrained by an embedded reference budget. All four estimator classes are selected and prepared so that they remain deployable on a common STM32H755ZI-class target with a conservative 1 MB flash and 1 MB RAM reference limit. This prevents an unfair comparison between compact analytical estimators and neural models that would only work as large offline networks.

DM requires no compression because it consists only of the scalar charge-state update and the associated reset logic. HECM also does not require neural-network compression, but its flash demand is dominated by the two SOH×SOC lookup grids for charge and discharge parameterization of the ECM observer. The neural components used by HDM and DD are deployment-prepared before the robustness benchmark. For the SOC-GRU, structured width reduction and short finetuning

reduce the hidden size from 96 to 67, followed by INT8 weight quantization. For the SOH-LSTM, the deployment-prepared variant uses hidden size 112 after width reduction, finetuning, and INT8 quantization. The resulting reusable component footprints are listed in Table 6.1.

The table is a component-level estimate rather than a list of the four complete estimator variants. The complete variants are obtained by combining the SOC core used by each estimator with the shared SOH-LSTM where applicable: DM consists only of the Coulomb-counting core, HDM combines the Coulomb-counting core with the SOH-LSTM, HECM combines the 2RC observer with the SOH-LSTM, and DD combines the SOC-GRU with the SOH-LSTM. This gives lower-bound persistent footprints of 4.2 KiB flash and 16 B RAM for DM, 412.5 KiB and 1808 B for HDM, 850.8 KiB and 1844 B for HECM, and 436.2 KiB and 2060 B for DD. These RAM values count only architecture-intrinsic persistent numerical states, not temporary buffers, stack usage, framework overhead, or complete runtime memory. They therefore provide a strict lower bound, but they show that all four deployment-prepared benchmark variants stay below the 1 MB flash and RAM reference limits.

Table 6.1: Estimated persistent flash and RAM footprints of the reusable estimator components used in the deployment-prepared benchmark variants. For the recurrent neural components, flash estimates refer to the structured-pruned, finetuned, and INT8-quantized versions. RAM* counts only architecture-intrinsic persistent numerical states. Temporary buffers, framework overhead, stack usage, and system-level runtime memory are excluded.

Component	Flash [KiB]	RAM* [B]	Basis
DM core (Coulomb counting)	4.2	16	Four persistent scalar states in the online update logic
SOC-GRU core	27.9	268	One recurrent hidden state with size 67 stored as float32
SOH-LSTM core	408.3	1792	Two-layer LSTM with hidden and cell states of size 112 stored as float32
HECM core (2RC observer)	442.5	52	Flash dominated by two SOH×SOC parameter tables for charge and discharge parameterization. RAM: observer state $x \in \mathbb{R}^3$, covariance $P \in \mathbb{R}^{3\times3}$, previous current

6.5.5 Performance benchmark design

The benchmark perturbs only signals that a BMS controller would actually receive from its sensors: current, voltage, temperature, sample availability, and sampling timing. Model-internal quantities, such as LSTM hidden states or the model weights themselves, are never modified directly. This restriction keeps all disturbance scenarios physically interpretable: every failure mode tested corresponds to something that can go wrong at the sensor, wiring, or communication layer of a real embedded system, rather than to artificial tampering with the estimator internals.[7]

Figure 6.4 separates the resulting cases into input disturbances, initialization mismatch, and measurement-continuity faults, meaning disturbances of sample availability, sample timing, or frozen data updates. The numerical levels in Table 6.2 combine three roles: realistic acquisition limits, adverse but plausible measurement disturbances, and deliberate stress cases for recovery analysis.[12, 14, 16, 15, 88, 89] During every run, all four estimators receive the identical disturbed sensor time series, namely the same modified sequence of voltage, current, temperature, and timing samples, so that any difference in output is attributable to estimator structure rather than to differences in the input. Each estimator then reconstructs its auxiliary quantities (cumulative charge Q_c, equivalent full cycles EFC, current derivative, hourly SOH aggregates) causally and online from this disturbed time series, not from precomputed clean dataset columns. A sensor bias therefore propagates into the derived features exactly as it would in a real BMS, which keeps the comparison physically consistent across all estimator classes.

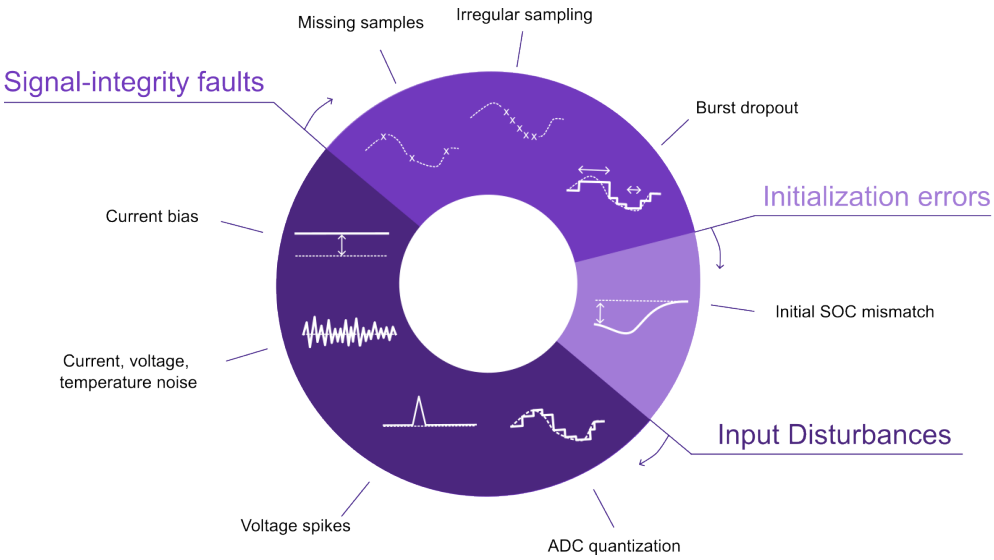

Figure 6.4: Disturbance taxonomy used in the robustness benchmark. Input disturbances, initialization mismatch, and measurement-continuity faults are separated so that drift, loss of correction, delayed recovery, and transient instability can be interpreted as different estimator-side effects.

Table 6.2: Scenario protocol used in the robustness benchmark. The entries define the fixed disturbance settings and the practical motivation behind each measurement or continuity/timing case.

Scenario	Benchmark setup	Practical rationale
ADC quantization	$\Delta I = 0{,}01\,\mathrm{A}$, $\Delta U = 0{,}005\,\mathrm{V}$, $\Delta T = 0{,}5\,^{\circ}\mathrm{C}$	Deployment-level discretization, sensor scaling, and rounding effects in embedded acquisition chains.[12, 14, 13]
Current bias	Constant $+0{,}05\,\mathrm{A}$ in the compact matrix, with an additional sweep of 0.5%, 1.5%, and 3.0% of $\lvert I\rvert_{\max}$	Sensor offset, gain error, miscalibration, and long-term drift stress on integration-driven estimators.[14, 16, 15]
Current noise	Gaussian noise with $\sigma_I = 0{,}10\,\mathrm{A}$ in the compact matrix, with a local-detail sweep up to $\sigma_I = 0{,}20\,\mathrm{A}$	Adverse current-measurement noise and noise-driven disturbance propagation.[5, 6]
Voltage noise	Additive Gaussian noise with $\sigma_U = 0{,}01\,\mathrm{V}$	Wiring noise, front-end noise, insufficient filtering, and EMI-sensitive correction channels.[5, 12, 88]
Temperature noise	Additive Gaussian noise with $\sigma_T = 1{,}0\,^{\circ}\mathrm{C}$	Sensor tolerance, conversion noise, thermal gradients, and sensor-to-cell mismatch.[13, 84]
Initial SOC mismatch	Additive initial mismatch of -0.10 SOC, corresponding to a 10-percentage-point offset. In DD, this is realized through perturbed online Q_c	Restart conditions, missing recent history, or weak initial anchoring under practical deployment.[5, 19]
Missing samples	Every 50th sample frozen on the nominal 1 s grid	Packet loss, blocked updates, logger dropouts, or periodic frozen-data events.[6, 89]
Irregular sampling	Uniform timing jitter of $\pm 0{,}1\,\mathrm{s}$ around the nominal 1 s grid	Scheduling jitter, bus latency, and asynchronous acquisition timing.[7, 6, 89]
Burst dropout	One central frozen gap of 3600 s	Communication interruption, frozen tasks, or severe delayed-information recovery stress.[7, 6]
Voltage spikes	One single-sample spike of $\pm 0{,}20\,\mathrm{V}$ every 1000 samples (1000 s)	Transient voltage-sample corruption caused by EMI or switching disturbances.[88]

The compact scenario matrix gives one common protocol for the main result figures. Current bias and current noise are additionally examined through local sweeps, while ADC quantization remains included as a deployment-level sensing case even though it is not expected to dominate the ranking. In this form, the benchmark is a controlled estimator comparison under matched disturbances, not a complete validation of a production BMS.

6.5.6 Metrics and evaluation criteria

The benchmark evaluates three performance dimensions: nominal accuracy, convergence behavior after state inconsistency, and robustness under both longer-lasting and short-term measurement disturbances. These three dimensions are not independent post-hoc labels, but the aggregation logic used later in the result interpretation and in the decision-oriented synthesis. Nominal accuracy is read from the clean baseline run, robustness is read from the additional error caused by each disturbed scenario relative to that estimator's own baseline, and convergence is read from the time needed to return to a baseline-related error band after an imposed inconsistency or interrupted signal stream.

For each estimator m and disturbed scenario s, the main global robustness penalty is therefore defined as

$$\Delta\mathrm{MAE}_{m,s} = \mathrm{MAE}_{m,s} - \mathrm{MAE}_{m,\mathrm{base}}. \tag{6.13}$$

Positive values indicate that the disturbance degrades the estimator relative to its own clean-operation reference. Values close to zero indicate that the disturbance does not materially change the full-trajectory average error. Global disturbed-operation performance is quantified through MAE, RMSE, bias, maximum error, and the 95th percentile absolute error. MAE and RMSE describe the average error level, bias describes systematic over- or underestimation, the maximum error reports the single largest deviation, and P95 is the absolute-error value that 95 % of all evaluated samples do not exceed. P95 therefore describes the larger-error range of the trajectory without being determined by the single worst sample.

Local disturbance performance is additionally analysed through scenario-specific quantities such as jump counts, output variance, absolute-error variance, excess rolling MAE, recovery time, and aligned post-disturbance error, because several scenarios do not primarily manifest as a collapse of full-run MAE but as short-

term output fluctuations, delayed drift, or slow re-synchronization. For scenarios in which the affected samples or time interval are explicitly known, the analysis separates the trajectory into pre-disturbance, disturbance, and post-disturbance phases. This makes it possible to quantify error growth during the disturbance, recovery time after the event, and residual error after the input stream has returned to normal. In the current benchmark phase, ADC quantization, current bias, voltage noise, temperature noise, missing samples, and irregular sampling are interpreted mainly through global metrics, whereas burst dropout, voltage spikes, current-noise detail, and initialization mismatch additionally require local evidence because their most relevant effects are episodic, recovery-related, or strongly localized in time.

Recovery after initialization errors is interpreted through two baseline-relative operating bands rather than through one universal fixed threshold. This keeps the recovery criterion tied to the normal clean-condition performance level of each estimator and avoids judging models with very different baseline accuracy against the same absolute tolerance. The strict band is defined as

$$B_{\text{strict}} = \max(\text{P95}_{\text{base}},\, 1.5 \cdot \text{MAE}_{\text{base}})\,. \tag{6.14}$$

In the strict criterion, P95_{base} ensures that the recovery band includes the larger errors that already occur during clean baseline operation, while $1.5 \cdot \text{MAE}_{\text{base}}$ provides a minimum margin around the average baseline error. Taking the maximum of both terms means that recovery is judged against the more tolerant of these two baseline-derived limits, so recovery under this band means re-entry close to the normal baseline regime without imposing unrealistically narrow limits. The fair band is

$$B_{\text{fair}} = \max(2 \cdot \text{P95}_{\text{base}},\, 3 \cdot \text{MAE}_{\text{base}},\, 0.02)\,, \tag{6.15}$$

and acts as a more deployment-tolerant criterion for estimators that re-enter a practically acceptable operating range without immediately returning to their strict near-baseline regime. The lower bound of 0.02 avoids unrealistically tight fair-band thresholds for very accurate baseline models. Both recovery bands require sustained re-entry over a fixed horizon so that the reported recovery times reflect stable resynchronization rather than an accidental short crossing of the threshold.

Reporting both bands avoids making the recovery conclusion depend on one manually chosen limit. If an estimator returns only to the fair band, it has reached a practically acceptable range but not its near-baseline regime. If it also returns to the

strict band, the recovery is stronger. This multi-scale metric design therefore evaluates performance as a structured combination of nominal accuracy, convergence behavior, global degradation, local disturbance response, and long-term drift.

Table 6.3: Connection between raw benchmark metrics, result figures, and the three performance dimensions used in the robustness interpretation. The table makes explicit how the later statements on accuracy, robustness, and recovery/convergence are derived from the same metric set rather than from separate evaluation criteria.

Dimension	Primary metric evidence	Role in the result interpretation	Typical result sections
Nominal accuracy	Baseline MAE, RMSE, P95 absolute error, maximum error, and bias from the undisturbed run	Quantifies clean-sensing estimator quality before any robustness penalty is applied. This is why an estimator can lead in accuracy even if it is not the most robust under disturbances.	Baseline performance and decision-oriented accuracy score
Disturbed-scenario robustness	$\Delta\text{MAE}_{m,s}$ relative to the estimator's own baseline, complemented by disturbed-window MAE, P95 error, maximum error, bias, jump counts, and local excess rolling MAE where needed	Quantifies how strongly each disturbance changes the estimator's behavior. This supports scenario-specific robustness claims instead of ranking models only by absolute disturbed MAE.	Current bias, noise sensitivity, continuity/timing faults, voltage spikes, cross-scenario comparison
Convergence and recovery	First sustained re-entry into the strict or fair baseline-related error band after initialization mismatch or signal interruption. Non-recovery inside the inspected window is treated as weakest evidence	Quantifies whether an estimator can return to a useful operating regime after state inconsistency. This explains why recovery behavior is evaluated separately from both clean accuracy and global disturbed-operation MAE.	Initialization mismatch, burst-dropout recovery, decision-oriented recovery score

For the decision-oriented synthesis in Section 6.8.3, the same logic is condensed into relative lower-is-better scores across the four estimator classes. The accuracy score is based on the clean baseline errors, the robustness score is based on ΔMAE across the disturbed scenarios, and the recovery score is based on recovery time after initialization mismatch. This aggregation is used only as a compact decision aid. The individual scenario figures and local recovery analyses remain the primary evidence for the detailed conclusions.

6.6 Experimental Setup

6.6.1 Dataset and data acquisition

The benchmark uses the LFP DoE Cycle Ageing Dataset described in Section 4.1.[49] It provides high-resolution ageing-test records from repeated cycling and check-up phases that extend from early-life operation into low-SOH operation, including sections below the common 80 % end-of-life reference level. This makes the dataset suitable for reproducible disturbance injection and evaluation-label construction over a broad degradation range. In the current benchmark phase, all estimator classes are evaluated on the same representative ageing trajectory from one LFP cell and under the same disturbance protocol so that performance differences can be attributed to estimator structure under matched data and label definitions before extending the study to additional cells and cross-cell validation.

The available raw channels are current, voltage, temperature, and time. Feature engineering converts these measurements into two separate data types: evaluation labels and causal online features. The labels are derived from capacity-test intervals and physically motivated SOC reset criteria and contain the capacity, SOH, and SOC traces used for benchmark scoring. The online features are reconstructed during each benchmark run from the same input stream that each estimator receives. They include the cumulative charge state Q_c, equivalent full cycles EFC, derivative channels such as dU/dt and dI/dt, and hourly SOH-context aggregates. For these hourly aggregates, the online channels $\{U, I, T, \mathrm{EFC}, Q_c\}$ are summarized by mean, standard deviation, minimum, and maximum.

6.6.2 Evaluation labels and online feature reconstruction

For label construction, the signed transferred charge is computed sample-wise as

$$Q_k = \frac{I_k \Delta t_k}{3600}, \tag{6.16}$$

and the capacity label used as evaluation reference is obtained from the absolute transferred charge during dedicated capacity-test intervals,

$$C_{\mathrm{ref}} = \sum_{k \in \mathcal{T}_{\mathrm{cap}}} |Q_k| \,. \tag{6.17}$$

The SOH label used as evaluation reference is then defined as

$$\mathrm{SOH}_{\mathrm{ref}} = \frac{C_{\mathrm{ref}}}{C_{\mathrm{ref},0}}, \tag{6.18}$$

where $C_{\mathrm{ref},0}$ denotes the first measured reference capacity of the trajectory, and the corresponding capacity and SOH columns are interpolated between the dedicated capacity-test intervals. The SOC label used as evaluation reference follows from Coulomb counting,

$$\mathrm{SOC}_{\mathrm{ref}} = 1 + \frac{Q_c}{C_{\mathrm{ref},0}}. \tag{6.19}$$

During preprocessing, the cumulative charge state is reset to $Q_c = 0$ only after the voltage has remained close to the upper voltage limit for the configured CV duration. In the benchmark implementation, this duration is 300 s. This prevents a short voltage excursion from being interpreted as a full-charge point. At the lower-voltage limit, Q_c is reset to $-C_{\mathrm{ref},0}$.

In the benchmark implementation, the online EFC feature is reconstructed from absolute charge throughput,

$$Q_{\mathrm{thr}}(k) = \sum_{j=1}^{k} \frac{|I_j| \Delta t_j}{3600}, \qquad \mathrm{EFC}(k) = \frac{Q_{\mathrm{thr}}(k)}{C_{\mathrm{nom}}}. \tag{6.20}$$

Here C_{nom} is the nominal cell capacity used in the benchmark feature pipeline. Unlike the SOC and SOH labels, EFC is an online feature and is therefore recomputed from the current and timing stream of each benchmark run.

This distinction between labels and online features is central to the benchmark. The labels remain fixed reference values for scoring the online estimates, whereas Q_c, EFC, dU/dt, dI/dt, and the hourly aggregates are rebuilt from the same clean

or disturbed inputs that are passed to the estimators. Consequently, disturbed measurements affect the estimator inputs and derived features, but not the reference labels used for scoring.

6.6.3 Test trajectory and benchmark scenarios

All estimator classes are run on the same benchmark trajectory and the same scenario protocol from Table 6.2. This ensures that differences in the results are attributable to estimator structure and disturbance propagation rather than to different data splits or label definitions. The reported scenarios cover the nominal baseline, current bias, additive current noise, voltage noise, temperature noise, initial SOC mismatch, missing samples, irregular sampling, burst dropout, voltage spikes, and ADC quantization. Together they probe long-term integration drift, voltage-correction sensitivity, disturbance propagation through derived features, frozen updates, timing uncertainty, and local post-event recovery. Input disturbances are applied directly to measured channels, initialization errors are injected only where structurally meaningful, and continuity/timing cases manipulate sample availability or timing while preserving online execution logic.

6.6.4 Implementation details

All compared estimators are executed under online conditions. Disturbed inputs are processed sample by sample, and the required auxiliary quantities are rebuilt directly from the disturbed signals rather than copied from precomputed dataset columns. This applies in particular to Q_c, EFC, dU/dt, dI/dt, and the hourly SOH aggregates, so that no hidden dataset-side variables leak into the benchmark runs.

For the recurrent neural estimators, the benchmark uses the intended stateful embedded execution mode. The SOH-LSTM carries its hidden and cell states from one hourly update to the next, while the SOC-GRU preserves its recurrent state along the sample-wise measurement stream. The neural variants used in the benchmark correspond to the compact pruned-and-quantized architectures introduced above. In freeze and dropout scenarios, disturbance propagation is handled consistently by freezing or distorting the derived online features together with the corresponding measured inputs, which is necessary for a physically meaningful comparison between model classes.

The direct-measurement estimators include the same sustained-CV full-charge

reset criterion that is also used to define the full-charge reference point in preprocessing, whereas the data-driven initial-SOC test is implemented through an initial offset of the online Q_c feature because the neural estimator does not expose an explicit SOC state variable. The simulation environment therefore keeps the base nominal capacity used for online feature reconstruction fixed across all estimator classes, while all models receive the same disturbed input streams and are evaluated against the same reference labels and global and local metrics. This is one of the main reasons why the benchmark remains close to later embedded deployment: the estimators are not evaluated as offline regressors, but as causal state estimators under matched measurement-side constraints.

6.7 Results

The following evaluation uses the clean benchmark trajectory as the reference point and then examines how each estimator changes when the same online signal stream is affected by bias, noise, missing data, timing faults, spikes, ADC quantization, or initialization mismatch. The results are therefore organized not only by global error level, but also by local recovery and transient behaviour, because deployment quality depends on both the magnitude of the error and the way in which it develops over time. Numerical SOC errors are kept on the normalized 0 to 1 scale used in the benchmark tables and figures, while percentage-point equivalents are given in the prose where they support interpretation.

6.7.1 Baseline performance

Under clean measurements, the hybrid direct-measurement model is the nominal-accuracy leader on the selected ageing benchmark trajectory. It reaches MAE = 0.0024 (0,24 %), followed by HECM with 0.0090 (0,90 %), DD with 0.0100 (1,00 %), and DM with 0.0311 (3,11 %). This baseline ranking is the reference point for the later ΔMAE comparisons and recovery bands, but it is not yet a deployment recommendation by itself. The complete baseline metric set, including RMSE, P95 error, maximum error, and signed bias, is listed in Appendix Table A.1. Figure 6.5 visualizes the same clean-condition comparison.

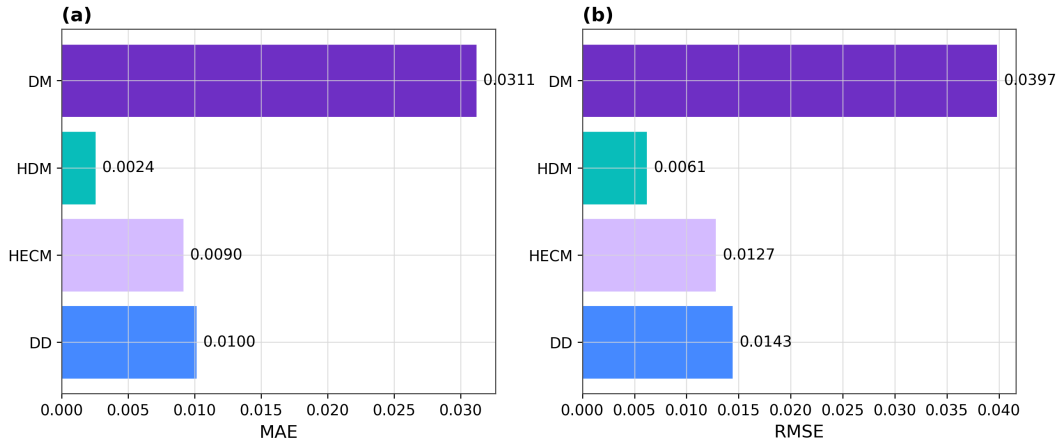

Figure 6.5: Baseline performance on the selected ageing benchmark trajectory. The summary compares the nominal MAE, RMSE, 95th-percentile error, and signed bias of the four estimator classes and therefore provides the clean-condition reference point for all later disturbed-scenario comparisons. The hybrid direct-measurement model provides the lowest baseline error, whereas the direct-measurement model shows the largest nominal deviation. Error values shown in the figure are SOC fractions on a 0 to 1 scale. Multiplication by 100 gives percentage points.

6.7.2 Current-bias sensitivity

Persistent current bias is the most severe integration stress in the benchmark. The dedicated sweep applies offsets of 0.5%, 1.5%, and 3.0% of the maximum absolute current of the benchmark trajectory, so the percentages describe the bias magnitude relative to $|I|_{\max}$ rather than an error percentage of SOC. Across this sweep, HECM remains the most robust estimator, DD is intermediate, and DM and HDM degrade strongly because the biased current accumulates directly through their integration path. For the strongly biased case reported in the compact scenario matrix, HECM reaches MAE = 0.0932, whereas DD, HDM, and DM increase to 0.2859, 0.3068, and 0.3143, respectively. The full MAE, RMSE, and P95 values are collected in Appendix Table A.2. Figure 6.6 shows the corresponding local bias example, sweep trend, and trajectory divergence.

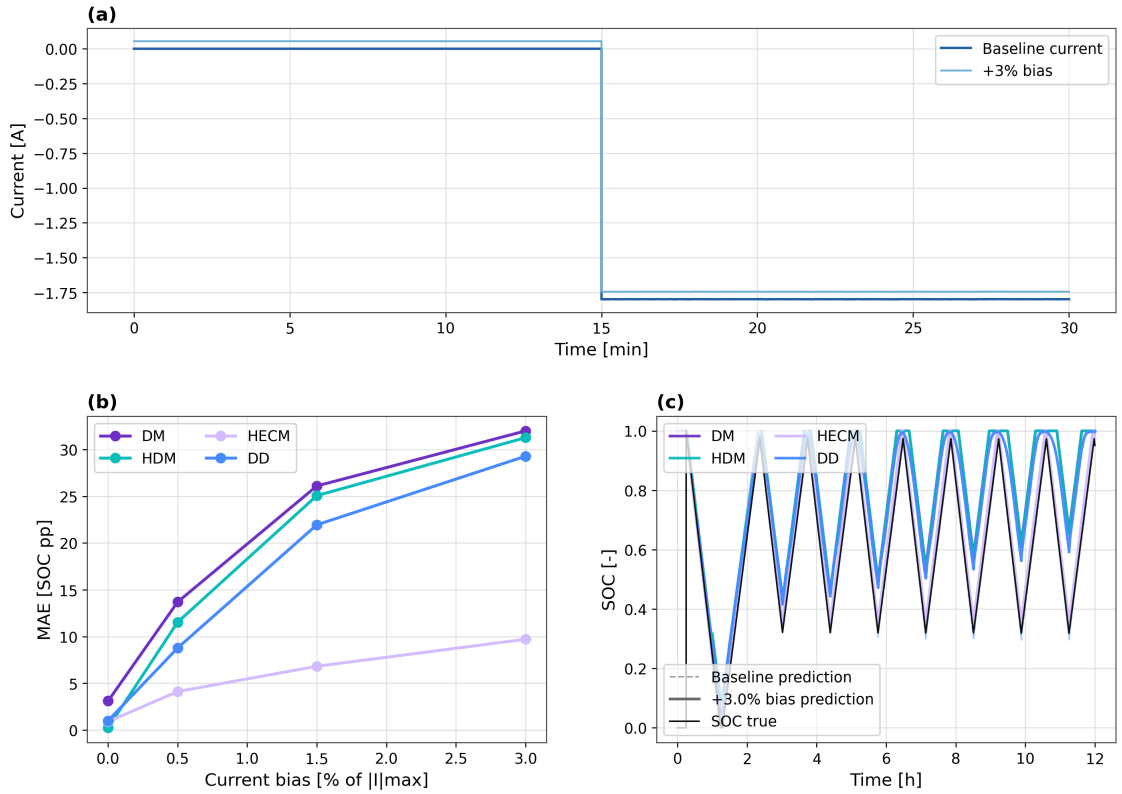

Figure 6.6: Sensitivity to current-measurement bias. The figure combines a representative local example of the applied bias, the global degradation of SOC accuracy across the matched bias sweep, and the resulting SOC divergence over time for a strongly biased case. Together, these views show why persistent current bias clearly separates the integration-based estimators from the voltage-corrected observer variant.

6.7.3 Noise sensitivity

The noise block is interpreted separately from the current-bias scenario because the injected noise terms are zero-mean disturbances rather than persistent offsets. The relevant question is therefore not only whether SOC drifts over the full trajectory, but also how a corrupted measurement channel propagates into observer correction, neural input features, and local output variability. For current noise, this propagation occurs through the measured current itself and through current-derived quantities such as Q_c, $\mathrm{d}I/\mathrm{d}t$, EFC, and the hourly SOH aggregation. In the compact benchmark matrix, the high-current-noise case uses $\sigma_I = 0{,}10\,\mathrm{A}$. The additional sweep in Figure 6.7 extends beyond this value to make local output-variability trends visible. Under the high-current-noise case, HECM shows the

largest global penalty with $\Delta\text{MAE} = 0.0075$, whereas HDM, DD, and DM change by 0.0019, 0.0009, and -0.0001, respectively. This does not mean that current noise is irrelevant for the other estimators. Rather, it shows that zero-mean noise mainly appears through local variability and disturbance propagation instead of through the same long-term integration drift caused by persistent bias.

Voltage noise with $\sigma_U = 0{,}01\,\text{V}$ mainly tests the voltage-correction path in HECM and the voltage-dependent input path in the learned estimators. In this scenario HECM remains essentially unchanged at MAE = 0.0090, while DM, HDM, and DD increase to 0.0524, 0.0751, and 0.0698, respectively. Temperature noise with $\sigma_T = 1{,}0\,^{\circ}\text{C}$ is weaker in the global metrics for DM and HECM, but it still affects the SOH-aware paths because temperature enters the learned health context: HDM increases to MAE = 0.0067 and DD to 0.0181. The numerical global and local noise metrics are given in Appendix Tables A.2 and A.3.

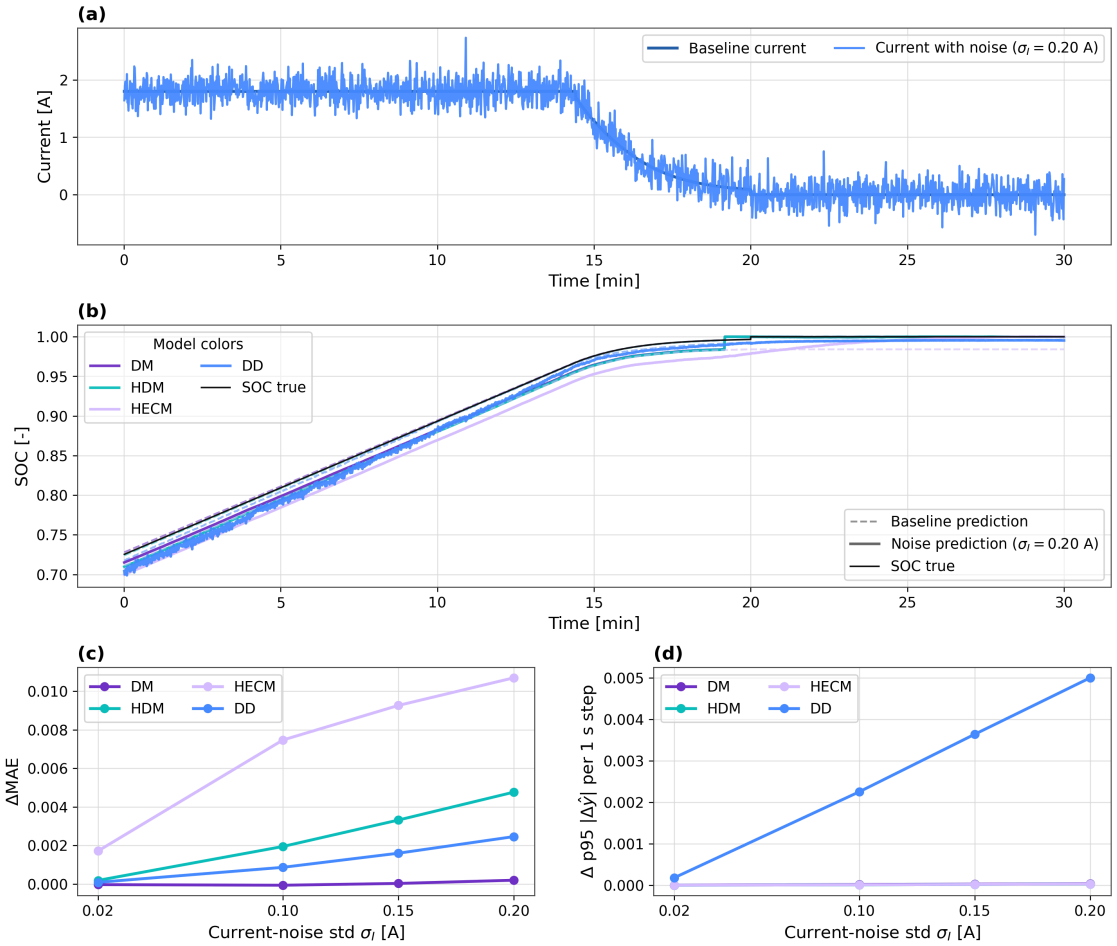

Figure 6.7: Noise sensitivity under additive current noise. The figure combines a representative disturbed window, a matched SOC comparison in the same interval, the global MAE penalty in the compact benchmark matrix, and the local output-variability trend across an extended current-noise sweep. This makes visible that zero-mean current noise is mainly a local disturbance-propagation test rather than the same type of long-term drift test as persistent current bias.

6.7.4 Initialization error and recovery

Initialization mismatch is a recovery problem rather than a simple full-run MAE problem, because the relevant question is whether the estimator can re-synchronize after starting from a wrong SOC value. The benchmark therefore uses a reset-free evaluation window and injects an initial mismatch of -0.10 on the normalized SOC scale, meaning that the online estimate starts 10 percentage points too low. This disturbance represents a deliberately visible restart inconsistency, for example after

missing recent history or weak anchoring of the initial SOC. For DD, the mismatch is implemented as an offset of the online cumulative-charge feature instead of an artificial reset of an inaccessible latent state, which keeps the test aligned with deployment-time observables.

Recovery is evaluated with the strict and fair baseline-related bands defined in Section 6.5.6. These thresholds are not times and not fixed SOC setpoints. They are admissible absolute SOC errors on the normalized 0 to 1 scale. They differ between estimator classes because they are computed from each estimator's own clean-condition MAE and P95 error. The strict band marks return to the near-baseline regime, while the fair band is a more tolerant operating range. In the inspected window, DD returns to the strict band after about 1.05 h and to the fair band after about 1.00 h. DM returns after about 1.17 h and 0.25 h, respectively. HECM returns after about 2.37 h and 1.16 h. HDM does not re-enter either band. The exact threshold values and recovery times are listed in Appendix Table A.3. Figure 6.8 shows the corresponding trajectory-level recovery behavior.

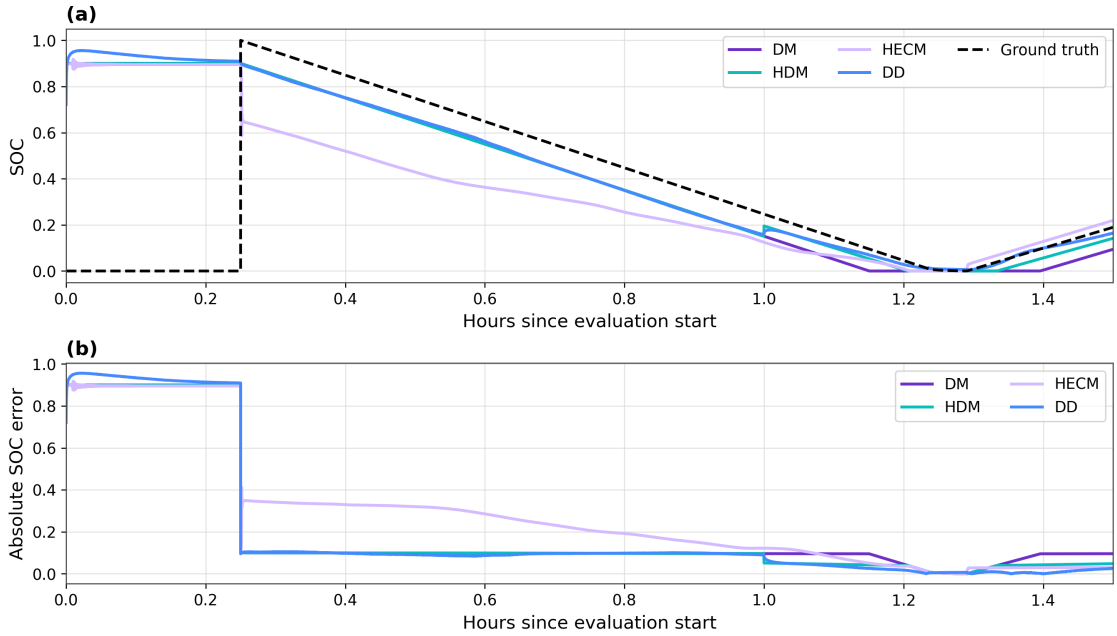

Figure 6.8: Recovery after initialization mismatch in a reset-free evaluation window. The figure shows the disturbed SOC trajectories after the imposed initial-state error together with the corresponding absolute SOC error over time, so that re-synchronization to the baseline-relative recovery bands becomes directly visible. This local view is required because full-run averages alone do not capture restart behavior adequately.

6.7.5 Missing samples, irregular sampling, and burst dropout

This block covers disturbances that affect the continuity and timing of the measurement time series rather than the amplitude of one sensor channel alone. In this context, signal integrity means that the sampled input stream remains complete, time-consistent, and updated. It does not refer only to electrical signal quality. The three cases therefore belong together: individual samples can be frozen, the sampling interval can jitter, or the input stream can be held constant over a longer dropout window.

Missing samples are the strongest global case in this block when the effect is measured by ΔMAE relative to each estimator's own baseline. Under the every-50th-sample freeze, DD changes least with $\Delta\text{MAE} = 0.0016$, whereas DM, HDM, and HECM show larger increases of 0.0076, 0.0079, and 0.0029, respectively. Irregular sampling with $\pm 0{,}1\,\text{s}$ timing jitter remains nearly neutral in the present implementation, with ΔMAE values close to zero for all estimator classes. Burst dropout is different: the ΔMAE values remain moderate, but one central 3600 s frozen-data gap causes long post-gap re-synchronization times for all estimators. The recovery times are 27.35 h for DM, 27.41 h for HDM, 28.56 h for HECM, and 27.44 h for DD. Figures 6.9, 6.10, and 6.11 therefore combine the global ΔMAE summary with transition and recovery views. The underlying metrics are listed in Appendix Tables A.2 and A.3, and the scenario-to-evidence mapping is summarized in Appendix Table A.4.

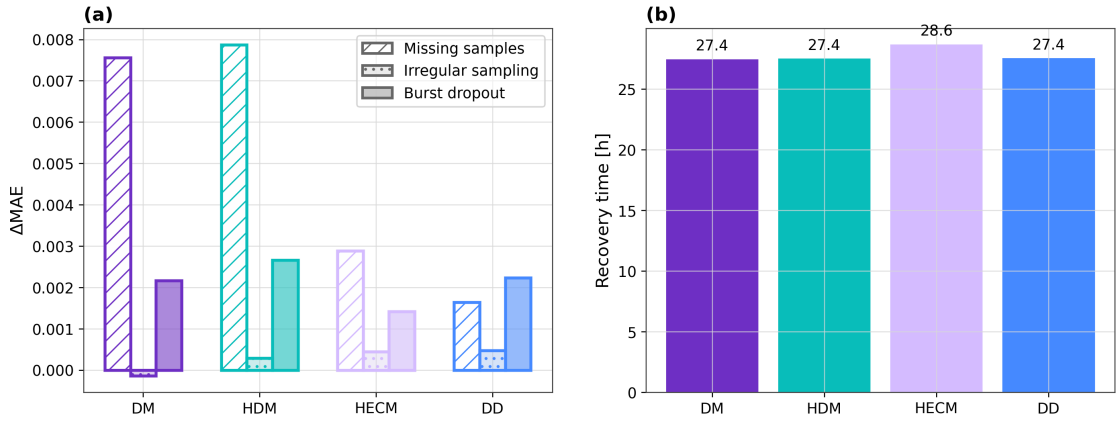

Figure 6.9: Continuity and timing disturbance summary. The figure compares the global ΔMAE penalties for missing samples, irregular sampling, and burst dropout, and it complements these full-run penalties with the burst-dropout recovery times. This compact summary shows why missing samples, timing irregularity, and longer frozen-data gaps cannot be judged from one metric alone.

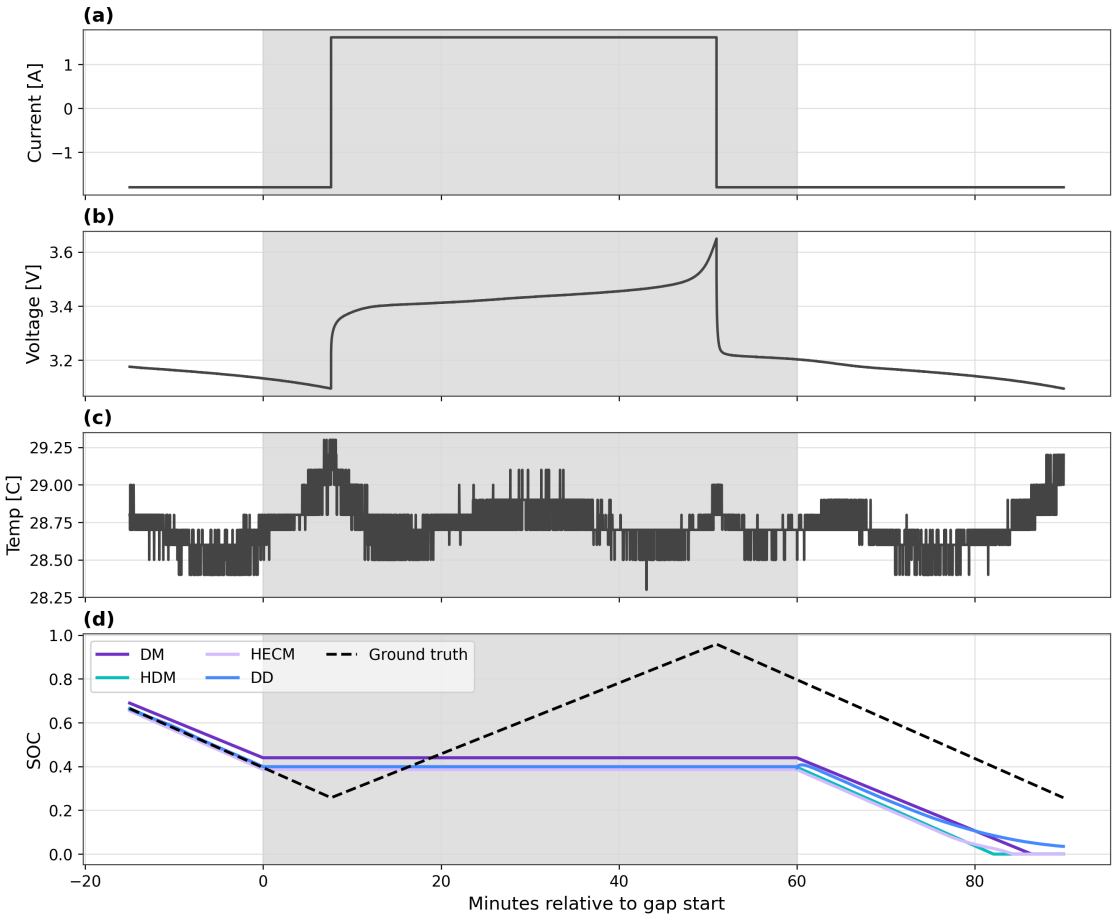

Figure 6.10: Local transition into the burst-dropout interval. The shaded region marks the frozen-data window and makes visible how the estimator outputs behave while the input stream is held constant. This view isolates the immediate local response that is hidden by full-run aggregate metrics.

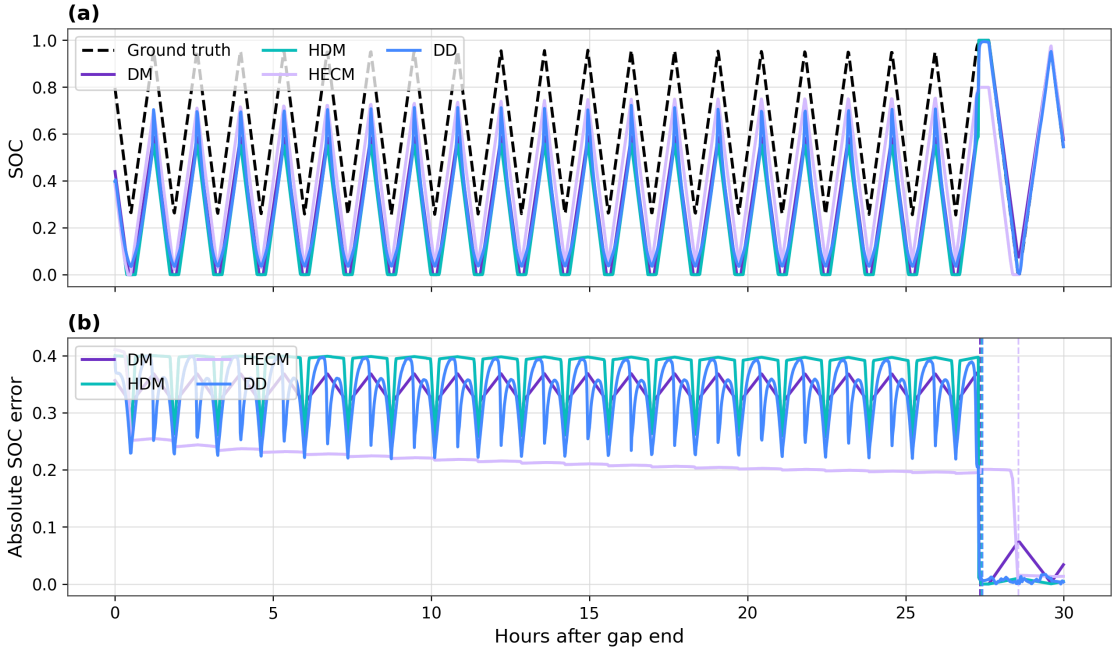

Figure 6.11: Post-gap recovery after burst dropout. The figure shows the disturbed SOC trajectories after the frozen interval together with the corresponding absolute SOC error, so that the delayed return toward the baseline bands becomes directly visible. It therefore complements the moderate full-run ΔMAE values with the practically more revealing local recovery behavior.

6.7.6 Spike sensitivity

Single voltage spikes lead to strongly different local reactions even when the full-run MAE remains nearly unchanged for several classes. The spike protocol injects one ±0,20 V single-sample spike every 1000 samples, corresponding to every 1000 s on the nominal 1 s time base. HECM is almost unaffected under this protocol, whereas DD shows the clearest spike sensitivity in both the representative local window and the aligned post-spike excess-error response. In the global metrics, the DD MAE increases from 0.0100 to 0.0144, while the changes for DM, HDM, and HECM remain smaller or close to their baseline level. The post-spike excess error is evaluated event-wise rather than as a global full-trajectory percentile: for each of the first 40 valid spike events, the SOC-error trajectory is aligned in a window from −60 s to 180 s around the injected spike, the mean absolute error over the final 10 s before the spike defines the local pre-spike baseline, and the largest positive increase above this baseline after the spike is stored as the event severity.

The compact severity value in Figure 6.12 is the 95th percentile of these event-wise peak increases. The figure therefore emphasizes local spike behavior instead of relying only on signed full-run averages. The corresponding global and local metrics are listed in Appendix Tables A.2 and A.3.

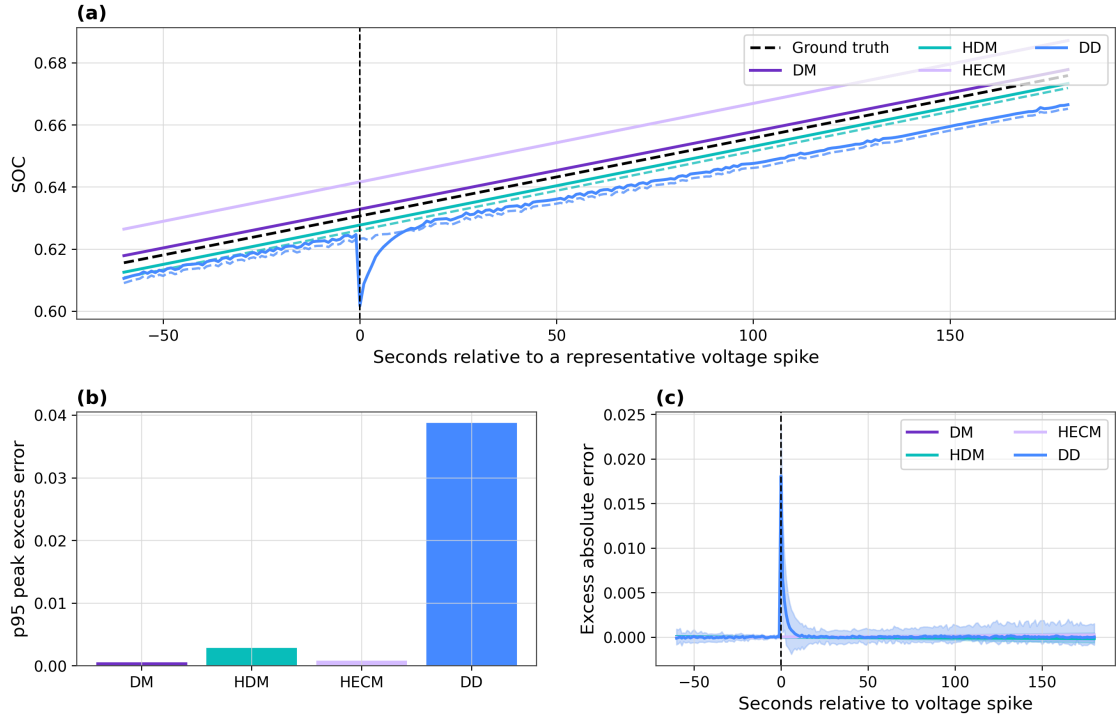

Figure 6.12: Voltage-spike sensitivity. The figure combines a representative local SOC response around an injected spike, a compact spike-severity summary based on the 95th percentile of event-wise post-spike peak excess errors, and the aligned post-spike excess-error behavior across repeated spike events. This local perspective is necessary because isolated spike effects can remain understated in signed full-run averages even when the transient response differs strongly across estimators.

6.7.7 Cross-scenario comparison

The cross-scenario comparison confirms that no single estimator dominates all criteria. HDM is the clean-condition leader, HECM is strongest in the scenarios that most clearly change the estimator ranking, especially persistent current bias and voltage noise, and DD is most attractive when missing-sample stability or recovery after initialization mismatch is prioritized. ADC quantization remains comparatively weak in this benchmark and is kept in the scenario space for complete-

ness rather than because it changes the main ranking. Figure 6.13 condenses the disturbed-scenario ΔMAE values into one heatmap, while Appendix Tables A.2, A.4, and A.5 provide the numerical details. Appendix Figure A.1 adds the dedicated ADC-quantization illustration from the benchmark protocol.

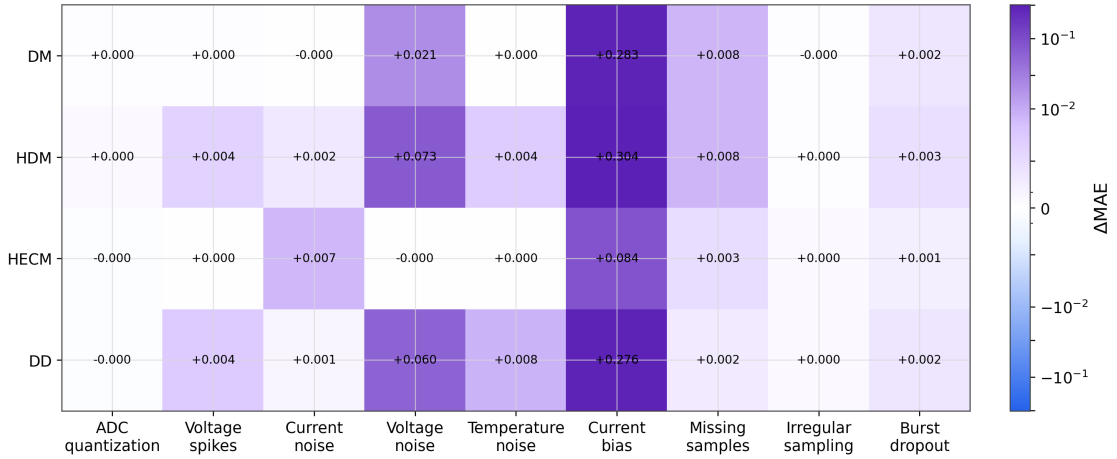

Figure 6.13: Cross-scenario error increase relative to the baseline case. The heatmap condenses the disturbed-scenario penalties, including the ADC-quantization case, and makes estimator-specific ranking shifts visible across disturbance classes. It therefore serves as the compact scenario-overview view for identifying where robustness advantages are stable and where they are disturbance-specific. Error values in the figure are SOC fractions on a 0 to 1 scale. Multiplication by 100 gives percentage points.

6.8 Discussion

6.8.1 Model-specific performance behavior

Because the shared LSTM-based SOH support has already been fixed in the methodology, the following comparison focuses on the SOC-estimator families under a common learned health-context signal rather than comparing independently designed SOH models.

The direct-measurement model behaves as the expected drift-prone reference. Persistent current disturbances accumulate directly into the SOC estimate, and missing information cannot be corrected through an additional feedback path. HDM improves nominal calibration by adapting the effective capacity through the shared LSTM-based SOH context, but it remains structurally exposed to the

same current-integration failure mode and additionally inherits the dynamics of this learned health signal.

HECM benefits from interpretable parameterization, explicit voltage-residual correction, and causal observer structure. Combined with the same shared online SOH support used by the other SOH-aware variants, this explains why it is the overall robustness leader under persistent current bias and voltage noise. The same result does not mean that all noise channels are irrelevant for HECM: current noise can still affect the observer locally through current-dependent parameter scheduling and update terms. DD behaves as a compact context-rich estimator with channel-dependent strengths and weaknesses. It is especially strong under missing samples and initialization mismatch, but more exposed to voltage spikes and to propagated noise in derived inputs such as $\mathrm{d}I/\mathrm{d}t$, Q_c, EFC, and the SOH context. In the present benchmark, this should be read as a design lesson of the compact deployed instance rather than as a hard class limit, because the data-driven family still retains a larger tuning space through architecture, retraining, feature selection, and state-handling choices than the more fixed-form direct or ECM-based alternatives.

6.8.2 Accuracy and Performance Trade-Offs

A central insight of the chapter is therefore that low clean-condition MAE does not imply strong disturbed-operation performance. The trade-off is not only between accuracy and computational complexity, but also between calibration strength, correction capability, recovery speed, disturbance propagation, and the degree to which a model relies on one dominant trigger variable. The benchmark also shows that estimator comparison should not collapse everything into one leaderboard: a deployment-oriented comparison must separate nominal accuracy, disturbance robustness, recovery after inconsistency, and implementation feasibility.

6.8.3 Implications for BMS deployment

If lowest nominal SOC error under clean sensing is the main design priority, HDM is the recommended class because it achieves the lowest baseline MAE of 0.0024 (0,24 %). The corresponding trade-off is strong sensitivity to current bias and the lack of recovery to the baseline bands in the inspected initialization-mismatch window. If the primary priority is best overall robustness under uncertain measurement quality, HECM is the strongest choice because it is most resilient under

current bias, remains nearly unchanged under voltage noise, and still preserves competitive baseline accuracy. The trade-off is the larger flash footprint and slower strict-band recovery than DD. If fastest resynchronization after restart or wrong initial state dominates the design, DD is the strongest option because it shows the fastest strict-band recovery in the local initialization analysis, with the trade-off of greater sensitivity to voltage spikes and weaker performance than HECM under current bias.

If resilience to missing samples and signal freezing dominates the design, DD again becomes attractive because it shows the smallest MAE increase under missing samples and strong post-disturbance stability, while clean-condition accuracy remains below HDM and voltage-driven disturbances require a careful feature-pipeline design. If iterative development headroom and future model refinement dominate the design strategy, DD is also attractive because architecture, features, and training can be updated directly while keeping the deployment target fixed. The present compact instance is still weaker than HECM in the strongest bias- and voltage-driven stress cases, but it retains the clearest path for systematic model iteration. If the main objective is maximum implementation simplicity and the largest resource margin, DM remains the simplest class by far, but it also shows the weakest robustness under biased current measurements and missing-sample stress.

The practical message of the chapter is therefore not that one estimator wins universally. Estimator selection must instead be tied to the disturbance profile, recovery requirements, implementation constraints, and expected model-maintenance strategy of the target BMS. The footprint analysis in Section 6.5.4 further narrows this decision space: all four deployment-prepared variants remain within the common embedded budget, persistent state memory is negligible for all classes, HECM is the tightest flash case, and DM keeps the largest implementation margin.

Table 6.4: Decision-oriented recommendation map derived from the robustness benchmark. The table condenses the scenario-wise findings into dominant deployment priorities, the estimator class most strongly supported by the present evidence, and the corresponding trade-off that must still be considered in design decisions.

Primary deployment priority	Recommended class	Main supporting benchmark evidence	Main trade-off to consider
Lowest nominal SOC error under clean sensing	HDM	Lowest baseline MAE on the benchmark trajectory (0.0024, 0,24 %)	Strong sensitivity to current bias and no return to the baseline bands in the inspected initialization-mismatch window
Best overall robustness under uncertain measurement quality	HECM	Strongest current-bias robustness, nearly unchanged under voltage noise, and competitive baseline accuracy	Larger flash footprint and slower strict-band recovery after initialization mismatch than DD
Fastest re-synchronization after restart or wrong initial state	DD	Fastest strict-band recovery in the local initialization analysis	More sensitive than HECM to voltage spikes and less robust than HECM under current bias
Highest resilience to missing samples and signal freezing	DD	Smallest MAE increase under missing samples and strong post-disturbance stability	Clean-condition accuracy remains below HDM, and voltage-driven disturbances require careful feature-pipeline design
Largest development headroom for iterative model improvement	DD	Architecture, features, and training can be updated directly while retaining embedded-feasible deployment and strong performance in recovery and missing-sample scenarios	Present compact instance is still weaker than HECM in the strongest bias- and voltage-driven stress cases
Simplest implementation and largest	DM	Minimal architecture and smallest flash footprint by a wide	Weakest robustness under biased current measurements and

Table 6.4 condenses the benchmark into a deployment-oriented recommendation map so that the reader is not left with a large set of disconnected scenario results. Figure 6.14 complements this table with a compact visual synthesis of the same evidence. The plotted values are relative scores rather than additional physical measurements. For each lower-is-better metric, the four estimator classes are normalized by their minimum and maximum as $(x_{\max} - x)/(x_{\max} - x_{\min})$, so that the best class within the present comparison receives 1 and the weakest class receives 0. The accuracy dimension averages the normalized baseline MAE, RMSE, and P95 error, the robustness dimension averages the normalized ΔMAE values across the disturbed scenarios, and the recovery dimension uses the strict recovery time after the initial-SOC perturbation, with non-recovery in the inspected window treated as the weakest score.

The three profiles in Figure 6.14 then apply illustrative priority weights of 0.60/0.20/0.20, 0.20/0.60/0.20, and 0.20/0.20/0.60 to accuracy, robustness, and recovery, respectively. These profiles should therefore be read as accuracy-prioritized, robustness-prioritized, and recovery-prioritized compromises rather than as replacements for the raw scenario-wise metrics or for the recommendation map in Table 6.4.

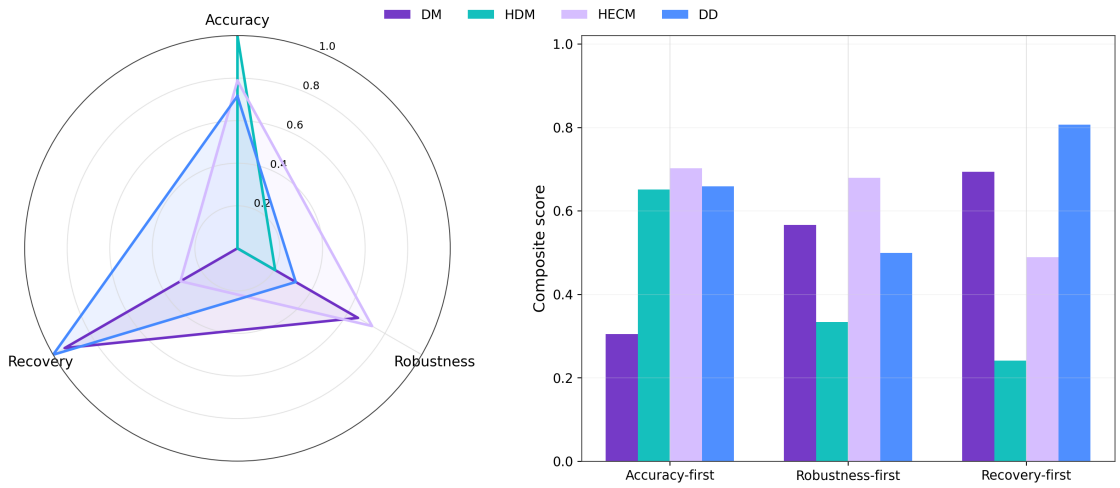

Figure 6.14: Decision-oriented synthesis of the robustness benchmark. Panel (a) collapses the benchmark into three relative decision dimensions, namely nominal accuracy, disturbed-scenario robustness, and recovery behavior, using lower-is-better metrics normalized across the four estimator classes. Panel (b) applies three illustrative deployment-priority profiles and is intended as a visual complement to Table 6.4, not as a replacement for the raw scenario-wise evidence.

6.8.4 Limitations and future extensions

The current benchmark phase is based on one representative dataset trajectory and should later be extended to additional cells, broader ageing states, and cross-cell validation before stronger generalization claims are made. The spike study currently uses isolated spike events. Clustered disturbances, longer spike sequences, and hardware-level disturbance injection may reveal additional model differences. The initialization test for DD remains an online-state proxy based on Q_c perturbation rather than an explicit latent-state reset. This is methodologically justified, but it should still be interpreted as a class-specific approximation. The next benchmark stages should add the remaining extended scenarios, more cells, pack-level experiments, and later hardware-in-the-loop or real embedded runtime measurements while keeping the same performance-oriented evaluation logic.

6.9 Conclusion

Taken together, Chapter 6 shows that SOC-estimator quality for battery-management deployment cannot be reduced to clean-condition accuracy alone. The chapter compares representative estimator families, namely direct measurement, SOH-supported direct measurement, equivalent-circuit-based hybrid estimation, and data-driven neural estimation, under the same disturbed measurement protocol. The purpose is therefore not to declare one model universally best, but to identify which strengths and weaknesses become visible when different modelling paradigms are exposed to bias, noise, missing information, timing disturbance, spikes, and initialization mismatch.

The clean-condition ranking alone would point to HDM, because it achieves the lowest baseline error with MAE = 0.0024 (0,24 %). The disturbed scenarios change that interpretation. Under the emphasized disturbance profile, HECM is the strongest overall robustness compromise because its voltage-feedback structure limits several measurement-driven error mechanisms. This becomes clearest under persistent current bias and voltage noise, where HECM remains substantially more stable than the integration-dominated variants. DD does not dominate the full benchmark, but it is the most attractive class when recovery after initialization mismatch or resilience to missing samples is prioritized. DM provides the simplest reference and the largest resource margin, but it exposes the expected vulnerability to drift and missing information.

If one broadly useful estimator class had to be preferred for the particular disturbance profile examined here, the evidence supports the hybrid equivalent-circuit approach as the strongest compromise. Its combination of interpretable physical parameterization, voltage-based correction, and shared data-driven SOH support gives ECM-based estimation a clear role in embedded BMS design. At the same time, the data-driven class remains important because it has the largest development headroom: architecture, features, training data, and state handling can be adapted directly while preserving the same benchmark framework. The present compact DD instance should therefore not be read as the final limit of neural SOC estimation, but as evidence that neural estimators must be trained and feature-engineered to use current, voltage, temperature, timing, and health context in a balanced way instead of relying too strongly on one dominant trigger variable.

The broader implication is that practical BMS evaluation needs more than MAE or another average accuracy score. Average error remains necessary, but it must be complemented by local disturbance response, recovery behavior, tail errors, signal-continuity sensitivity, and the way disturbed measurements propagate through derived online features. The benchmark is deliberately extensible: the next stages should add more cells, broader ageing states, clustered or longer disturbance events, and eventually hardware-in-the-loop or real embedded runtime validation. Chapter 7 builds on this robustness perspective and turns to recurrent neural SOC and SOH estimation on microcontrollers, where causal state handling, compression, quantization, and runtime feasibility become part of the estimator design itself.

Chapter 7

Embedded Artificial Intelligence for SOC and SOH Estimation on Microcontrollers

7.1 Embedded Deployment Objective

Accurate online estimation of state of charge and state of health is central to battery-management operation, yet many recent neural estimators remain evaluated mainly as offline research models rather than as continuously running embedded components.[90, 91, 53, 17] Across the broader state-estimation literature, measurement-derived, model-based, and data-driven approaches continue to coexist.[91, 53, 17, 20] Carefully tuned equivalent-circuit and electrochemical observers can reach very low laboratory errors,[82, 19, 83, 84] while recent sequence models report strong SOC accuracy on specialized datasets,[92, 29] and TinyML-style microcontroller deployments typically operate in the low-single-digit percent range.[33, 32, 34]

This chapter therefore examines how recurrent SOC and SOH estimators can be transferred from floating-point training releases to practical microcontroller execution under explicit flash, SRAM, latency, and energy limits. The focus is not only on predictive accuracy, but on the combined interaction between estimation utility, compression, firmware integration, and runtime cost on the target controller. The application context is modular stationary storage in remote microgrids within the MG-FARM programme, where storage modules must operate autonomously and the battery management system must compute SOC and SOH on an STM32H753ZI-class microcontroller without depending on cloud-side infrastructure.[93, 94, 95, 96]

Against this background, the chapter addresses an embedded-AI gap in the lit-

erature. A small number of TinyML-style battery studies already report microcontroller deployments,[33, 34, 86] but detailed treatment of scaling statistics, hidden-state handling, quantization constants, and long-horizon streaming behaviour on the actual controller remains uncommon. Recurrent neural estimators are particularly relevant in this context because they can absorb long temporal dependencies and evolving degradation behaviour without requiring an explicit electrochemical model at runtime.[28, 84] Within the dissertation, this chapter therefore extends the earlier SOH-estimation study from Chapter 5 toward embedded SOC and SOH deployment on one concrete MCU platform. The central contribution is therefore the complete deployment path from trained recurrent models to executable MCU firmware: stateful LSTM + MLP estimators, structured pruning, INT8 weight quantization, manual export into transparent C kernels, and on-device benchmark measurements on the STM32 target. In this sense, battery AI is treated explicitly as an embedded-systems problem: an estimator is only practically useful if it remains accurate under long-horizon streaming replay and still fits into the resource and timing budget of the controller.

7.2 Dataset Basis and Experimental Scope

All experiments rely on the LFP DoE Cycle Ageing Dataset described in Section 4.1.[49, 97] The three-factor DoE design, the representative trajectory structure, and the full SOH spread across cells are documented there (Figures 4.2, 4.3, and 4.4).[52] Between ageing loops, dedicated check-ups with capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterisation are inserted, and in addition stationary-storage load profiles derived from residential PV home storage are injected so that the data contain both tightly controlled diagnostics and application-oriented operating conditions.[75] The raw measurements are available at 1 Hz and contain voltage, current, terminal temperature, and auxiliary channels. This combination is important because it exposes the estimators simultaneously to fast SOC transients, long-duration integration horizons, and the much slower SOH evolution that unfolds across the full ageing campaign.

7.3 Feature Engineering and Reference Labels

The SOC estimator consumes the six-dimensional feature vector

$$\mathbf{x}_t^{\mathrm{SOC}} = \Big[U(t),\, I(t),\, T(t),\, Q_c(t),\, \tfrac{\mathrm{d}U}{\mathrm{d}t}(t),\, \tfrac{\mathrm{d}I}{\mathrm{d}t}(t)\Big],$$

where the temporal derivatives are computed by centred finite differences. The SOH estimator uses a slower-varying feature stack,

$$\mathbf{x}_t^{\mathrm{SOH}} = \Big[t,\, U(t),\, I(t),\, T(t),\, \mathrm{EFC}(t),\, Q_c(t)\Big],$$

with cumulative time stamp t, equivalent full cycles $\mathrm{EFC}(t)$, and cumulative charge state $Q_c(t)$ capturing the longer degradation context. All channels are robustly normalised so that the exact same scaling statistics can be embedded into the firmware. Instead of mean and variance, the chapter follows the paper in using median m_k and interquartile range r_k,

$$\tilde{x}_{t,k} = \frac{x_{t,k} - m_k}{r_k + \varepsilon}, \qquad \varepsilon = 10^{-6},$$

which reduces sensitivity to outliers and keeps host-side replay and MCU execution numerically aligned.[98, 99]

The SOC and SOH reference labels are constructed from the Coulomb-counting and capacity-check-up procedures defined in Section 4.1 (Equations 4.2 and 4.3). Both label traces rely on information unavailable during deployment, namely sustained CV-based full-charge resets and periodic full-capacity discharges, and are used exclusively for training and evaluation. On the STM32, the estimators operate only on streamed measurements and their internal recurrent states. The embedded deployment then follows a strictly stateful streaming regime: hidden and cell states are forwarded sample by sample along the full sequence without sliding input windows, so that host-side replay and STM32 replay use identical inference conditions. Figure 7.1 summarizes this shared stateful LSTM + MLP structure that is later specialized to the SOC and SOH tasks.

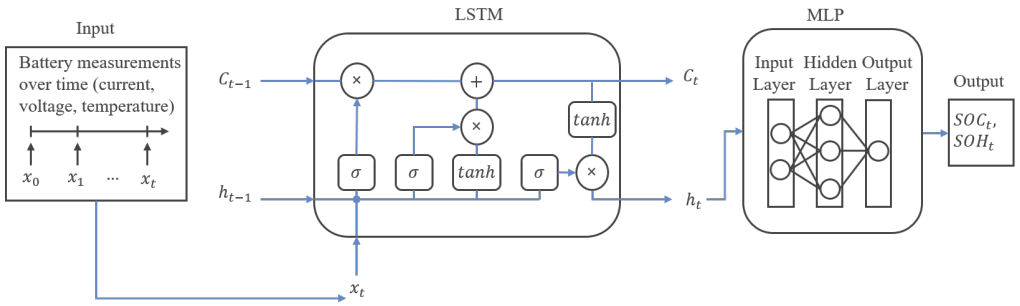

Figure 7.1: Stateful LSTM + MLP estimator structure used for both SOC and SOH estimation. Preprocessed feature streams are processed step by step by a single-layer LSTM whose hidden and cell states are propagated across time. A lightweight MLP head then maps the hidden representation to either SOC or SOH. The SOC instance uses hidden size 64 with an MLP width of 64 and a sigmoid output, the SOH instance hidden size 128 with an MLP width of 128 and a linear output. This structure is the basis for the later pruning, quantization, and MCU export steps.

7.4 LSTM Architecture and Training Setup

Both SOC and SOH estimators employ the same recurrent equation set, but with task-specific feature sets and sizes, as summarised in Figure 7.2. The SOC model uses a single-layer LSTM with hidden size $H = 64$ that consumes a six-dimensional feature vector $\{U, I, T, Q_c, \mathrm{d}U/\mathrm{d}t, \mathrm{d}I/\mathrm{d}t\}$, followed by an MLP head with one ReLU-activated hidden layer of width 64 and a sigmoid output neuron that constrains the estimate to $[0, 1]$. The SOH model uses the same structure with hidden size $H = 128$ and an MLP hidden width of 128, but consumes a different six-dimensional feature vector $\{t, U, I, T, \mathrm{EFC}, Q_c\}$ and ends in a purely linear output neuron because the slowly varying SOH target does not benefit from a saturating output activation. In contrast to the hourly-aggregated SOH estimator of Chapter 6, both models here operate directly on the sample-wise 1 Hz stream. The implementation follows the standard LSTM gate formulation in PyTorch, with propagated hidden and cell

states as described in the recurrent-neural-network literature.[26, 100]

$$
\begin{aligned}
\mathbf{z}_t &= [\mathbf{x}_t, \mathbf{h}_{t-1}], \\
\mathbf{i}_t &= \sigma(W_i \mathbf{z}_t + \mathbf{b}_i), \qquad \mathbf{f}_t = \sigma(W_f \mathbf{z}_t + \mathbf{b}_f), \\
\mathbf{o}_t &= \sigma(W_o \mathbf{z}_t + \mathbf{b}_o), \qquad \mathbf{g}_t = \tanh(W_g \mathbf{z}_t + \mathbf{b}_g), \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
\end{aligned}
\tag{7.1}
$$

Here $\mathbf{x}_t$ denotes the normalized input vector at sample t, $\mathbf{h}_{t-1}$ and $\mathbf{c}_{t-1}$ are the previous hidden and cell states, and $\sigma(\cdot)$ denotes the logistic activation. Writing the equations explicitly is important for the embedded implementation because the same gate order, state update, and activation functions are reproduced manually in C.

This restriction to a single-layer LSTM + MLP architecture is deliberate. The focus is not an extensive architecture search over GRUs, Transformers, or deeper recurrent stacks, but a transparent and reproducible embedded tool flow for stateful estimators whose hidden-state handling can later be re-implemented faithfully in C. A single-layer LSTM reduces to one compact gate computation per step and keeps pruning, quantization, and firmware export directly traceable. This differs from the shared SOH context model used in Chapter 6, which is an hourly sequence model with feature projection, a two-layer LSTM, residual refinement, and a deployment-prepared hidden size of 112 for the robustness benchmark. Chapter 6 uses that model to provide a common health context for comparing SOC-estimator families. The present chapter instead asks how stand-alone recurrent SOC and SOH estimators can be compressed, exported, and measured on the microcontroller. The two architectures therefore serve different experimental purposes, and their results should not be interpreted as a direct architecture ranking.

Both SOC and SOH models are trained with mean-squared-error loss against the corresponding reference label at each training sample, while validation additionally tracks MAE, RMSE, and R^2. The resulting checkpoints, configuration files, and optimizer states are archived so that all later pruning, quantization, and firmware-export steps can be replayed deterministically. Training is carried out on NVIDIA A100-class GPU hardware in mixed precision (FP16), and both tasks use the AdamW optimiser, cosine-annealing warm restarts, gradient clipping, and mini-batch training.[101, 102, 103, 104] For SOC, the learning rate is 1.5×10^{-4}, the batch size is 512, and the sequence chunk length is 2024. For SOH, the learning

rate is 2.0×10^{-4}, the batch size is 128, and the chunk length is 2048. Although deployment is fully stateful and step-wise, training uses truncated backpropagation through time over these chunks: within each chunk, the hidden and cell states are propagated exactly as in deployment, so the stateful update mechanism is already learned during optimisation while the chunking still enables randomized sampling, faster convergence, and better generalization across the diverse ageing trajectories.

The base models are first evaluated in streaming mode on the full ageing trajectory of a representative LFP cell and serve as the numerical reference for all subsequent experiments. These same checkpoints are then pruned, quantized, and exported to STM32 firmware, where identical feature streams are replayed so that accuracy, flash, RAM, latency, and energy always refer to the same stateful sequence. Figure 7.3 summarizes this end-to-end path from training to embedded benchmarking.

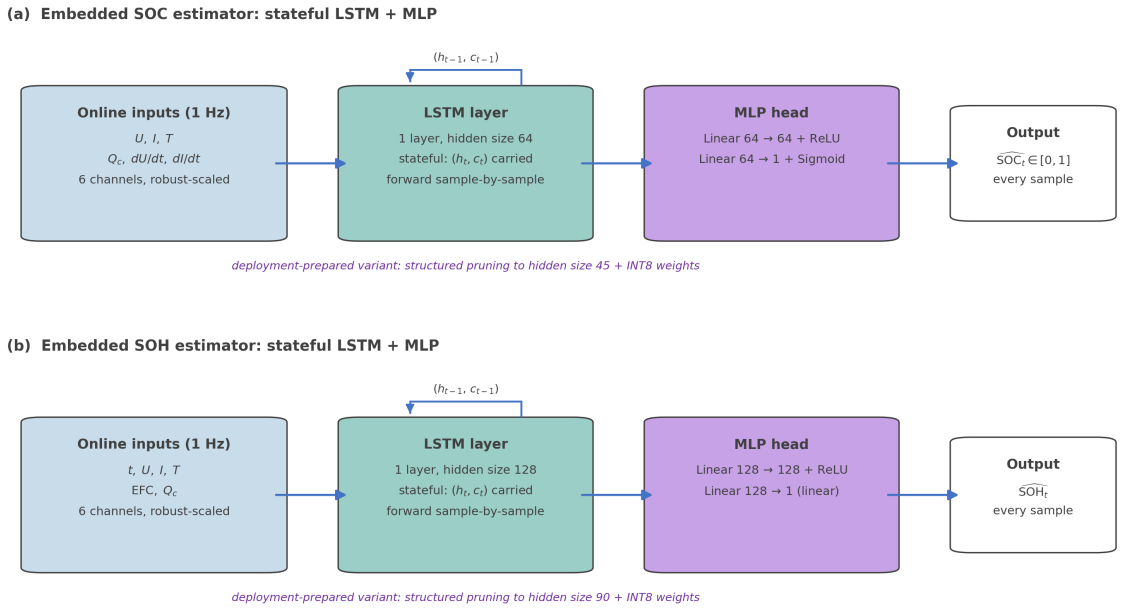

Figure 7.2: Block-level architectures of the two stand-alone embedded estimators. (a) The SOC estimator processes a six-channel feature vector $\{U, I, T, Q_c, \mathrm{d}U/\mathrm{d}t, \mathrm{d}I/\mathrm{d}t\}$ with a single stateful LSTM layer of hidden size 64, followed by an MLP head with one ReLU-activated hidden layer of width 64 and a sigmoid output. (b) The SOH estimator uses the same structure with hidden size 128 and an MLP width of 128, consumes the alternative feature vector $\{t, U, I, T, \mathrm{EFC}, Q_c\}$, and ends in a linear output neuron. Both models forward their hidden and cell states sample by sample along the 1 Hz stream. The purple annotations give the reduced hidden sizes of the deployment-prepared variants after structured pruning and INT8 weight quantization.

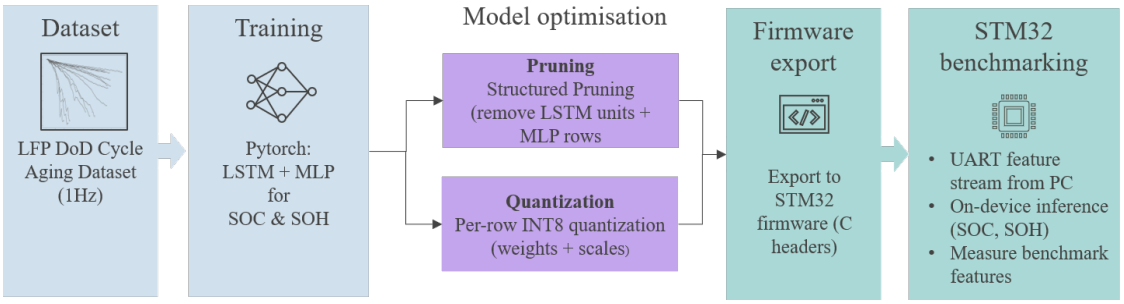

Figure 7.3: End-to-end deployment pipeline from the LFP ageing dataset and PyTorch training to pruning, quantization, C-header export, and final STM32 benchmarking. The figure makes explicit that the same trained SOC and SOH checkpoints are reused across the floating-point, pruned, and quantized deployments so that later trade-off comparisons remain numerically consistent.

7.5 Structured Pruning for MCU Deployment

Pruning is applied as structured magnitude pruning at the level of complete LSTM hidden units, because whole-channel removal is far better suited to efficient C implementations on a microcontroller than unstructured sparse masks.[10, 41] The pruning score aggregates the L2 norms of the incoming weights for each hidden unit across all four LSTM gates,

$$s_h = \sum_{g \in \{i,f,o,g\}} \left(\|W_{\text{ih}}^{(g)}[h,:]\|_2 + \|W_{\text{hh}}^{(g)}[h,:]\|_2 \right),$$

so that hidden units with the smallest aggregated saliency are removed first. This follows the same structured-sparsity intuition used in recurrent-network pruning literature.[43, 46, 45]

In the final deployment, the SOC estimator is reduced from 64 to 45 hidden units and the SOH estimator from 128 to 90 hidden units, which in both cases corresponds to roughly 30% channel-wise sparsity. Rows and columns are removed jointly from the recurrent gate matrices and the downstream MLP weights so that the pruned network remains a valid stateful recurrent mapping, and after each pruning step the model is fine-tuned again. The chapter therefore treats pruning not as a single static compression event, but as a repeated prune-and-recover cycle that progressively reduces recurrent state size while preserving the learned temporal mapping. The engineering purpose of pruning is direct: reduce recurrent state size, shrink flash and RAM demand, and shorten on-device inference time without materially compromising streaming accuracy. Figure 7.4 illustrates this channel-wise removal process at the level of hidden units and associated MLP rows.

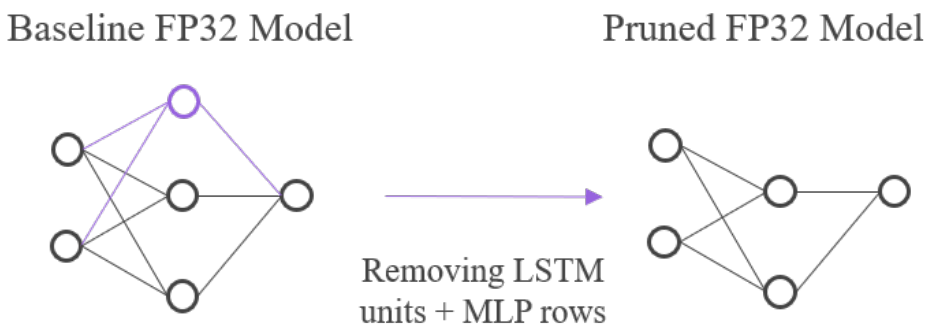

Figure 7.4: Structured pruning concept used for the embedded SOC/SOH estimators. The left-hand side highlights a low-saliency hidden channel together with its incident gate connections, while the right-hand side shows the reduced recurrent architecture after removing the corresponding hidden unit and associated downstream weights.

7.6 INT8 Quantization and Manual STM32 Export

Quantization complements pruning by reducing the numerical precision of the remaining weights while leaving the topology unchanged. In this chapter, the recurrent weights are stored as signed INT8 values with associated floating-point scales so that the firmware can retain a transparent mapping back to the PyTorch reference.[11, 42] This aligns with the broader observation from recent pruning/quantization benchmarks that aggressive compression can preserve useful accuracy when the deployment path is engineered carefully.[47] A signed INT8 representation uses the codebook $\{-128, \ldots, 127\}$ and therefore provides a symmetric dynamic range around zero, which is better matched to centered neural-network weight distributions than an unsigned mapping. Conceptually, each FP32 weight w is mapped to an integer code q and reconstructed as

$$q = \mathcal{Q}_s(w), \tag{7.2}$$

$$\hat{w} = s\, q, \tag{7.3}$$

where $s > 0$ is the scale factor and $\mathcal{Q}_s(\cdot)$ rounds w/s to the nearest integer while saturating the result to the closed interval $[-128, 127]$.

The implementation uses post-training per-row symmetric INT8 quantization for the input-to-gate and hidden-to-gate matrices of the LSTM. For a row vector

$w^{(r)} = (w_h)_h$, the scale is chosen as

$$s = \frac{\max_h |w_h|}{127},$$

so that the largest absolute row entry maps close to ± 127 while zero remains exactly representable. Biases and activations remain in FP32. This design is pragmatic because it preserves numerical stability, keeps the firmware implementation close to the floating-point reference, and still reduces recurrent weight storage by roughly a factor of four relative to FP32 storage. During inference, the embedded gate computation reconstructs the effective weights from INT8 codes and FP32 scales according to

$$g = (xW_{\text{ih}}^{\text{int8}}) \cdot s_{\text{ih}} + (hW_{\text{hh}}^{\text{int8}}) \cdot s_{\text{hh}} + b_{\text{fp32}},$$

where x is the current input, h the previous hidden state, $W_{\text{ih}}^{\text{int8}}$ and $W_{\text{hh}}^{\text{int8}}$ are the quantized input-to-gate and hidden-to-gate matrices, and s_{ih} and s_{hh} are the corresponding per-row scale vectors. A built-in streaming comparison validates this conversion and typically yields MAE differences below 10^{-4} between exported and reference implementations, which indicates that the recurrent dynamics are preserved numerically. The exported model parameters are written as C headers and integrated into manually implemented stateful LSTM + MLP kernels. This manual route is chosen because common export toolchains such as STM32Cube.AI do not natively support the continuous hidden-state handling required for these estimators.[105, 106, 107] The result is a fully transparent firmware path: every scaling constant, every recurrent state update, and every arithmetic step can be audited and reproduced outside the proprietary toolchain. Figure 7.5 summarizes the FP32-to-INT8 conversion, scale handling, and STM32 export flow.

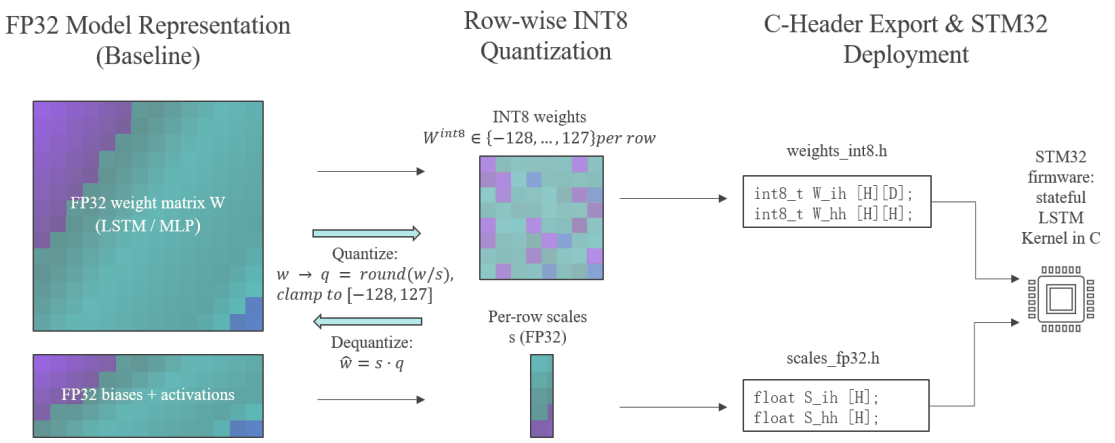

Figure 7.5: Per-row INT8 quantization and export concept used for the STM32 deployment. The figure shows how FP32 recurrent weight rows are mapped to the signed INT8 codebook together with floating-point scale factors and then exported as C headers for the stateful STM32 kernel, which reconstructs the effective gate weights during inference.

7.7 Benchmark Methodology and KPI Logic

The benchmark is executed on an STM32 Nucleo-144 board with an STM32H753ZI microcontroller, providing 2 MB flash and 1 MB SRAM, which together define the available code, weight, and runtime-buffer budget for the deployed estimators.[108, 9] Starting from the trained LSTM + MLP checkpoints, three embedded variants are evaluated for each task: the dense FP32 baseline, the structured-pruned FP32 model, and the quantized INT8-weight deployment. For all of them, identical feature streams from the LFP dataset are replayed both on the host and on the STM32 so that accuracy, latency, and resource measurements always refer to the same stateful input sequence.

The host-side replay operates directly on the reference PyTorch implementation and produces MAE, RMSE, R^2, error histograms, parity-style comparisons, and long-horizon trajectory views. These simulations serve as the high-precision numerical reference against which the embedded deployments are later judged. The embedded benchmark injects the same feature vectors via UART into dedicated SOC and SOH firmware images, measures end-to-end host latency, measures pure on-device inference time through the DWT cycle counter, and derives energy per inference from the average inference time and an empirically calibrated board power of 0,5 W. RAM is computed from three parts: initialized static and global vari-

ables copied from flash to SRAM at startup, zero-initialized or uninitialized static and global variables reserved in SRAM, and the maximum observed stack usage during streaming. In linker-map terminology, the first two memory classes correspond to the `.data` and `.bss` sections. Flash is taken from the linker output of the STM32CubeIDE projects. Tables 7.1 and 7.2 therefore summarize specifically the STM32 resource and timing indicators, whereas the trajectory and distribution metrics remain in the host-side streaming analysis. The chapter distinguishes carefully between communication-dominated system latency and pure on-device inference time, because UART framing and host scheduling can dominate the end-to-end delay even though the actual neural computation remains well within the real-time budget of the target platform.

Table 7.1: STM32 engineering KPIs for the SOC deployments on the STM32H753ZI under full-stream replay. The table separates end-to-end host latency from pure on-device inference time and reports the measured flash footprint, total RAM usage, and energy per inference for the dense, pruned, and quantized variants.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	Energy [mJ]
Base FP32	12.70	1.40	105.32	4.93	0.70
Pruned FP32	12.20	0.80	62.27	4.03	0.40
Quant INT8	18.48	6.99	52.48	3.96	3.49

Table 7.2: STM32 engineering KPIs for the SOH deployments on the STM32H753ZI under full-stream replay. As in the SOC case, the table distinguishes end-to-end latency from core inference time and reports measured flash, RAM, and energy for the three deployment variants.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	Energy [mJ]
Base FP32	34.88	22.73	335.00	8.69	11.37
Pruned FP32	24.69	12.72	182.41	6.96	6.36
Quant INT8	41.23	29.21	138.00	6.70	14.60

7.8 Streaming Prediction Accuracy

The evaluation starts with full-sequence streaming replay over the complete ageing trajectory. Figures 7.6 and 7.7 therefore provide the central long-horizon view of

how the dense, pruned, and quantized deployments behave under identical stateful replay conditions. The PC reference replay and the exported STM32 firmware process the same ordered feature stream with the same stateful update logic, so the reported values are full-stream averages over the same trajectory rather than short-window averages from selected excerpts. Differences between variants therefore reflect pruning, quantization, and numerical representation rather than a change of evaluation data.

For SOC, the tracking remains tight across the entire dynamic range: the residuals stay centered around zero, and the quantized model overlaps the full-precision baseline so closely that the two traces are often visually almost indistinguishable. The full-stream SOC MAE values are 2,68 % for the dense FP32 model, 2,34 % for the pruned FP32 model, and 2,79 % for the INT8-weight deployment. The corresponding RMSE values are 3,81 %, 3,13 %, and 3,88 %. This confirms that per-row symmetric quantization preserves the recurrent dynamics sufficiently well for accurate long-horizon SOC tracking, while structured pruning slightly improves the aggregate error in this particular deployment. The pruned SOC model still shows slightly stronger smoothing in highly dynamic regions, which is the expected consequence of the reduced hidden-state dimensionality after structured channel removal.

For SOH, the task is structurally harder because the estimator has to infer slow degradation from sparse check-up events and continuous monitoring data. The discrete capacity tests are removed from the estimator input and only the remaining check-up blocks and operating segments are retained, so that the estimator must infer the degradation trend from causally available evidence. A causal smoothing stage is applied to the raw SOH predictions. The chapter follows the paper in using an exponential moving average with smoothing factor $\alpha = 10^{-6}$ together with a downward-rate limiter of 2×10^{-8} per step in order to suppress unphysical jumps while maintaining a monotonic long-term degradation trend. After this filtering, the full-stream SOH MAE values are 0,85 % for the dense FP32 model, 1,46 % for the pruned FP32 model, and 1,41 % for the INT8-weight deployment. The corresponding RMSE values are 1,15 %, 1,87 %, and 1,89 %. The compressed variants therefore preserve the long-term trend, but they introduce a visible and measurable accuracy penalty compared with the dense SOH baseline.

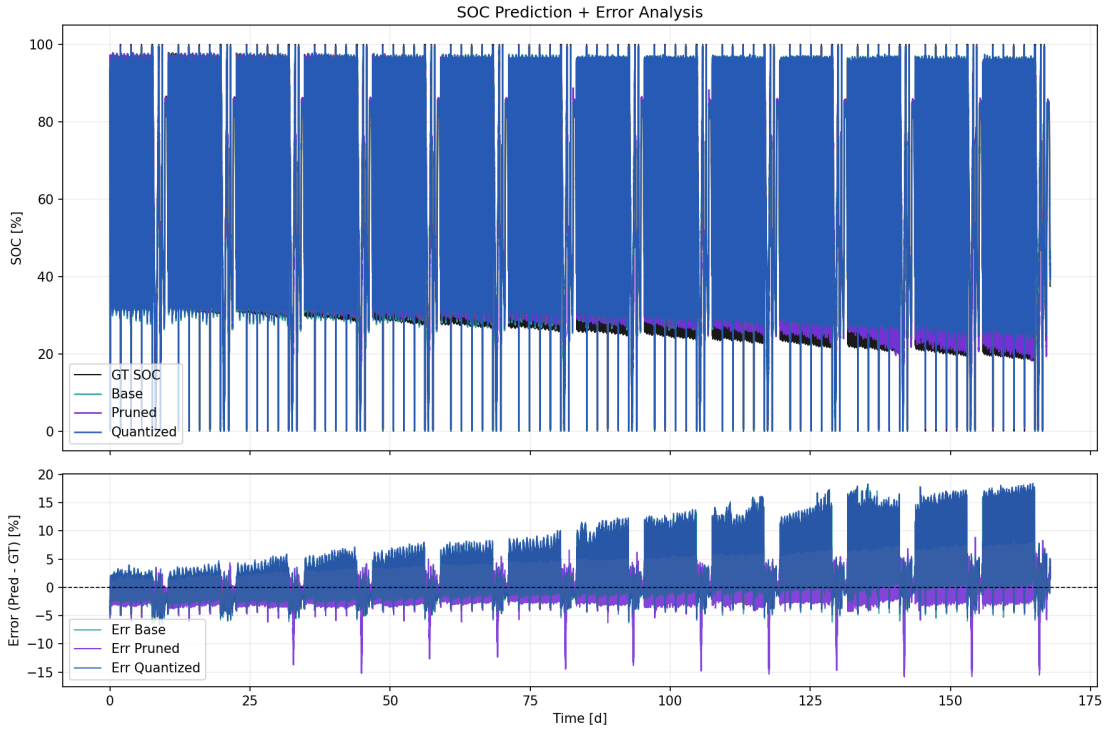

Figure 7.6: Streaming SOC dashboard for the base, pruned, and quantized SOC deployments over the full ageing trajectory. The upper panel compares the three stateful predictions with the SOC reference label, while the lower panel shows the instantaneous prediction error. The near-overlap of the dense and quantized traces is one of the central observations of the chapter.

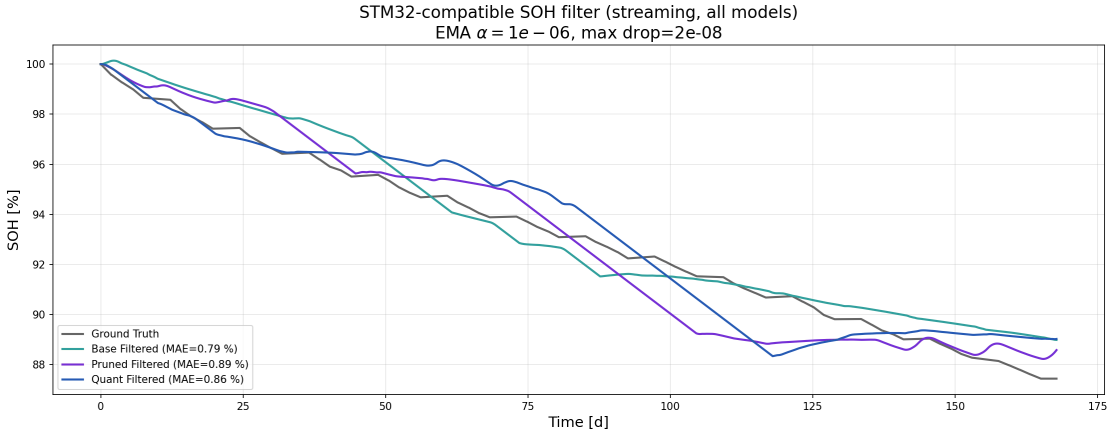

Figure 7.7: Streaming SOH dashboard with causal filtering for the three deployment variants. The upper panel shows the reference SOH together with the base, pruned, and quantized predictions after online smoothing, while the lower panel displays the corresponding prediction errors. The figure highlights the much slower degradation dynamics and the stronger role of causal post-processing in the SOH task.

7.9 Error Distribution and Local Transient Behaviour

Figures 7.8 and 7.9 analyse the statistical distribution of the prediction errors. For SOC, the distribution is narrow and symmetric, which indicates the absence of systematic bias across the full streaming replay. The SOC boxplots further show that the pruned model achieves the smallest median absolute error with 1,81 %, compared with 1,89 % for the dense FP32 model and 2,03 % for the INT8-weight deployment. In other words, pruning does not merely reduce resource demand for the SOC task, but acts in this case like a mild regularizer that slightly improves the dense baseline. For SOH, the error distribution is broader, which is consistent with the greater intrinsic difficulty of estimating capacity fade from operational data. Even so, the bulk of the SOH errors remains within a practically usable tolerance band of approximately ±2%, which keeps the embedded estimator relevant for onboard diagnostics despite the more difficult regression target.

Global metrics alone would still hide local failures. The chapter therefore also zooms into a high-dynamic pulse segment between 30k and 40k seconds (Figure 7.10) and into a representative check-up sequence between 657k and 897k seconds (Figure 7.11). In the pulse region, the quantized model follows the reference transients with high fidelity and captures voltage-recovery phases accurately,

whereas the pruned model smooths these sharper local features slightly because the reduced hidden-state dimension leaves less capacity for modelling fast electrochemical relaxation. In the check-up phase, all models converge well and remain stable over long integration horizons. This is an important result for the embedded formulation because it shows that the strictly stateful streaming execution does not accumulate visible drift even across long full-charge and full-discharge sequences.

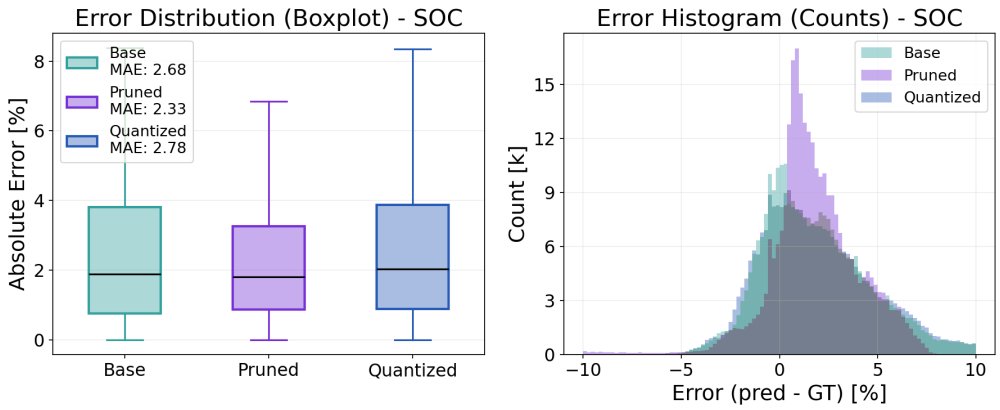

Figure 7.8: SOC error distribution for the embedded SOC deployments, including MAE boxplots and an error histogram. The narrow spread around zero confirms unbiased estimation, while the boxplots make visible that the pruned SOC model attains the smallest median error and interquartile range.

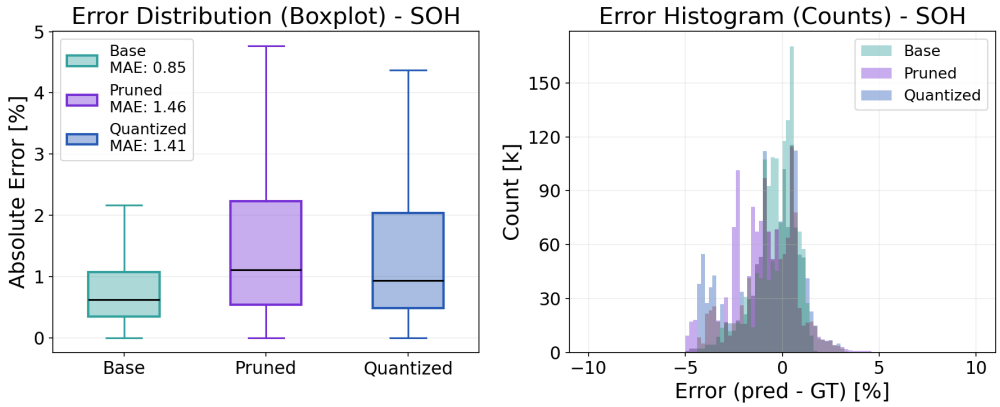

Figure 7.9: SOH error distribution for the embedded SOH deployments, including MAE boxplots and an error histogram. The spread is naturally broader than for SOC, but the majority of errors remains within a practically useful tolerance band around the reference trajectory.

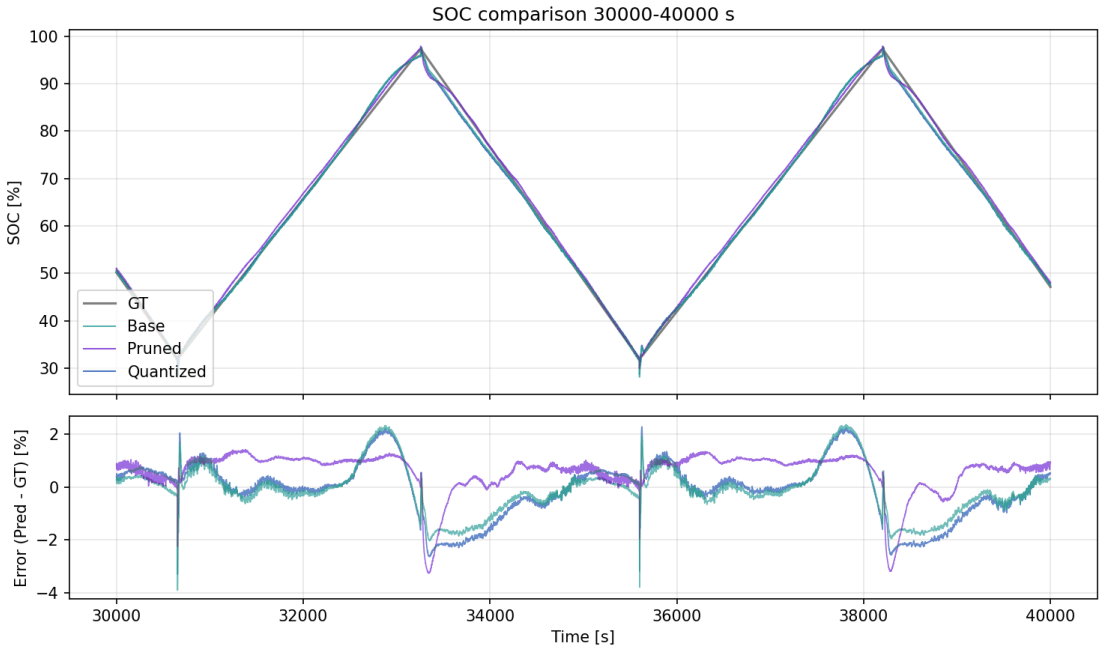

Figure 7.10: Detailed pulse-region analysis of embedded SOC streaming replay. The figure focuses on a highly dynamic load segment and shows how the three deployment variants capture fast transients and voltage-recovery behaviour under identical stateful inference conditions.

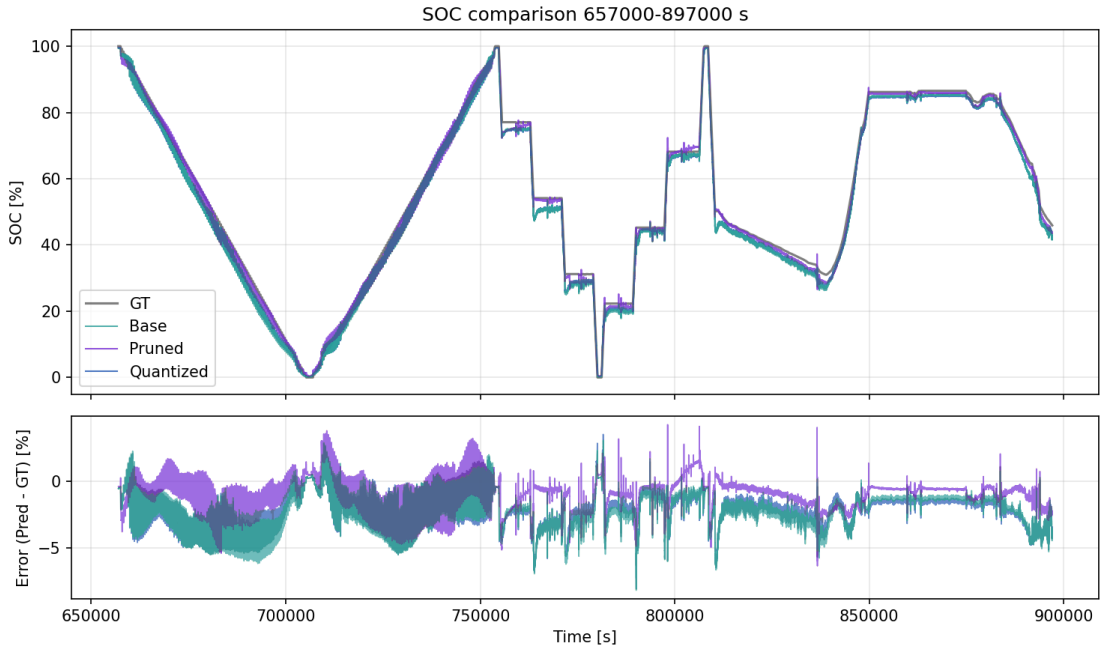

Figure 7.11: Detailed check-up-region analysis of embedded SOC streaming replay. The figure covers a representative diagnostic interval with long integration horizons and illustrates the stability of the stateful deployments during extended charge/discharge behaviour.

7.10 Resource Efficiency, Latency, and Trade-Off Summary

The KPI tables already show that pruning and quantization affect SOC and SOH differently, but Figures 7.12 and 7.13 condense the same trade-off visually across model size, memory, and timing. For each deployment, the model complexity can first be estimated from the parameter count under idealized storage assumptions, namely four bytes per FP32 weight and one byte per INT8 weight. The measured flash footprint is then taken from the STM32 linker output and is systematically larger because it also includes code, constant tables, and runtime support, a gap that is particularly visible for the larger SOH firmware. The RAM view in Figure 7.12 is based on the measured static data sections plus worst-case stack usage during streaming, and therefore approximates the true runtime requirement of the deployed kernels more realistically than a pure parameter-memory count.

For SOC, pruning reduces flash from 105,32 kB to 62,27 kB, reduces inference time from 1,40 ms to 0,80 ms, and reduces energy per inference from 0,70 mJ to

0,40 mJ, which corresponds to gains of roughly 41 to 43%. Quantization reduces the SOC flash footprint further to 52,48 kB, but increases pure inference time to 6,99 ms and energy per inference to 3,49 mJ because the software kernel has to reconstruct FP32 values from INT8 codes and scales. For SOH, pruning reduces flash from 335,00 kB to 182,41 kB, reduces inference time from 22,73 ms to 12,72 ms, and reduces energy per inference from 11,37 mJ to 6,36 mJ. Quantization reduces flash further to 138,00 kB, but raises inference time and energy to 29,21 ms and 14,60 mJ.

The latency distribution in Figure 7.13 reveals the same hierarchy. On-device SOC inference remains in the sub-10 ms regime even for the quantized kernel. The SOH updates require 12,72 ms for the pruned model, 22,73 ms for the dense FP32 model, and 29,21 ms for the INT8-weight deployment. Including UART overhead still leaves end-to-end latencies below roughly 20 ms for SOC and below roughly 50 ms for SOH, which is comfortably inside typical stationary-storage BMS timing limits.[2, 53] The model-size and latency plots therefore condense a clear pattern: quantization is the strongest lever for non-volatile memory, while pruning is the strongest lever for energy and inference time on the current hand-written STM32 implementation. To summarize this multi-objective trade-off more compactly, the chapter uses the composite utility score

$$U_v = \frac{1}{4}\left(\frac{\mathrm{MAE}_v}{\mathrm{MAE}_{\mathrm{Base}}} + \frac{F_v}{F_{\mathrm{Base}}} + \frac{R_v}{R_{\mathrm{Base}}} + \frac{E_v}{E_{\mathrm{Base}}}\right),$$

where MAE denotes the streaming mean absolute error, F the measured flash footprint, R the measured total RAM usage, and E the measured energy per inference. By construction, the dense baseline has $U = 1$, and values below one indicate an average improvement across these four objectives.

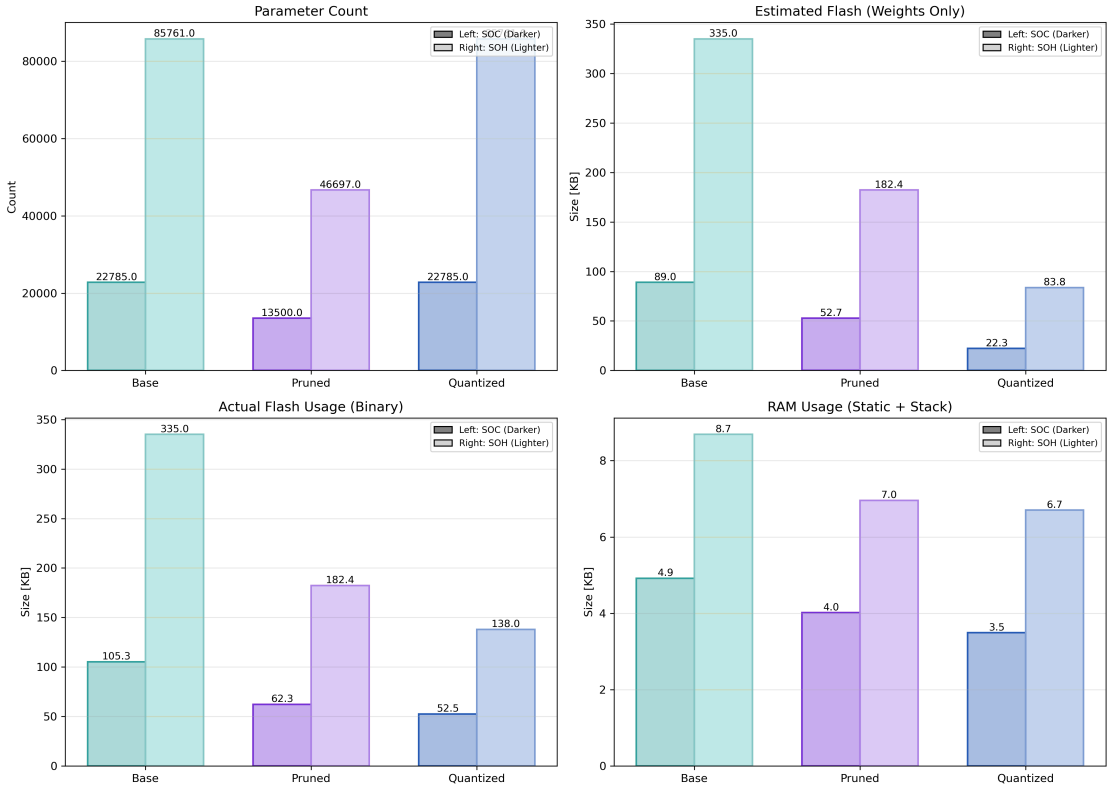

Figure 7.12: Resource landscape of the embedded SOC and SOH variants. The figure compares parameter count, idealized flash estimates from weight storage alone, measured flash on the STM32 firmware, and measured RAM usage. The separation between estimated and measured flash clarifies the additional code and runtime overhead of the embedded implementation.

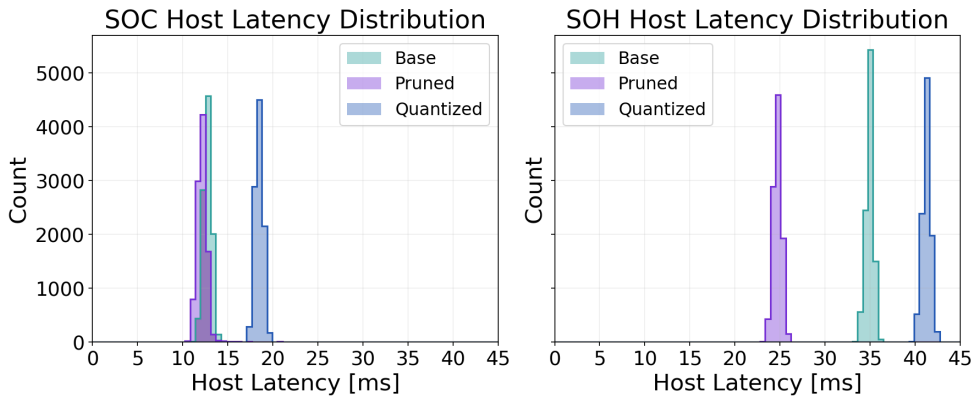

Figure 7.13: Latency distributions for SOC and SOH variants on the STM32H753ZI. The histograms summarize repeated on-device measurements and make visible the distinction between faster pruned kernels and slower quantized kernels with additional de-quantization overhead.

Table 7.3: Composite pruning/quantization summary. Values are reported as SOC/SOH. ΔMAE is given in percentage points of full scale, flash/RAM/energy are relative changes with respect to the corresponding dense baseline, and the utility score U follows the normalized average defined in the text, with lower values indicating a more favorable overall trade-off.

Variant	ΔMAE [pp]	ΔFlash [%]	ΔRAM [%]	ΔEnergy [%]	U
Pruned	−0.35 / +0.61	−41 / −46	−18 / −20	−43 / −44	0.71 / 0.91
Quantized	+0.10 / +0.56	−50 / −59	−20 / −23	+399 / +29	1.83 / 1.03

7.11 Discussion

7.11.1 Compression Behaviour of SOC and SOH Estimators

The results show that pruning and quantization do not have a uniform effect across the two estimation tasks. For SOC, the streaming trajectories and error distributions indicate that the single-layer 64-unit LSTM contains enough redundancy for compression to be applied without a loss of estimation quality. The pruned SOC model even reaches a slightly lower MAE and a narrower error distribution than the dense baseline. This behaviour is consistent with an over-parameterized recurrent model in which structured channel removal acts partly as regularization:

redundant hidden units are removed, while the remaining recurrent state still captures the Coulomb-counting and voltage-relaxation dynamics needed for full-stream tracking. The quantized SOC model follows the dense baseline almost indistinguishably in the long-horizon replay. Its deviations remain dominated by the numerical effects of de-quantization rather than by a visible change in the learned dynamics.

For SOH, the same compression methods lead to a more delicate trade-off. The SOH target evolves slowly, is tied to sparse capacity information from periodic diagnostic phases, and is evaluated after causal filtering of the raw network output. Under these conditions the 128-unit baseline uses its recurrent capacity more effectively than the SOC baseline, so removing hidden channels or representing weights with INT8 codes introduces a noticeable but still bounded accuracy penalty. The broader SOH error tails therefore do not indicate a failure of embedded deployment. They reflect the higher difficulty of estimating degradation from operational data and the fact that the model must infer long-term capacity fade without direct access to the capacity-test labels during deployment. If substantially tighter SOH margins are required, the results suggest that compression should be combined with additional temporal filtering, more task-specific architecture design, or broader training coverage rather than applied as a purely mechanical post-processing step.

7.11.2 Embedded Operating Points

From an engineering perspective, the three deployment variants represent different operating points rather than a simple accuracy ranking. The dense FP32 models provide the most direct and transparent tool flow, but they require the largest flash footprint and the longest on-device execution time. For SOC, structured pruning reduces the flash footprint from 105,32 kB to 62,27 kB, shortens the pure on-device inference time from 1,40 ms to 0,80 ms, and lowers the estimated energy per inference from 0,70 mJ to 0,40 mJ. The corresponding SOH changes are similarly relevant: flash decreases from 335,00 kB to 182,41 kB, inference time from 22,73 ms to 12,72 ms, and energy from 11,37 mJ to 6,36 mJ. Pruning is therefore the strongest lever for energy and runtime in the present firmware implementation.

Quantization, in contrast, is most attractive when non-volatile memory is the limiting resource. It reduces the SOC flash footprint to 52,48 kB and the SOH flash footprint to 138,00 kB, and it also provides the lowest reported RAM requirements in both cases. However, the manual STM32 kernels reconstruct effective FP32 weights from INT8 codes and scales during inference. This additional de-

quantization work changes the runtime balance. For SOC, the quantized variant increases inference time to 6,99 ms and energy to 3,49 mJ. For SOH, it reaches 29,21 ms and 14,60 mJ. The quantized models therefore solve a memory problem but do not automatically solve a compute- or energy-efficiency problem on this software implementation. A hardware target with stronger integer acceleration or a more optimized kernel could shift this balance, which is why the result should be read as a measured property of this complete MCU tool flow and not as a universal statement about INT8 inference.

7.11.3 Latency, Utility Score, and BMS Deployment

The distinction between communication-dominated system latency and pure on-device inference time is important for interpreting the results. The UART-based measurements report end-to-end latencies of 12,20 ms, 12,70 ms, and 18,48 ms for the pruned, dense, and quantized SOC variants. The corresponding SOH latencies are 24,69 ms, 34,88 ms, and 41,23 ms. The actual neural computation is substantially shorter. Tables 7.1 and 7.2 show that dense and pruned SOC inference remains below 1,5 ms, and even the quantized SOC kernel stays below 10 ms. The SOH updates require more time because of the larger hidden state and post-processing, with 12,72 ms for pruning, 22,73 ms for the dense model, and 29,21 ms for quantization. These margins are compatible with the low-rate sampling and diagnostic time scales typical of stationary-storage BMS operation.[2, 53]

The composite utility score in Table 7.3 makes the same point in a compact form. It is not intended to replace engineering constraints, but it shows how accuracy, flash, RAM, and energy move together when equal weighting is used. For SOC, the pruned model reaches $U = 0.71$ and is therefore the most balanced variant because it improves both accuracy and resource use. The quantized SOC variant reaches $U = 1.83$ despite its smaller flash footprint because the energy increase dominates the aggregate score. For SOH, both compressed variants remain near the baseline, with $U = 0.91$ for pruning and $U = 1.03$ for quantization. This indicates that the flash savings and the moderate accuracy/runtime penalties almost compensate each other. In practice, the more relevant design rule is therefore constraint-driven: define acceptable MAE or RMSE limits, impose hard flash and RAM budgets, and then select the feasible model with the best latency or energy behaviour for the target BMS.

7.11.4 Limitations and Future Extensions

The chapter also defines clear limits of the current embedded-AI study. The experimental evidence is based on one rich LFP ageing campaign, one cell chemistry, and one STM32H753ZI-class deployment target. The streaming replay is valuable because it preserves long temporal dependencies, but the disturbance robustness studied in Chapter 6 has not yet been repeated as live measurement-noise, sensor-drift, or fault-injection experiments directly on the MCU firmware and measurement chain. The current firmware also uses transparent hand-written kernels rather than an extensively optimized inference library, which is deliberate for auditability but influences the measured quantization overhead.

The next technical step is therefore to move from the evaluation-board setup toward the full custom BMS platform and eventually toward pack-level operation. In that setting, additional constraints such as communication bandwidth, safety monitoring, scheduler interaction, and sensing-chain imperfections will become part of the estimator problem. On the modelling side, the same deployment chain should be applied to additional cell chemistries, temperature ranges, and loading regimes, and the LSTM backbone should be compared against GRUs, compact transformer-style models, and hybrid observers that combine equivalent-circuit structure with neural residuals.[26, 109, 19, 84] A particularly relevant extension is combined pruning and quantization design: quantizing an already pruned model could combine the low weight storage of INT8 with the reduced state size and lower operation count of pruning, potentially making deployments feasible on still smaller or lower-power microcontrollers.

7.12 Conclusion and Outlook

This chapter establishes a complete deployment path from data-driven SOC and SOH estimators to an STM32-based battery-management controller. Starting from LSTM + MLP models trained on the LFP DoE Cycle Ageing Dataset, it derives dense, structured-pruned, and INT8-weight variants and evaluates them under the same stateful streaming replay with respect to accuracy, flash, RAM, latency, and energy. The contribution is therefore not only that the models can be executed on the MCU, but that the full route from training checkpoint to transparent C implementation and measured hardware behaviour is made explicit.

For SOC, the experiments show that strong compression is possible without sac-

rificing estimation quality. The pruned SOC model reduces recurrent state dimension, slightly improves the error distribution, lowers flash demand by about 41%, and lowers both inference time and energy by about 43% relative to the FP32 baseline. The quantized SOC deployment reduces flash even further and keeps the trajectory close to the dense model, but its software-based de-quantization increases inference time and energy. For SOH, the qualitative pattern is similar, but the accuracy-resource trade-off is tighter because degradation estimation is driven by slower and sparser information. Pruning reduces SOH flash, runtime, and energy by roughly 44 to 46%, while quantization gives the smallest memory footprint at the cost of higher computation time and energy than the pruned variant.

The resource and latency measurements demonstrate that all investigated deployments meet typical stationary-storage timing requirements with comfortable margin, and that co-locating SOC and SOH estimation on a single STM32H7-class controller is feasible within the measured memory budgets. At the same time, the chapter shows that embedded model selection cannot be based on floating-point accuracy alone. Architecture, compression method, firmware implementation, numerical representation, and target hardware jointly determine the useful estimator. This result connects directly to the dissertation-wide argument: deployment-oriented battery AI must be evaluated as a complete BMS implementation problem, not as an isolated offline prediction task.

Future work should transfer the same estimators and benchmark logic onto the custom BMS platform, include live disturbance injection and sensor-chain effects, and extend the evaluation to broader chemistries, temperatures, loading regimes, and pack sizes. The pruning and quantization workflow should also be expanded toward combined compression strategies and alternative recurrent or hybrid estimator architectures. These extensions will clarify how far the measured STM32 trade-offs generalize and where further algorithmic or firmware-level improvements are needed for next-generation resource-aware BMS.

Chapter 8

General Discussion and Cross-Paper Synthesis

8.1 Integrated Synthesis of the Research Questions

The global result of the dissertation can be read along the three research questions introduced in Chapter 1. The first question asks how comparatively simple neural models can be made time- and ageing-aware without immediately moving to large recurrent architectures. Chapter 5 answers this through representation rather than model complexity: cumulative-current information and lag-sequence history inject degradation-relevant temporal context into an otherwise memoryless MLP. The resulting average SOH error of about 1,10 % shows that the structure of the input can be as important as the structure of the network, especially for slowly evolving ageing targets.

The second question asks how estimator quality changes under realistic measurement and operating conditions. Chapter 6 shows that the answer cannot be obtained from a clean-condition MAE alone. Under nominal signals, the SOH-supported direct-measurement estimator reaches the lowest SOC MAE of about 0,24 %. Under a strong persistent current bias, however, it drifts to roughly 30 % error, while the voltage-corrected equivalent-circuit observer remains closer to 9 %. The data-driven recurrent estimator is not the universal winner either, but it is strongest under missing-sample stress and fastest in recovery after initialization mismatch. The main result is therefore not a fixed ranking of models, but the need to evaluate accuracy, disturbance sensitivity, and convergence behaviour as separate properties.

The third question asks what remains of neural estimation once embedded hardware is treated as part of the problem. Chapter 7 shows that stateful recurrent SOC and SOH estimators can be executed on the STM32 target with comfortable timing

margins, but that compression changes the evaluation. Structured pruning is the most balanced lever for the SOC estimator because it reduces flash, RAM, inference time, and energy simultaneously, with a composite utility score of $U = 0.71$. INT8 weight quantization reduces non-volatile memory more strongly, but the current firmware path reconstructs effective floating-point weights at runtime and therefore increases SOC energy consumption, reflected in $U = 1.83$. The embedded result is thus not merely that a neural estimator can be deployed, but that model architecture, compression method, firmware implementation, and measured hardware behaviour must be selected together.

Across these questions, the dissertation moves from representation design to disturbed-operation validation and finally to measured embedded execution. The custom BMS platform and the shared measurement chain provide the system context that makes this path practically meaningful. Without this layer, the work would remain an offline comparison of algorithms. With it, estimator design can be connected to sensing, firmware, memory, timing, and energy constraints.

8.2 Cross-Paper Findings Across BMS Requirements

Table 8.1: Synthesis of the contribution chapters with respect to the three requirement dimensions used throughout the dissertation.

Dissertation part	Performance	Hardware / embedded	Deployment logic
Neural ageing estimation	SOH accuracy through representation quality	indirect relevance	lag-sequence and feature design for compact models
Robustness benchmark	nominal accuracy, disturbance sensitivity, and recovery	embedded feasibility as boundary condition	matched multi-scenario evaluation across estimator classes
Embedded AI implementation	retained accuracy after compression	flash, RAM, latency, and energy measured on MCU	pruning, quantization, export, and firmware validation
BMS platform and datasets	measurement basis for evaluation	sensing chain and controller architecture	reproducible integration and estimator exchange

Table 8.1 shows why the three requirement dimensions introduced in Chapter 2 are not interchangeable. The performance dimension is not limited to mean error: it also includes robustness against disturbed measurements, recovery after state inconsistency, and the ability to retain useful behaviour when the input stream is incomplete or corrupted. This is why the robustness chapter is not just another accuracy experiment, but the point where nominal model ranking is deliberately challenged.

The hardware dimension changes the same comparison again. A model that is attractive in software must still fit the flash and RAM budget, meet the update-rate requirement, and avoid excessive energy demand. The embedded chapter therefore evaluates retained accuracy after pruning or quantization rather than unconstrained floating-point accuracy alone. This is a different criterion from the one used during

model training.

The deployment dimension links these two views. It includes the feature pipeline, recurrent-state handling, model export, firmware implementation, and the ability to exchange estimators on a reproducible measurement chain. The dissertation therefore treats deployment as a design requirement in its own right, not as a final formatting step after a model has already been selected.

8.3 Transferable Design Lessons

Four design lessons can be transferred from the individual chapters to BMS estimator design more generally.

The first lesson is *feature representation before model complexity*. Chapter 5 shows that neural networks do not have to become large merely because the battery state is history-dependent. With cumulative-current features and lag-sequence inputs, a compact MLP can exploit temporal and throughput-related information even though the estimator itself has no recurrent state. This principle also applies to recurrent models: an LSTM or GRU benefits from meaningful causal input channels just as much as a feedforward model does. Chapter 7 adds the complementary result that more expressive time-dependent models can still be made embedded-feasible when their architecture is structured well enough for pruning and quantization. The design message is therefore not MLP versus LSTM. It is to design the representation first, choose only as much model complexity as the task requires, and then optimize the selected architecture for deployment.

The second lesson is that *single-metric accuracy is not enough for deployment decisions*. A clean-condition MAE is a necessary reference, but it does not reveal why an estimator fails. Chapter 6 shows that there is no universally best estimator class under all tested conditions. Current bias, additive noise, missing samples, burst dropout, irregular sampling, voltage spikes, ADC quantization, and initialization mismatch each probe a different weakness of the estimation chain. The practical value of the robustness benchmark is therefore not a fixed leaderboard. It is a way to identify the weak points of a chosen estimator so that they can be addressed through feature design, retraining, observer tuning, filtering, reset logic, or fault handling before deployment.

The third lesson is that *compression must be matched to the active embedded constraint*. Pruning and quantization are useful for different reasons. Structured

pruning removes recurrent channels and downstream matrix rows, so it can reduce flash, RAM, inference time, and energy at the same time. Quantization reduces the weight storage more strongly, but the present STM32 implementation shows that it can increase inference time when INT8 weights have to be reconstructed through software-side scaling during every update. If flash memory is the limiting resource, quantization can be the decisive step. If latency or energy is limiting, pruning can be the more balanced option. The most promising direction is therefore not to treat pruning and quantization as alternatives, but to combine them: first reduce the state size and operation count, then quantize the remaining weights to reduce the persistent footprint.

The fourth lesson is that *deployment constraints have to shape model design from the beginning*. The estimator architecture, feature pipeline, firmware implementation, and target hardware are coupled. A very complex model may look attractive in an offline benchmark, but it is harder to export, optimize, validate, and maintain on a microcontroller than a compact or structurally regular model. Model design should therefore include the expected BMS target from the start: available sensor signals, sampling rate, memory budget, arithmetic support, update frequency, and firmware integration path. Validation should also not end with an offline test. The estimator should be replayed on the target controller or in a hardware-facing test stand where timing, memory, communication, and numerical behaviour can be measured together. The Custom BMS Platform is important for this reason: it provides the environment in which state-estimation software can be tested as deployable BMS functionality rather than only as a model file.

8.4 Position Within the State of Research and Practical Implications

Compared with much of the battery-estimation literature, the contribution of this dissertation is not only a new model or a single accuracy result. Many existing studies focus on one dimension at a time: a neural architecture under curated signals, an observer under a specific fault assumption, or an embedded implementation after the model has already been chosen. The present work connects these layers. It links representation design, disturbance-based benchmarking, compression, MCU measurement, and BMS hardware integration into one deployment-oriented evaluation chain.

This positioning matters because BMS hardware is becoming more capable, with stronger microcontrollers, embedded-AI support, and tighter interaction between sensing ICs and application processors. These developments make neural and hybrid estimators more realistic for BMS deployment, but they do not remove the need for rigorous validation. More capable hardware expands the design space, but it does not make a nominal MAE sufficient.

The practical implication is a clear design order. Estimator selection should begin with the target use case and the expected sensing conditions, not with an architecture preference. The feature representation should then be designed for the target state, the estimator should be tested under clean and disturbed scenarios, the compression strategy should be matched to the resource bottleneck, and the final model should be verified on the intended controller or an equivalent embedded path. The dissertation therefore contributes less a single winning estimator than a structured way to judge estimator classes under BMS-relevant conditions.

8.5 Limitations and Transferability

The results must be interpreted within the boundary conditions of the studies. The ageing-estimation chapter uses an NMC cyclic-ageing dataset, whereas the robustness and embedded chapters use an LFP design-of-experiments campaign. Their numerical results are therefore complementary rather than directly interchangeable. The robustness benchmark focuses on one representative ageing trajectory to isolate estimator-side behaviour, which improves interpretability but limits cross-cell statistics.

The embedded results are also platform-specific. They refer to STM32H7-class hardware, hand-written recurrent kernels, and the selected measurement setup for latency and energy. Other controllers, vendor toolchains, memory hierarchies, or fixed-point kernels could change the quantitative balance between pruning and quantization. Similarly, the custom BMS platform is a laboratory system rather than a field-deployed product, so its role is to demonstrate a reproducible validation chain rather than final operational qualification.

The transferable result is therefore the evaluation logic, not every numerical value. Estimator classes should be compared under matched nominal, disturbed, recovery, and embedded conditions, and the final judgement should be based on how the method behaves across this complete chain.

Chapter 9

Conclusion and Outlook

9.1 Overall Conclusion

This dissertation examined AI-based battery-state estimation from the perspective of a practical BMS rather than from the perspective of an isolated offline model. The work began with the observation that SOC and SOH estimators are only useful in operation if they satisfy more than one requirement at the same time. They must represent the relevant battery history, remain reliable under imperfect measurements, fit within embedded hardware limits, and be transferable into a reproducible firmware and validation chain. The central conclusion is therefore that deployment-oriented battery intelligence is not defined by the lowest clean-data error, but by the behaviour of the full estimator path from data representation to hardware execution.

The contribution chapters address this path step by step. The ageing-estimation study shows that simple models can become time-aware when the input representation contains physically meaningful history, as demonstrated by the MLP-based SOH estimator with cumulative-current and lag-sequence features. The robustness benchmark then shows why such accuracy claims have to be stress-tested under realistic measurement disturbances: bias, noise, missing samples, timing irregularities, voltage spikes, ADC quantization, and initialization mismatch can change the ranking of estimator classes and reveal weaknesses that clean-condition metrics hide. The embedded-AI study extends the argument to microcontroller execution and demonstrates that model quality after pruning, quantization, export, and on-device measurement is the relevant quantity for BMS deployment. The hardware chapter provides the system basis that makes this chain concrete by offering an adaptable Custom BMS Platform with accessible sensing, firmware partitioning, and estimator integration.

Taken together, the dissertation contributes a requirement-based way of evalu-

ating battery-state estimators. The purpose is not to declare one estimator family universally superior. Direct-measurement, model-based, hybrid, and data-driven approaches each have useful operating regions and failure modes. The contribution is instead to show how these methods can be compared in a BMS-relevant way: first by choosing a suitable representation, then by extending the evaluation beyond clean-condition accuracy to disturbance and recovery behaviour, then by selecting pruning, quantization, or their combination according to the actual hardware bottleneck, and finally by validating the result on the controller. This evaluation chain is the main outcome of the dissertation.

9.2 Remaining Gaps

The conclusions must be interpreted within the experimental scope of the work. The studies use two cell chemistries and two ageing datasets, namely an NMC cyclic-ageing campaign for the MLP-based SOH study and an LFP design-of-experiments campaign for the robustness and embedded-deployment studies. These datasets provide complementary evidence, but they do not form a single unified cross-chemistry benchmark. The robustness benchmark intentionally focuses on one representative ageing trajectory to isolate estimator-side behaviour, which limits the statistical breadth across cells. The embedded results are tied to STM32H7-class hardware, hand-written C kernels, and the specific measurement and replay workflow used in this dissertation.

Several open points therefore remain. The robustness scenarios have so far been evaluated mainly as controlled replay experiments rather than as live disturbance-injection tests on the complete Custom BMS Platform. The embedded deployment results confirm memory, timing, energy, and numerical behaviour for the investigated models, but they do not yet cover a broader family of recurrent, feedforward, transformer-style, and hybrid neural-equivalent-circuit estimators. The hardware platform itself is a compact laboratory system and not yet a field-scale BMS for larger packs. These limitations do not weaken the methodological conclusion, but they define where further validation is required before the quantitative results can be generalized.

9.3 Outlook

Future work should first broaden the empirical basis. The same evaluation chain should be applied to additional cells, chemistries, ageing states, temperature ranges, and operating profiles so that the observed relationships between accuracy, robustness, recovery, and compressibility can be tested beyond the present trajectories. This is especially important for separating general estimator behaviour from dataset-specific effects. A broader benchmark would also make it possible to quantify how much training coverage is required before data-driven estimators can be trusted outside the ageing and load conditions represented in the original data.

A second direction is to move the disturbance protocols closer to the embedded platform. Bias, noise, missing samples, timing irregularities, and spikes should be injected into the live firmware and measurement chain of the Custom BMS Platform rather than only into host-side replay data. This would close the loop between the robustness and embedded-deployment parts of the dissertation and would reveal interactions that may only appear when disturbed signals, compressed models, firmware scheduling, communication overhead, and controller timing occur together.

A third direction is architecture and compression co-design. The dissertation shows that pruning and quantization are not neutral post-processing steps and that their benefit depends on the architecture and on the firmware kernel. Future work should therefore compare MLPs, GRUs, LSTMs, compact transformer-style models, and neural-augmented equivalent-circuit models under the same accuracy, robustness, memory, latency, and energy criteria. A particularly relevant next step is the combined use of pruning and quantization, because pruning can reduce state size and operation count while quantization can reduce the remaining persistent weight memory. The goal would be to turn the qualitative design lesson of this dissertation into quantitative selection rules for embedded BMS estimator architectures.

Finally, the Custom BMS Platform should be extended from a compact laboratory system toward larger pack configurations and longer autonomous operation. This includes transfer to higher cell counts, pack-level thermal and electrical effects, longer cycling experiments, and eventually online or continual adaptation on the controller. Such adaptation would be the logical next step for long-lived decentralized and mobile systems: the estimator would not only be deployed once, but would remain maintainable as the battery ages and as the operating environment changes.

In this sense, the long-term objective remains a BMS in which AI-based state estimation is not an offline add-on, but an integrated, testable, and resource-aware part of the battery system itself.

Bibliography

[1] Florian Rzepka, Philipp Hematty, Mano Schmitz, and Julia Kowal. "Neural Network Architecture for Determining the Aging of Stationary Storage Systems in Smart Grids". In: *Energies* 16.17 (Aug. 22, 2023), p. 6103. DOI: 10.3390/en16176103. URL: https://www.mdpi.com/1996-1073/16/17/6103 (visited on 12/04/2025) (cited on pp. 3, 52).

[2] Gregory L. Plett. *Battery Management Systems: Volume 1: Battery Modeling*. Artech House Power Engineering and Power Electronics. Boston London: Artech House, 2015. 1 p. (cited on pp. 7, 11, 20, 101, 105).

[3] Long Zhou, Xin Lai, Bin Li, Yi Yao, Ming Yuan, Jiahui Weng, and Yuejiu Zheng. "State Estimation Models of Lithium-Ion Batteries for Battery Management System: Status, Challenges, and Future Trends". In: *Batteries* 9.2 (Feb. 13, 2023), p. 131. DOI: 10.3390/batteries9020131. URL: https://www.mdpi.com/2313-0105/9/2/131 (visited on 06/24/2026) (cited on p. 7).

[4] Ming Jiang, Dongjiang Li, Zonghua Li, Zhuo Chen, Qinshan Yan, Fu Lin, Cheng Yu, Bo Jiang, Xuezhe Wei, Wensheng Yan, and Yong Yang. "Advances in Battery State Estimation of Battery Management System in Electric Vehicles". In: *Journal of Power Sources* 612 (Aug. 2024), p. 234781. DOI: 10.1016/j.jpowsour.2024.234781. URL: https://linkinghub.elsevier.com/retrieve/pii/S037877532400733X (visited on 06/24/2026) (cited on pp. 7, 11).

[5] Xue Li, Jiuchun Jiang, Caiping Zhang, Le Yi Wang, and Linfeng Zheng. "Robustness of SOC Estimation Algorithms for EV Lithium-Ion Batteries against Modeling Errors and Measurement Noise". In: *Mathematical Problems in Engineering* 2015 (2015), pp. 1–14. DOI: 10.1155/2015/719490. URL: http://www.hindawi.com/journals/mpe/2015/719490/ (visited on 03/23/2026) (cited on pp. 8, 12, 45–47, 57).

[6] F. Naseri, C. Barbu, A.C.S. Jensen, P. Setoodeh, J.A. Romero Baena, N. Hevele, and E. Schaltz. "Toward Reliable Wireless BMS: Robust Battery State Estimation Under Noise and Uncertainty". In: *IEEE Transactions on Vehicular Technology* (2025), pp. 1–12. DOI: 10.1109/TVT.2025.3638613. URL: https://ieeexplore.ieee.org/document/11271354/ (visited on 03/23/2026) (cited on pp. 8, 12, 45, 47, 57).

[7] Franziska Berger, Dominik Joest, Elias Barbers, Katharina Quade, Ziheng Wu, Dirk Uwe Sauer, and Philipp Dechent. "Benchmarking Battery Management System Algorithms - Requirements, Scenarios and Validation for Automotive Applications". In: *eTransportation* 22 (Dec. 2024), p. 100355. DOI: 10.1016/j.etran.2024.100355. URL: https://linkinghub.elsevier.com/retrieve/pii/S2590116824000456 (visited on 03/23/2026) (cited on pp. 8, 45, 47, 55, 57).

[8] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, and Song Han. "Tiny Machine Learning: Progress and Futures". In: *IEEE Circuits and Systems Magazine* 23.3 (Aut. 2023), pp. 8–34. DOI: 10.1109/MCAS.2023.3302182. arXiv: 2403.19076 [cs]. URL: http://arxiv.org/abs/2403.19076 (visited on 12/02/2025) (cited on pp. 9, 13).

[9] Mina Naguib, Phillip Kollmeyer, Carlos Vidal, Josimar Duque, Oliver Gross, and Ali Emadi. "Microprocessor Execution Time and Memory Use for Battery State of Charge Estimation Algorithms". In: WCX SAE World Congress Experience. Detroit & Online, Michigan, United States, Mar. 29, 2022, pp. 2022-01–0697. DOI: 10.4271/2022-01-0697. URL: https://saemobilus.sae.org/papers/microprocessor-execution-time-memory-use-battery-state-charge-estimation-algorithms-2022-01-0697 (visited on 12/03/2025) (cited on pp. 9, 13, 93).

[10] Song Han, Jeff Pool, John Tran, and William J. Dally. *Learning Both Weights and Connections for Efficient Neural Networks*. Version 3. 2015. DOI: 10.48550/ARXIV.1506.0 URL: https://arxiv.org/abs/1506.02626 (visited on 12/02/2025). Pre-published (cited on pp. 9, 13, 90).

[11] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. Version 1. 2017. DOI: 10.48550/ARXIV.1712.05877. URL: https://a

rxiv.org/abs/1712.05877 (visited on 12/02/2025). Pre-published (cited on pp. 9, 13, 91).

[12] *BQ79616 16-Series Battery Monitor, Balancer, and Integrated Hardware Protector*. 2023. URL: https://www.ti.com/lit/ds/symlink/bq79616.pdf?ts=1775312350051&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FBQ79616 (cited on pp. 11, 55, 57).

[13] *10s Battery Pack Monitoring, Balancing, and Comprehensive Protection, 50-A Discharge Reference Design*. URL: https://www.ti.com/lit/ug/tiduar8c/tiduar8c.pdf (cited on pp. 11, 57).

[14] *Design Considerations of Current Sensing With BQ769x2 Family for Battery Management System*. URL: https://www.ti.com/lit/an/sluab10a/sluab10a.pdf?ts=1775276566199 (cited on pp. 11, 55, 57).

[15] *TLE4972 High Precision Coreless Current Sensor for Automotive Applications*. URL: https://www.infineon.com/assets/row/public/documents/24/49/infineon-tle4972-ae35d5-datasheet-en.pdf?fileId=8ac78c8c7c72fb9a017c7e1109e20a17 (cited on pp. 11, 55, 57).

[16] *TMCS1108 3% Functional Isolation Hall-Effect Current Sensor With ±100-V Working Voltage*. URL: https://www.ti.com/lit/ds/symlink/tmcs1108.pdf (cited on pp. 11, 55, 57).

[17] Prashant Shrivastava, P. Amritansh Naidu, Sakshi Sharma, Bijaya Ketan Panigrahi, and Akhil Garg. "Review on Technological Advancement of Lithium-Ion Battery States Estimation Methods for Electric Vehicle Applications". In: *Journal of Energy Storage* 64 (Aug. 2023), p. 107159. DOI: 10.1016/j.est.2023.107159. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X2300556X (visited on 01/14/2025) (cited on pp. 11, 45–47, 83).

[18] Zeeshan Ahmad Khan, Prashant Shrivastava, Syed Muhammad Amrr, Saad Mekhilef, Abdullah A. Algethami, Mehdi Seyedmahmoudian, and Alex Stojcevski. "A Comparative Study on Different Online State of Charge Estimation Algorithms for Lithium-Ion Batteries". In: *Sustainability* 14.12 (June 17, 2022), p. 7412. DOI: 10.3390/su14127412. URL: https://www.mdpi.com/2071-1050/14/12/7412 (visited on 01/14/2025) (cited on pp. 12, 46).

[19] Yizhao Gao, Gregory L. Plett, Guodong Fan, and Xi Zhang. "Enhanced State-of-Charge Estimation of LiFePO4 Batteries Using an Augmented Physics-Based Model". In: *Journal of Power Sources* 544 (Oct. 2022), p. 231889. DOI: 10.1016/j.jpowsour.2022.231889. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775322008758 (visited on 01/14/2025) (cited on pp. 12, 46, 48, 57, 83, 106).

[20] Feng Guo, Luis D. Couto, Grietus Mulder, Khiem Trad, Guangdi Hu, Odile Capron, and Keivan Haghverdi. "A Systematic Review of Electrochemical Model-Based Lithium-Ion Battery State Estimation in Battery Management Systems". In: *Journal of Energy Storage* 101 (Nov. 2024), p. 113850. DOI: 10.1016/j.est.2024.113850. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X24034364 (visited on 01/14/2025) (cited on pp. 12, 45, 46, 48, 83).

[21] Xiao Sun, Long Zuo, Mingkang Zhang, Yanzhi Su, Qiang Fu, and Jiahui Jiang. "Equivalent Circuit Models for Lithium-Ion Batteries: A Comprehensive Review". In: *Electronics* 15.9 (May 6, 2026), p. 1968. DOI: 10.3390/electronics15091968. URL: https://www.mdpi.com/2079-9292/15/9/1968 (visited on 06/24/2026) (cited on p. 12).

[22] M.S. Hossain Lipu, M.A. Hannan, Aini Hussain, Afida Ayob, Mohamad H.M. Saad, Tahia F. Karim, and Dickson N.T. How. "Data-Driven State of Charge Estimation of Lithium-Ion Batteries: Algorithms, Implementation Factors, Limitations and Future Trends". In: *Journal of Cleaner Production* 277 (Dec. 2020), p. 124110. DOI: 10.1016/j.jclepro.2020.124110. URL: https://linkinghub.elsevier.com/retrieve/pii/S095965262034155X (visited on 04/14/2026) (cited on pp. 12, 29).

[23] Feng Zhao, Yun Guo, and Baoming Chen. "A Review of Lithium-Ion Battery State of Charge Estimation Methods Based on Machine Learning". In: *World Electric Vehicle Journal* 15.4 (Mar. 25, 2024), p. 131. DOI: 10.3390/wevj15040131. URL: https://www.mdpi.com/2032-6653/15/4/131 (visited on 06/24/2026) (cited on p. 12).

[24] Davide Pio Laudani, Davide Milillo, Michele Quercio, Francesco Riganti Fulginei, and Lorenzo Sabino. "A Comprehensive Review of Equivalent Circuit Models and Neural Network Models for Battery Management Systems". In: *Batteries* 12.1 (Jan. 22, 2026), p. 37. DOI: 10.3390/batteries12010037.

URL: https://www.mdpi.com/2313-0105/12/1/37 (visited on 06/24/2026) (cited on p. 12).

[25] Rasel Ahmed, Md. Shaharia Hossen, Nusrat Tabassum Tithi, Humayra Khatun, Kamrul Hasan Manik, Juhi Jannat Mim, and Nayem Hossain. "Role of Deep Learning in Battery Management System (BMS) for Electric Vehicles – A Review". In: *Energy Reports* 15 (June 2026), p. 109028. DOI: 10.1016/j.egyr.2025.109028. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352484725009047 (visited on 06/24/2026) (cited on p. 12).

[26] Zachary C. Lipton, John Berkowitz, and Charles Elkan. *A Critical Review of Recurrent Neural Networks for Sequence Learning*. Version 4. 2015. DOI: 10.48550/ARXIV.1506.00019. URL: https://arxiv.org/abs/1506.00019 (visited on 12/02/2025). Pre-published (cited on pp. 12, 87, 106).

[27] Pratik Mondal, Divyakumar Bhavsar, Kanupriya Mittal, and Mayank Mittal. "Estimating State-of-Charge in Lithium-Ion Batteries Through Deep Learning Techniques: A Comparative Evaluation". In: *IEEE Access* 12 (2024), pp. 78773–78786. DOI: 10.1109/ACCESS.2024.3408220. URL: https://ieeexplore.ieee.org/document/10546296/ (visited on 03/23/2026) (cited on pp. 12, 46, 48, 51).

[28] Weihan Li, Neil Sengupta, Philipp Dechent, David Howey, Anuradha Annaswamy, and Dirk Uwe Sauer. "Online Capacity Estimation of Lithium-Ion Batteries with Deep Long Short-Term Memory Networks". In: *Journal of Power Sources* 482 (Jan. 2021), p. 228863. DOI: 10.1016/j.jpowsour.2020.228863. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775320311678 (visited on 01/14/2025) (cited on pp. 12, 52, 84).

[29] Wenjie Sun, Huan Xu, Bangyu Zhou, Yuanjun Guo, Yongbing Tang, Wenjiao Yao, and Zhile Yang. "State-of-Charge Estimation of Sodium-Ion Batteries: A Fusion Deep Learning Approach". In: *Journal of Energy Storage* 91 (June 2024), p. 111527. DOI: 10.1016/j.est.2024.111527. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X24011125 (visited on 01/13/2025) (cited on pp. 12, 83).

[30] Hongxu Chen, Ying Chen, Changzheng Sun, Liping Huo, Wenjun Zhang, Ping Shen, Lvwei Huang, Weiling Luan, and Haofeng Chen. "Towards Practical Data-Driven Battery State of Health Estimation: Advancements and Insights Targeting Real-World Data". In: *Journal of Energy Chemistry* 110

(Nov. 2025), pp. 657–680. DOI: 10.1016/j.jechem.2025.07.022. URL: https://linkinghub.elsevier.com/retrieve/pii/S2095495625005741 (visited on 06/24/2026) (cited on p. 12).

[31] Ruth Cordova-Cardenas, Daniel Amor, and Álvaro Gutiérrez. "Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems". In: *Electronics* 14.24 (Dec. 11, 2025), p. 4877. DOI: 10.3390/electronics14244877. URL: https://www.mdpi.com/2079-9292/14/24/4877 (visited on 06/24/2026) (cited on p. 13).

[32] Spyridon Giazitzis, Maciej Sakwa, Sonia Leva, Emanuele Ogliari, Susheel Badha, and Filippo Rosetti. "A Case Study of a Tiny Machine Learning Application for Battery State-of-Charge Estimation". In: *Electronics* 13.10 (May 16, 2024), p. 1964. DOI: 10.3390/electronics13101964. URL: https://www.mdpi.com/2079-9292/13/10/1964 (visited on 05/19/2025) (cited on pp. 13, 46, 47, 83).

[33] Giulia Crocioni, Danilo Pau, Jean-Michel Delorme, and Giambattista Gruosso. "Li-Ion Batteries Parameter Estimation With Tiny Neural Networks Embedded on Intelligent IoT Microcontrollers". In: *IEEE Access* 8 (2020), pp. 122135–122146. DOI: 10.1109/ACCESS.2020.3007046. URL: https://ieeexplore.ieee.org/document/9133084/ (visited on 05/19/2025) (cited on pp. 13, 83, 84).

[34] Danilo Pietro Pau and Alberto Aniballi. "Tiny Machine Learning Battery State-of-Charge Estimation Hardware Accelerated". In: *Applied Sciences* 14.14 (July 18, 2024), p. 6240. DOI: 10.3390/app14146240. URL: https://www.mdpi.com/2076-3417/14/14/6240 (visited on 05/19/2025) (cited on pp. 13, 46, 47, 83, 84).

[35] Muh-Tian Shiue, Yang-Chieh Ou, Chih-Feng Wu, Yi-Fong Wang, and Bing-Jun Liu. "Design and Implementation of a Decentralized Node-Level Battery Management System Chip Based on Deep Neural Network Algorithms". In: *Electronics* 15.2 (Jan. 9, 2026), p. 296. DOI: 10.3390/electronics15020296. URL: https://www.mdpi.com/2079-9292/15/2/296 (visited on 06/24/2026) (cited on p. 13).

[36] *STM32N6*. June 25, 2026. URL: https://www.st.com/en/microcontrollers-microprocessors/stm32n6-series.html (cited on p. 13).

[37] *MCX N Series Microcontrollers*. URL: https://www.nxp.com/products/processors-and-microcontrollers/arm-microcontrollers/general-purpose-mcus/mcx-arm-cortex-m/mcx-n-series-microcontrollers:MCX-N-SERIES (cited on p. 13).

[38] *Renesas Sets New MCU Performance Bar with 1-GHz RA8P1 Devices with AI Acceleration: RA8P1*. URL: https://www.renesas.com/en/about/newsroom/renesas-sets-new-mcu-performance-bar-1-ghz-ra8p1-devices-ai-acceleration (cited on p. 13).

[39] *Alif Semiconductor: Ensemble E1/E3/E5/E7*. URL: https://alifsemi.com/products/ensemble/ (cited on p. 13).

[40] *Introducing Our Highest-Performance Wireless SoCs and MCU: EFR32xG26 / PG26*. URL: https://www.silabs.com/wireless/efr32xg26 (cited on p. 13).

[41] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. *What Is the State of Neural Network Pruning?* Version 1. 2020. DOI: 10.48550/ARXIV.2003.03033. URL: https://arxiv.org/abs/2003.03033 (visited on 12/02/2025). Pre-published (cited on pp. 13, 90).

[42] Raghuraman Krishnamoorthi. *Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper*. Version 1. 2018. DOI: 10.48550/ARXIV.1806.08342. URL: https://arxiv.org/abs/1806.08342 (visited on 12/02/2025). Pre-published (cited on pp. 13, 91).

[43] Wei Wen, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. *Learning Structured Sparsity in Deep Neural Networks*. Version 4. 2016. DOI: 10.48550/ARXIV.1608.03665. URL: https://arxiv.org/abs/1608.03665 (visited on 12/02/2025). Pre-published (cited on pp. 13, 90).

[44] Wei Wen, Yuxiong He, Samyam Rajbhandari, Minjia Zhang, Wenhan Wang, Fang Liu, Bin Hu, Yiran Chen, and Hai Li. *Learning Intrinsic Sparse Structures within Long Short-Term Memory*. Version 7. 2017. DOI: 10.48550/ARXIV.1709.05027. URL: https://arxiv.org/abs/1709.05027 (visited on 12/08/2025). Pre-published (cited on p. 13).

[45] Sharan Narang, Erich Elsen, Gregory Diamos, and Shubho Sengupta. *Exploring Sparsity in Recurrent Neural Networks*. Version 2. 2017. DOI: 10.48550/ARXIV.1704.051 URL: https://arxiv.org/abs/1704.05119 (visited on 12/02/2025). Pre-published (cited on pp. 13, 90).

[46] Ekaterina Lobacheva, Nadezhda Chirkova, Alexander Markovich, and Dmitry Vetrov. “Structured Sparsification of Gated Recurrent Neural Networks”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 34.04 (Apr. 3, 2020), pp. 4989–4996. DOI: 10.1609/aaai.v34i04.5938. URL: https://ojs.aaai.org/index.php/AAAI/article/view/5938 (visited on 12/08/2025) (cited on pp. 13, 90).

[47] Jonas Schulze, Nils Strassenburg, and Tilmann Rabl. “PQ Bench: Benchmarking Pruning and Quantization Techniques”. In: *Proceedings of the Workshop on Data Management for End-to-End Machine Learning*. SIGMOD/PODS ’25: International Conference on Management of Data. Berlin Germany: ACM, June 22, 2025, pp. 1–6. DOI: 10.1145/3735654.3735940. URL: https://dl.acm.org/doi/10.1145/3735654.3735940 (visited on 12/05/2025) (cited on pp. 13, 91).

[48] Hamza A. Abushahla, Dara Varam, Ariel Justine N. Panopio, and Mohamed I. AlHajri. *Neural Network Quantization for Microcontrollers: A Comprehensive Survey of Methods, Platforms, and Applications*. Jan. 7, 2026. DOI: 10.48550/arXiv.2508.15008. arXiv: 2508.15008 [cs.LG]. URL: http://arxiv.org/abs/2508.15008 (visited on 06/24/2026). Pre-published (cited on p. 13).

[49] Florian Rzepka. *Design of Experiments for Cyclic Aging of LFP 18650 Cells: Impact of Charge/Discharge C-Rates, Depth of Discharge, and Temperature on Battery Degradation*. Technische Universität Berlin, Nov. 13, 2025. DOI: 10.14279/DEPOSITONCE-24800. URL: https://depositonce.tu-berlin.de/handle/11303/25972 (visited on 12/02/2025) (cited on pp. 16, 18, 62, 84).

[50] Anup Barai, Kotub Uddin, Matthieu Dubarry, Limhi Somerville, Andrew McGordon, Paul Jennings, and Ira Bloom. “A Comparison of Methodologies for the Non-Invasive Characterisation of Commercial Li-ion Cells”. In: *Progress in Energy and Combustion Science* 72 (May 2019), pp. 1–31. DOI: 10.1016/j.pecs.2019.01.001. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360128518300996 (visited on 04/14/2026) (cited on p. 16).

[51] Peter Kurzweil and Otto K. Dietlmeier. *Elektrochemische Speicher: Superkondensatoren, Batterien, Elektrolyse-Wasserstoff, Rechtliche Rahmenbedingungen*. Wiesbaden: Springer Fachmedien Wiesbaden, 2018. DOI: 10.1007/978-

3-658-21829-4. URL: http://link.springer.com/10.1007/978-3-658-21829-4 (visited on 04/14/2026) (cited on p. 16).

[52] Karl Siebertz, David Van Bebber, and Thomas Hochkirchen. *Statistische Versuchsplanung*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2017. DOI: 10.1007/978-3-662-55743-3. URL: http://link.springer.com/10.1007/978-3-662-55743-3 (visited on 12/03/2025) (cited on pp. 18, 84).

[53] M. Berecibar, I. Gandiaga, I. Villarreal, N. Omar, J. Van Mierlo, and P. Van Den Bossche. "Critical Review of State of Health Estimation Methods of Li-ion Batteries for Real Applications". In: *Renewable and Sustainable Energy Reviews* 56 (Apr. 2016), pp. 572–587. DOI: 10.1016/j.rser.2015.11.042. URL: https://linkinghub.elsevier.com/retrieve/pii/S1364032115013076 (visited on 12/02/2025) (cited on pp. 20, 83, 101, 105).

[54] Younes Sahri, Youcef Belkhier, Salah Tamalouzt, Nasim Ullah, Rabindra Nath Shaw, Md. Shahariar Chowdhury, and Kuaanan Techato. "Energy Management System for Hybrid PV/Wind/Battery/Fuel Cell in Microgrid-Based Hydrogen and Economical Hybrid Battery/Super Capacitor Energy Storage". In: *Energies* 14.18 (Sept. 11, 2021), p. 5722. DOI: 10.3390/en14185722. URL: https://www.mdpi.com/1996-1073/14/18/5722 (visited on 04/14/2026) (cited on p. 27).

[55] Mohamed S. Soliman, Youcef Belkhier, Nasim Ullah, Abdelyazid Achour, Yasser M. Alharbi, Ahmad Aziz Al Alahmadi, Habti Abeida, and Yahya Salameh Hassan Khraisat. "Supervisory Energy Management of a Hybrid Battery/PV/Tidal/Wind Sources Integrated in DC-microgrid Energy Storage System". In: *Energy Reports* 7 (Nov. 2021), pp. 7728–7740. DOI: 10.1016/j.egyr.2021.11.056. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352484721012014 (visited on 04/14/2026) (cited on p. 27).

[56] Bin Gou, Yan Xu, and Xue Feng. "State-of-Health Estimation and Remaining-Useful-Life Prediction for Lithium-Ion Battery Using a Hybrid Data-Driven Method". In: *IEEE Transactions on Vehicular Technology* 69.10 (Oct. 2020), pp. 10854–10867. DOI: 10.1109/TVT.2020.3014932. URL: https://ieeexplore.ieee.org/document/9162518/ (visited on 04/14/2026) (cited on p. 27).

[57] Tsuyoshi Oji, Yanglin Zhou, Song Ci, Feiyu Kang, Xi Chen, and Xiulan Liu. "Data-Driven Methods for Battery SOH Estimation: Survey and a Critical Analysis". In: *IEEE Access* 9 (2021), pp. 126903–126916. DOI: 10.1109/AC-

CESS.2021.3111927. URL: https://ieeexplore.ieee.org/document/9535528/ (visited on 04/14/2026) (cited on p. 27).

[58] Jiabo Li, Min Ye, Kangping Gao, Xinxin Xu, Meng Wei, and Shengjie Jiao. "Joint Estimation of State of Charge and State of Health for Lithium-ion Battery Based on Dual Adaptive Extended Kalman Filter". In: *International Journal of Energy Research* 45.9 (July 2021), pp. 13307–13322. DOI: 10.1002/er.6658. URL: https://onlinelibrary.wiley.com/doi/10.1002/er.6658 (visited on 04/14/2026) (cited on p. 28).

[59] Yalan Bi, Yilin Yin, and Song-Yul Choe. "Online State of Health and Aging Parameter Estimation Using a Physics-Based Life Model with a Particle Filter". In: *Journal of Power Sources* 476 (Nov. 2020), p. 228655. DOI: 10.1016/j.jpowsour.2020.228655. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775320309599 (visited on 04/14/2026) (cited on p. 28).

[60] Jinpeng Tian, Rui Xiong, and Quanqing Yu. "Fractional-Order Model-Based Incremental Capacity Analysis for Degradation State Recognition of Lithium-Ion Batteries". In: *IEEE Transactions on Industrial Electronics* 66.2 (Feb. 2019), pp. 1576–1584. DOI: 10.1109/TIE.2018.2798606. URL: https://ieeexplore.ieee.org/document/8270363/ (visited on 04/14/2026) (cited on p. 28).

[61] Fabian Ebert, Gerhard Sextl, Jorn Adermann, Christoph Reiter, and Markus Lienkamp. "Detection of Cell-Stack Inhomogeneities via Mechanical SOC and SOH Measurement". In: *2017 IEEE Transportation Electrification Conference and Expo (ITEC)*. 2017 IEEE Transportation Electrification Conference and Expo (ITEC). Chicago, IL, USA: IEEE, June 2017, pp. 545–549. DOI: 10.1109/ITEC.2017.7993329. URL: https://ieeexplore.ieee.org/document/7993329/ (visited on 04/14/2026) (cited on p. 28).

[62] Daniel Ioan Stroe, Vaclav Knap, and Erik Schaltz. "State-of-Health Estimation of Lithium-Ion Batteries Based on Partial Charging Voltage Profiles". In: *ECS Transactions* 85.13 (June 19, 2018), pp. 379–386. DOI: 10.1149/08513.0379ecst. URL: https://iopscience.iop.org/article/10.1149/08513.0379ecst (visited on 04/14/2026) (cited on p. 28).

[63] Daniel-Ioan Stroe and Erik Schaltz. "SOH Estimation of LMO/NMC-based Electric Vehicle Lithium-Ion Batteries Using the Incremental Capacity Analysis Technique". In: *2018 IEEE Energy Conversion Congress and Exposition (ECCE)*. 2018 IEEE Energy Conversion Congress and Exposition (ECCE).

Portland, OR, USA: IEEE, Sept. 2018, pp. 2720–2725. DOI: 10.1109/ECCE.2018.8557998. URL: https://ieeexplore.ieee.org/document/8557998/ (visited on 04/14/2026) (cited on p. 28).

[64] Yu Shi, Shakeel Ahmad, Qing Tong, Tuti M. Lim, Zhongbao Wei, Dongxu Ji, Chika M. Eze, and Jiyun Zhao. "The Optimization of State of Charge and State of Health Estimation for Lithium-ions Battery Using Combined Deep Learning and Kalman Filter Methods". In: *International Journal of Energy Research* 45.7 (June 10, 2021), pp. 11206–11230. DOI: 10.1002/er.6601. URL: https://onlinelibrary.wiley.com/doi/10.1002/er.6601 (visited on 04/14/2026) (cited on p. 28).

[65] Seongwoo Kim, Pyeong-Yeon Lee, Miyoung Lee, Jonghoon Kim, and Woonki Na. "Improved State-of-health Prediction Based on Auto-Regressive Integrated Moving Average with Exogenous Variables Model in Overcoming Battery Degradation-Dependent Internal Parameter Variation". In: *Journal of Energy Storage* 46 (Feb. 2022), p. 103888. DOI: 10.1016/j.est.2021.103888. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X2101553X (visited on 04/14/2026) (cited on p. 28).

[66] Wei Xiong, Yimin Mo, and Cong Yan. "Online State-of-Health Estimation for Second-Use Lithium-Ion Batteries Based on Weighted Least Squares Support Vector Machine". In: *IEEE Access* 9 (2021), pp. 1870–1881. DOI: 10.1109/ACCESS.2020.3026552. URL: https://ieeexplore.ieee.org/document/9311733/ (visited on 04/14/2026) (cited on p. 28).

[67] Xin Sui, Shan He, Alejandro Gismero, Remus Teodorescu, and Daniel-Ioan Stroe. "Robust Fuzzy Entropy-Based SOH Estimation for Different Lithium-Ion Battery Chemistries". In: *2022 IEEE Energy Conversion Congress and Exposition (ECCE)*. 2022 IEEE Energy Conversion Congress and Exposition (ECCE). Detroit, MI, USA: IEEE, Oct. 9, 2022, pp. 1–8. DOI: 10.1109/ECCE50734.2022.994779 URL: https://ieeexplore.ieee.org/document/9947792/ (visited on 04/14/2026) (cited on p. 28).

[68] Sungwoo Jo, Sunkyu Jung, and Taemoon Roh. "Battery State-of-Health Estimation Using Machine Learning and Preprocessing with Relative State-of-Charge". In: *Energies* 14.21 (Nov. 2, 2021), p. 7206. DOI: 10.3390/en14217206. URL: https://www.mdpi.com/1996-1073/14/21/7206 (visited on 04/14/2026) (cited on p. 28).

[69] Le Xu, Zhongwei Deng, Yi Xie, Xianke Lin, and Xiaosong Hu. "A Novel Hybrid Physics-Based and Data-Driven Approach for Degradation Trajectory Prediction in Li-Ion Batteries". In: *IEEE Transactions on Transportation Electrification* 9.2 (June 2023), pp. 2628–2644. DOI: 10.1109/TTE.2022.3212024. URL: https://ieeexplore.ieee.org/document/9911693/ (visited on 04/14/2026) (cited on p. 28).

[70] Hui Pang, Longxing Wu, Jiahao Liu, Xiaofei Liu, and Kai Liu. "Physics-Informed Neural Network Approach for Heat Generation Rate Estimation of Lithium-Ion Battery under Various Driving Conditions". In: *Journal of Energy Chemistry* 78 (Mar. 2023), pp. 1–12. DOI: 10.1016/j.jechem.2022.11.036. URL: https://linkinghub.elsevier.com/retrieve/pii/S2095495622006349 (visited on 04/14/2026) (cited on p. 28).

[71] Jungsoo Kim, Jungwook Yu, Minho Kim, Kwangrae Kim, and Soohee Han. "Estimation of Li-ion Battery State of Health Based on Multilayer Perceptron: As an EV Application". In: *IFAC-PapersOnLine* 51.28 (2018), pp. 392–397. DOI: 10.1016/j.ifacol.2018.11.734. URL: https://linkinghub.elsevier.com/retrieve/pii/S2405896318334542 (visited on 04/14/2026) (cited on p. 28).

[72] Hicham Chaoui, Chinemerem C. Ibe-Ekeocha, and Hamid Gualous. "Aging Prediction and State of Charge Estimation of a LiFePO 4 Battery Using Input Time-Delayed Neural Networks". In: *Electric Power Systems Research* 146 (May 2017), pp. 189–197. DOI: 10.1016/j.epsr.2017.01.032. URL: https://linkinghub.elsevier.com/retrieve/pii/S037877961730041X (visited on 04/14/2026) (cited on p. 28).

[73] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. "Statistical and Machine Learning Forecasting Methods: Concerns and Ways Forward". In: *PLOS ONE* 13.3 (Mar. 27, 2018). Ed. by Alejandro Raul Hernandez Montoya, e0194889. DOI: 10.1371/journal.pone.0194889. URL: https://dx.plos.org/10.1371/journal.pone.0194889 (visited on 04/14/2026) (cited on p. 29).

[74] Dickson N. T. How, M. A. Hannan, M. S. Hossain Lipu, and Pin Jern Ker. "State of Charge Estimation for Lithium-Ion Batteries Using Model-Based and Data-Driven Methods: A Review". In: *IEEE Access* 7 (2019), pp. 136116–136136. DOI: 10.1109/ACCESS.2019.2942213. URL: https://ie

eexplore.ieee.org/document/8843918/ (visited on 04/14/2026) (cited on p. 29).

[75] Daniel Kucevic, Benedikt Tepe, Stefan Englberger, Anupam Parlikar, Markus Mühlbauer, Oliver Bohlen, Andreas Jossen, and Holger Hesse. "Standard Battery Energy Storage System Profiles: Analysis of Various Applications for Stationary Energy Storage Systems Using a Holistic Simulation Framework". In: *Journal of Energy Storage* 28 (Apr. 2020), p. 101077. DOI: 10.1016/j.est.2019.101077. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X19309016 (visited on 12/15/2025) (cited on pp. 29, 84).

[76] M. Kassem, J. Bernard, R. Revel, S. Pélissier, F. Duclaud, and C. Delacourt. "Calendar Aging of a Graphite/LiFePO4 Cell". In: *Journal of Power Sources* 208 (June 2012), pp. 296–305. DOI: 10.1016/j.jpowsour.2012.02.068. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775312004284 (visited on 04/14/2026) (cited on p. 30).

[77] Fernando Puente León and HolgerBG Jäkel. *Signale Und Systeme:* DE GRUYTER, Sept. 25, 2015. DOI: 10.1515/9783110403862. URL: https://www.degruyterbrill.com/document/doi/10.1515/9783110403862/html (visited on 04/14/2026) (cited on p. 33).

[78] Tim Salimans and Diederik P. Kingma. *Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks*. Version 3. 2016. DOI: 10.48550/ARXIV.1602.07868. URL: https://arxiv.org/abs/1602.07868 (visited on 04/14/2026). Pre-published (cited on pp. 35, 36).

[79] Hans Weytjens, Enrico Lohmann, and Martin Kleinsteuber. "Cash Flow Prediction: MLP and LSTM Compared to ARIMA and Prophet". In: *Electronic Commerce Research* 21.2 (June 2021), pp. 371–391. DOI: 10.1007/s10660-019-09362-7. URL: https://link.springer.com/10.1007/s10660-019-09362-7 (visited on 04/14/2026) (cited on p. 36).

[80] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization". Version 4. In: (2016). DOI: 10.48550/ARXIV.1603.06560. URL: https://arxiv.org/abs/1603.06560 (visited on 04/14/2026) (cited on pp. 36, 39).

[81] Jiaqi Yao and Julia Kowal. "Towards a Smarter Battery Management System: A Critical Review on Deep Learning-Based State of Charge Estimation of Lithium-Ion Batteries". In: *Energy and AI* 21 (Sept. 2025), p. 100585. DOI: 10.1016/j.egyai.2025.100585. URL: https://linkinghub.elsevier.com/retrieve/pii/S266654682500117X (visited on 03/23/2026) (cited on pp. 45, 46, 48, 51).

[82] Yizhao Gao, Kailong Liu, Chong Zhu, Xi Zhang, and Dong Zhang. "Co-Estimation of State-of-Charge and State-of- Health for Lithium-Ion Batteries Using an Enhanced Electrochemical Model". In: *IEEE Transactions on Industrial Electronics* 69.3 (Mar. 2022), pp. 2684–2696. DOI: 10.1109/TIE.2021.3066946. URL: https://ieeexplore.ieee.org/document/9384220/ (visited on 01/14/2025) (cited on pp. 46, 83).

[83] Jiangwei Shen, Wensai Ma, Jian Xiong, Xing Shu, Yuanjian Zhang, Zheng Chen, and Yonggang Liu. "Alternative Combined Co-Estimation of State of Charge and Capacity for Lithium-Ion Batteries in Wide Temperature Scope". In: *Energy* 244 (Apr. 2022), p. 123236. DOI: 10.1016/j.energy.2022.123236. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544222001396 (visited on 01/14/2025) (cited on pp. 46, 83).

[84] Chunyu Wang, Naxin Cui, Zhongrui Cui, Haitao Yuan, and Chenghui Zhang. "Fusion Estimation of Lithium-Ion Battery State of Charge and State of Health Considering the Effect of Temperature". In: *Journal of Energy Storage* 53 (Sept. 2022), p. 105075. DOI: 10.1016/j.est.2022.105075. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X22010775 (visited on 01/14/2025) (cited on pp. 46, 57, 83, 84, 106).

[85] Yahia Mazzi, Hicham Ben Sassi, Ahmed Gaga, and Fatima Errahimi. "State of Charge Estimation of an Electric Vehicle's Battery Using Tiny Neural Network Embedded on Small Microcontroller Units". In: *International Journal of Energy Research* 46.6 (May 2022), pp. 8102–8119. DOI: 10.1002/er.7713. URL: https://onlinelibrary.wiley.com/doi/10.1002/er.7713 (visited on 05/19/2025) (cited on pp. 46, 47).

[86] M. Adib Kamali, Angela C. Caliwag, and Wansu Lim. "Novel SOH Estimation of Lithium-Ion Batteries for Real-Time Embedded Applications". In: *IEEE Embedded Systems Letters* 13.4 (Dec. 2021), pp. 206–209. DOI:

10.1109/LES.2021.3078443. URL: https://ieeexplore.ieee.org/document/9426956/ (visited on 05/19/2025) (cited on pp. 52, 84).

[87] Bin Gou, Yan Xu, and Xue Feng. "An Ensemble Learning-Based Data-Driven Method for Online State-of-Health Estimation of Lithium-Ion Batteries". In: *IEEE Transactions on Transportation Electrification* 7.2 (June 2021), pp. 422–436. DOI: 10.1109/TTE.2020.3029295. URL: https://ieeexplore.ieee.org/document/9216038/ (visited on 01/14/2025) (cited on p. 52).

[88] Orazio Aiello. "Electromagnetic Susceptibility of Battery Management Systems' ICs for Electric Vehicles: Experimental Study". In: *Electronics* 9.3 (Mar. 19, 2020), p. 510. DOI: 10.3390/electronics9030510. URL: https://www.mdpi.com/2079-9292/9/3/510 (visited on 04/04/2026) (cited on pp. 55, 57).

[89] Mengxi Liu, Daniel Geißler, Sizhen Bian, Bo Zhou, and Paul Lukowicz. *Assessing the Impact of Sampling Irregularity in Time Series Data: Human Activity Recognition As A Case Study*. Jan. 25, 2025. DOI: 10.48550/arXiv.2501.15330. arXiv: 2501.15330 [eess]. URL: http://arxiv.org/abs/2501.15330 (visited on 04/04/2026). Pre-published (cited on pp. 55, 57).

[90] Jiaqi Yao and Julia Kowal. "Towards a Smarter Battery Management System: A Critical Review on Deep Learning-Based State of Charge Estimation of Lithium-Ion Batteries". In: *Energy and AI* 21 (Sept. 2025), p. 100585. DOI: 10.1016/j.egyai.2025.100585. URL: https://linkinghub.elsevier.com/retrieve/pii/S266654682500117X (visited on 12/04/2025) (cited on p. 83).

[91] A. M. S. M. H. S. Attanayaka, J. P. Karunadasa, K. T. M. U. Hemapala, and Department of Electrical Engineering, University of Moratuwa, Moratuwa, Sri Lanka. "Estimation of State of Charge for Lithium-Ion Batteries - A Review". In: *AIMS Energy* 7.2 (2019), pp. 186–210. DOI: 10.3934/energy.2019.2.186. URL: http://www.aimspress.com/article/10.3934/energy.2019.2.186 (visited on 12/02/2025) (cited on p. 83).

[92] Wenjie Sun, Bangyu Zhou, Huan Xu, Wenjiao Yao, Yuanjun Guo, and Zhile Yang. "State of Charge Estimation of Sodium-Ion Batteries Based on N-BEATS Neural Network". In: *2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT)*. 2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT). Jiaxing, China: IEEE, Nov. 10, 2023, pp. 1395–

1401. DOI: 10.1109/ACAIT60137.2023.10528411. URL: https://ieeexplore.ieee.org/document/10528411/ (visited on 01/13/2025) (cited on p. 83).

[93] LEAP-RE. *MG FARM Smart Stand-Alone Micro-Grids as a Solution for Agriculture Farms Electrification*. URL: https://www.leap-re.eu/mg-farm/ (cited on p. 83).

[94] Microenergy. *MG-FARM Powering Sustainable Agriculture with Smart Micro-Grids*. URL: https://mg-farm.microenergy-consulting.com/ (cited on p. 83).

[95] A. Aqachtoul, K. Karam, A. Elamrani, Y. Alidrissi, M. Najib, A. Rochd, and M. Bakhouya. "MGFARM: An IoT and Edge-Driven Framework for Smart and Sustainable Agricultural Practices". In: *Connected Objects, Artificial Intelligence, Telecommunications and Electronics Engineering*. Ed. by Youssef Mejdoub, Abdelkebir Elamri, and Mustapha Kardouchi. Vol. 1584. Cham: Springer Nature Switzerland, 2026, pp. 419–425. DOI: 10.1007/978-3-032-01536-5_64. URL: https://link.springer.com/10.1007/978-3-032-01536-5_64 (visited on 12/03/2025) (cited on p. 83).

[96] Technische Universität Berlin. *MG Farm Smart Microgrids as a Solution for Agriculture Farms Electrification*. URL: https://www.tu.berlin/eet/forschung/projekte/mg-farm (cited on p. 83).

[97] JGNE. *JGCFR18650-1800mAh-3.2V Product Specification*. JGNE. URL: https://www.goldencellpower.com/product-item/18650-1800mah-3-2v-lifepo4-cells/ (cited on p. 84).

[98] Peter J. Huber and Elvezio M. Ronchetti. *Robust Statistics*. 1st ed. Wiley Series in Probability and Statistics. Wiley, Jan. 29, 2009. DOI: 10.1002/9780470434697. URL: https://onlinelibrary.wiley.com/doi/book/10.1002/9780470434697 (visited on 12/02/2025) (cited on p. 85).

[99] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Andreas Müller, Joel Nothman, Gilles Louppe, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. "Scikit-Learn: Machine Learning in Python". Version 4. In: (2012). DOI: 10.48550/ARXIV.1201.0490. URL: https://arxiv.org/abs/1201.0490 (visited on 12/02/2025) (cited on p. 85).

[100] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Dec. 3, 2019. DOI: 10.48550/arXiv.1912.01703. arXiv: 1912.01703 [cs]. URL: http://arxiv.org/abs/1912.01703 (visited on 12/02/2025). Pre-published (cited on p. 87).

[101] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. Version 3. 2017. DOI: 10.48550/ARXIV.1711.05101. URL: https://arxiv.org/abs/1711.05101 (visited on 12/02/2025). Pre-published (cited on p. 87).

[102] Ilya Loshchilov and Frank Hutter. *SGDR: Stochastic Gradient Descent with Warm Restarts*. Version 5. 2016. DOI: 10.48550/ARXIV.1608.03983. URL: https://arxiv.org/abs/1608.03983 (visited on 12/02/2025). Pre-published (cited on p. 87).

[103] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. *Mixed Precision Training*. Version 3. 2017. DOI: 10.48550/ARXIV.1710.03740. URL: https://arxiv.org/abs/1710.03740 (visited on 12/02/2025). Pre-published (cited on p. 87).

[104] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. *On the Difficulty of Training Recurrent Neural Networks*. Version 2. 2012. DOI: 10.48550/ARXIV.1211.5063. URL: https://arxiv.org/abs/1211.5063 (visited on 12/02/2025). Pre-published (cited on p. 87).

[105] *ONNX Runtime: High-Performance Inference Engine*. URL: https://onnxruntime.ai/ (cited on p. 92).

[106] *NVIDIA TensorRT: High-Performance Deep Learning Inference*. URL: https://developer.nvidia.com/tensorrt (cited on p. 92).

[107] *X-CUBE-AI: STM32Cube Expansion Package for Artificial Intelligence*. URL: https://www.st.com/en/embedded-software/x-cube-ai.html (cited on p. 92).

[108] *STM32 Nucleo-144 Boards*. STMicroelectronics, Dec. 2, 2025. URL: https://www.st.com/resource/en/data_brief/nucleo-h753zi.pdf (visited on 12/02/2025) (cited on p. 93).

[109] S. Singh and P.R. Budarapu. "Deep Machine Learning Approaches for Battery Health Monitoring". In: *Energy* 300 (Aug. 2024), p. 131540. DOI: 10.1016/j.energy.2024.1 URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544224013136 (visited on 01/13/2025) (cited on p. 106).

Appendix

Supplementary Material A

This appendix contains supplementary tables, additional hardware documentation, experiment matrices, and publication-specific material that would interrupt the flow of the main text if it were placed directly in the core chapters.

A.1 Supplementary Robustness-Benchmark Result Tables

The following tables collect the numerical robustness-benchmark values used in Chapter 6. Unless stated otherwise, SOC error metrics are reported on the normalized 0 to 1 SOC scale. Multiplying by 100 gives percentage points.

Table A.1: Baseline performance metrics for the selected robustness-benchmark trajectory. Values are reported on the normalized 0 to 1 SOC scale.

Class	MAE	RMSE	P95 error	Max error	Bias
DM	0.0311	0.0397	0.0800	1.0000	0.0311
HDM	0.0024	0.0061	0.0094	1.0000	0.0022
HECM	0.0090	0.0127	0.0241	1.0000	-0.0029
DD	0.0100	0.0143	0.0336	0.0686	-0.0062

Table A.2: Cross-scenario global performance metrics used in the Chapter 6 result discussion. Values are reported on the normalized 0 to 1 SOC scale.

Scenario	Class	MAE	ΔMAE	RMSE	ΔRMSE	P95 error
Current noise (high)	DM	0.0310	-0.0001	0.0394	-0.0002	0.0795
Current bias	DM	0.3143	0.2833	0.3683	0.3286	0.6670
Initial SOC error	DM	0.0322	0.0011	0.0413	0.0017	0.0833
Irregular sampling	DM	0.0309	-0.0001	0.0396	-0.0001	0.0799
Burst dropout	DM	0.0332	0.0022	0.0487	0.0090	0.0819
Missing samples	DM	0.0386	0.0076	0.0476	0.0079	0.0926
Voltage spikes	DM	0.0312	0.0001	0.0398	0.0001	0.0801

Scenario	Class	MAE	ΔMAE	RMSE	ΔRMSE	P95 error
Temperature noise	DM	0.0311	0.0000	0.0397	0.0000	0.0800
Voltage noise	DM	0.0524	0.0213	0.0648	0.0252	0.1228
Current noise (high)	HDM	0.0044	0.0019	0.0074	0.0013	0.0120
Current bias	HDM	0.3068	0.3043	0.3638	0.3577	0.6668
Initial SOC error	HDM	0.0036	0.0011	0.0136	0.0075	0.0101
Irregular sampling	HDM	0.0027	0.0003	0.0061	0.0001	0.0095
Burst dropout	HDM	0.0051	0.0027	0.0325	0.0265	0.0098
Missing samples	HDM	0.0103	0.0079	0.0129	0.0069	0.0221
Voltage spikes	HDM	0.0062	0.0038	0.0140	0.0079	0.0328
Temperature noise	HDM	0.0067	0.0043	0.0110	0.0049	0.0210
Voltage noise	HDM	0.0751	0.0727	0.0908	0.0847	0.1578
Current noise (high)	HECM	0.0165	0.0075	0.0280	0.0154	0.0701
Current bias	HECM	0.0932	0.0842	0.1303	0.1176	0.2954
Initial SOC error	HECM	0.0091	0.0001	0.0143	0.0016	0.0241
Irregular sampling	HECM	0.0095	0.0004	0.0131	0.0004	0.0248
Burst dropout	HECM	0.0104	0.0014	0.0228	0.0101	0.0246
Missing samples	HECM	0.0119	0.0029	0.0163	0.0036	0.0329
Voltage spikes	HECM	0.0090	0.0000	0.0127	0.0000	0.0241
Temperature noise	HECM	0.0090	0.0000	0.0127	0.0000	0.0241
Voltage noise	HECM	0.0090	0.0000	0.0127	0.0000	0.0241
Current noise (high)	DD	0.0109	0.0009	0.0151	0.0008	0.0345
Current bias	DD	0.2859	0.2759	0.3426	0.3283	0.6367
Initial SOC error	DD	0.0110	0.0010	0.0175	0.0032	0.0355
Irregular sampling	DD	0.0105	0.0005	0.0144	0.0001	0.0330
Burst dropout	DD	0.0122	0.0022	0.0316	0.0173	0.0348
Missing samples	DD	0.0117	0.0016	0.0149	0.0006	0.0302
Voltage spikes	DD	0.0144	0.0044	0.0213	0.0070	0.0474
Temperature noise	DD	0.0181	0.0081	0.0223	0.0080	0.0451
Voltage noise	DD	0.0698	0.0597	0.0839	0.0696	0.1501

Table A.3: Local behavior and recovery metrics used in the Chapter 6 result discussion. Thresholds are reported on the normalized 0 to 1 SOC scale where applicable.

Class	Scenario	Local metric	Value	Threshold
DM	Initial SOC error	Recovery time to strict band [h]	1.1658	0.0800
DM	Initial SOC error	Recovery time to fair band [h]	0.2500	0.1599
HDM	Initial SOC error	Recovery time to strict band [h]	n.a.	0.0094
HDM	Initial SOC error	Recovery time to fair band [h]	n.a.	0.0200
HECM	Initial SOC error	Recovery time to strict band [h]	2.3693	0.0241
HECM	Initial SOC error	Recovery time to fair band [h]	1.1564	0.0482
DD	Initial SOC error	Recovery time to strict band [h]	1.0464	0.0336
DD	Initial SOC error	Recovery time to fair band [h]	1.0014	0.0672
DM	Burst dropout	Recovery time after burst dropout [h]	27.3511	0.0216
HDM	Burst dropout	Recovery time after burst dropout [h]	27.4093	0.0008

Class	Scenario	Local metric	Value	Threshold
HECM	Burst dropout	Recovery time after burst dropout [h]	28.5611	0.0095
DD	Burst dropout	Recovery time after burst dropout [h]	27.4351	0.0070
DM	Voltage spikes	Median spike recovery time [s]	0.0000	0.0800
HDM	Voltage spikes	Median spike recovery time [s]	0.0000	0.0094
HECM	Voltage spikes	Median spike recovery time [s]	0.0000	0.0241
DD	Voltage spikes	Median spike recovery time [s]	0.0000	0.0336
DM	Current noise (high)	Late minus early excess rolling MAE	-0.0013	n.a.
DM	Current noise (high)	Mean excess rolling MAE	-0.0001	n.a.
HDM	Current noise (high)	Late minus early excess rolling MAE	-0.0011	n.a.
HDM	Current noise (high)	Mean excess rolling MAE	0.0019	n.a.
HECM	Current noise (high)	Late minus early excess rolling MAE	0.0038	n.a.
HECM	Current noise (high)	Mean excess rolling MAE	0.0073	n.a.
DD	Current noise (high)	Late minus early excess rolling MAE	-0.0002	n.a.
DD	Current noise (high)	Mean excess rolling MAE	0.0009	n.a.

Table A.4: Scenario overview linking disturbance configuration, primary evidence type, and main Chapter 6 figure. This table clarifies which scenarios are interpreted predominantly through global metrics and which require additional local evidence.

Scenario	Configuration	Primary evidence	Main figure(s)
ADC quantization	Steps of 0,01 A, 0,005 V, and 0,5 °C	Global ΔMAE. Detailed values in Appendix Table A.5	Figs. 6.13, A.1
Current bias	Constant current offset in the compact matrix, plus dedicated 0.5%/1.5%/3.0% sweep of $\|I\|_{\max}$	Global MAE degradation across the sweep	Fig. 6.6
Current noise	Compact matrix with $\sigma_I = 0{,}10$ A and local detail sweep up to $\sigma_I = 0{,}20$ A	Global ΔMAE plus local output-variability and rolling-MAE evidence	Fig. 6.7

Scenario	Configuration	Primary evidence	Main figure(s)
Voltage noise	Additive Gaussian noise with $\sigma_U = 0{,}01\,\mathrm{V}$	Global ΔMAE	Fig. 6.13
Temperature noise	Additive Gaussian noise with $\sigma_T = 1{,}0\,^\circ\mathrm{C}$	Global ΔMAE	Fig. 6.13
Initial SOC error	Additive initial mismatch of −0.10 SOC, corresponding to 10 percentage points. DD via online Q_c offset	Local recovery to strict and fair baseline bands. Global MAE is secondary	Fig. 6.8
Missing samples	Every 50th sample frozen on the nominal 1 s grid	Global ΔMAE	Figs. 6.9, 6.13
Irregular sampling	Uniform sampling-time jitter of ±0,1 s around the nominal 1 s grid	Global ΔMAE	Figs. 6.9, 6.13
Burst dropout	One central frozen gap of 3600 s	Global ΔMAE plus local transition and long-horizon recovery	Figs. 6.9, 6.10, 6.11
Voltage spikes	One ±0,20 V single-sample spike every 1000 samples	Global ΔMAE plus event-wise P95 of post-spike peak excess error	Figs. 6.12, 6.13

Table A.5: ADC-quantization results for the four estimator classes. Values are reported on the normalized 0 to 1 SOC scale.

Class	Baseline MAE	ADC MAE	ΔMAE
DM	0.0311	0.0313	+0.0002
HDM	0.0024	0.0029	+0.0005
HECM	0.0090	0.0087	-0.0003
DD	0.0100	0.0098	-0.0002

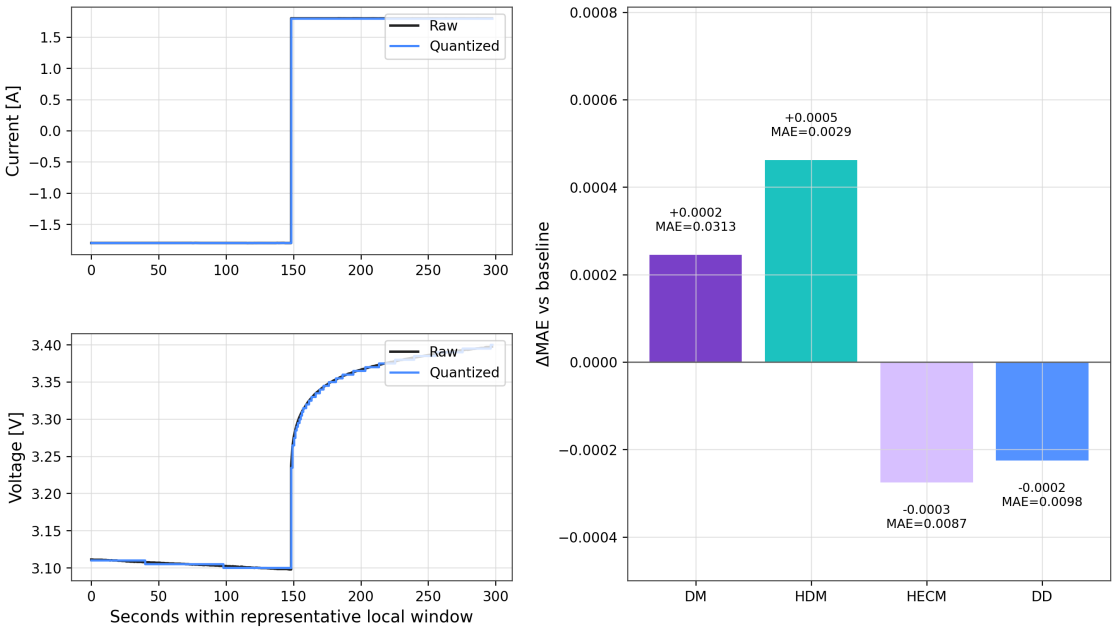

Figure A.1: ADC-quantization results for the robustness benchmark. The left panels illustrate representative current and voltage quantization in a local window, and the right panel summarizes the resulting global ΔMAE relative to the baseline case.